Predicting hydropower unit commitment in Tokke–Vinje: from feature engineering to a first-principles optimiser

TDT4173 Modern Machine Learning in Practice — SINTEF task — Group 300

Anders Bærheim (student ID 114407)
Peter Klivnoy (student ID 114439)

Kaggle team: [300]

November 2026

Abstract

We predict, for every hour of a one-week scheduling case and each of the 14 generating units of the Tokke–Vinje hydropower system, the probability that SINTEF's SHOP optimiser commits the unit. The work followed three tracks. *Track A* engineered features for gradient-boosted trees — economic spreads between price and water value, terminal water values, a count model over hours-on per unit-week, hourly block structure, a long (case, hour, unit) format — reaching 0.9638 micro ROC-AUC in walk-forward validation and 0.98058 on the public leaderboard. *Track B* starts from the observation that the labels are the output of a known optimisation problem whose inputs we hold in full: we rebuilt SHOP's weekly problem as a linear programme from the SHOP-ready topology file and use its solution *only as features* for LightGBM, which scores 0.9912 in walk-forward validation, improving every fold and all 14 units over Track A, and 0.99522 on the public leaderboard. *Track C* trains a sequence neural network — temporal convolution, a bidirectional GRU along each unit's week and attention across units — on 83 base features and 16 closed-form physics features derived from the generator efficiency curves, with no solver; it scores **0.99013** on the public leaderboard. Tracks B and C are our two selected predictions. Four predictor types compared on identical features show that Track B's gain comes from the optimiser features rather than from the learner.

Where to find the graded parts

asked for	where
names, student IDs, Kaggle team	title block above; first cell of both short notebooks
exploratory data analysis	Section 2, one subsection per activity in the course list: search domain knowledge (2.1), understand how the data was generated (2.2), check if the data is intuitive (2.3), explore individual features (2.4), explore pairs and groups of features (2.5), clean up features (2.6)
feature engineering	Section 4: feature extraction (4.1, 4.2, 4.3) and feature selection (4.4)
two or more types of predictors	Section 5: logistic regression, random forest, XGBoost and LightGBM on identical features and folds (5.1); a sequence neural network (5.4); every model we trained, helpful or not (5.2)
model interpretation	Section 7: gain importance, TreeSHAP contributions, ablation, calibration of the LP feature, one week explained
hyperparameter values in the notebooks	Section 6: every value and how it was obtained
the two selected predictions	Section 8; reproducibility in Appendix A
all steps, chronologically	Appendix B: the full lab log (37 entries)

Contents

1 The task and the data **4**

2 Exploratory data analysis **4**
2.1 Search domain knowledge . . . 4
2.2 Understand how the data was generated . . . 6
2.3 Check if the data is intuitive . . . 7
2.4 Explore individual features . . . 8
2.5 Explore pairs and groups of features . . . 9
2.6 Clean up features . . . 12
2.7 Training versus test period . . . 12
2.8 What the analysis implied for modelling . . . 13

3 Validation methodology **13**

4 Feature engineering **13**
4.1 Feature extraction I: economic and structural features (Track A) . . . 13
4.2 Feature extraction II: a first-principles optimiser as a feature generator (Track B) 15
4.3 Feature extraction III: closed-form physics features for a neural network (Track C) 16
4.4 Feature selection . . . 17
4.5 Inputs used by each final model . . . 18

5 Predictors: models and algorithms **18**
5.1 Four predictor types on identical features . . . 18
5.2 Every model we trained, helpful or not . . . 19
5.3 Track B: LightGBM on LP features . . . 20
5.4 Track C: a sequence neural network . . . 21

6 Hyperparameters: how the values in the notebooks were obtained **22**
6.1 Track A: XGBoost (first submission, not selected) . . . 22
6.2 Short notebook 1: LightGBM on LP features (Track B) . . . 23
6.3 Short notebook 2: UCNet (Track C) . . . 24

7 Model interpretation **26**
7.1 Track A: what the trees learned 26
7.2 Track B: the model leans on the optimiser 26
7.3 Track C: what the physics features contribute 29

8 The two selected predictions **30**

9 Discussion **30**

A Reproducibility **30**

B Full lab log **31**

1 The task and the data

Short-term hydropower scheduling decides, hour by hour, which generating units run and at what level. SINTEF's SHOP software solves this as a mixed-integer optimisation problem; the InterOpt project asks whether a machine-learning model can predict SHOP's on/off (unit-commitment) decisions from the inputs, to speed up or warm-start the optimiser.

The Tokke–Vinje watercourse has 17 reservoirs, 9 plants and 14 units (1,000 MW installed). Each *case* is a 168-hour week starting at 00:00 UTC; cases start every day, so consecutive cases overlap by 144 hours. There are 2,922 labelled cases (2015-01-01 to 2022-12-31) and 725 test cases (2023-01-01 to 2024-12-25). The model must output, for each test case, unit and hour, a value in $[0, 1]$: the probability that the unit is on. The metric is micro-averaged ROC-AUC over every (case, hour, unit) cell — 6.9M labelled cells and 1.7M test cells — so probabilities must be comparable across units as well as ordered within each unit-week.

file	resolution, coverage	content
core		
`Historical_day_ahead_price`	hourly, 2015–2025	day-ahead price, EUR/MWh
`Historical_inflow`	hourly, 1958–2025-10	inflow to the 17 reservoirs and one river, m^3/s
`Historical_volume`	daily, 2015–2025-01-01	storage at the start of each day, Mm^3
`Synthetic_water_value`	daily, 2015–2025-01-01	marginal water value per reservoir, EUR/MWh
`Unit_commitment_decisions`	per case, 2015–2022	labels: on/off for 14 units × 168 hours; SHOP solve time
extended (optional)		
`Constraint_min_volume`	hourly, 2015–2025	minimum storage (3 reservoirs), Mm^3
`Constraint_min_flow`	hourly, 2015–2025	minimum flow (6 rivers), m^3/s
`Tokke_Vinje_topology.yaml`	static	SHOP-ready system: reservoirs, rivers, tunnels, plants, generators
`Tokke_topology.pdf`	static	diagram of the watercourse (Figure 1)

Table 1: The data. All feature files cover the whole test period; labels stop at 2022-12-31.

Three tracks. The work ran in three tracks, and the report follows them through every section. *Track A* (Peter Klivnoy; logs 001–036) engineered features for gradient-boosted trees and produced our first submission (0.98058). *Track B* (Peter Klivnoy; log 037) rebuilt the optimisation problem that generated the labels as a linear programme and uses its solution as features for LightGBM. *Track C* (Anders Bærheim) built closed-form physics features from the generator curves and a sequence neural network over each case. Tracks B and C are the two selected predictions (Short notebooks 1 and 2).

2 Exploratory data analysis

Five EDA sessions (logs 001, 002, 008, 009, 011; Appendix B) shaped every later decision. This section is organised by the six activities in the course's EDA list; we did all six.

2.1 Search domain knowledge

We read the task statement, the dataset documentation and the SHOP-format topology file (whose attributes are defined in the SHOP documentation), and used the standard economics of hydropower scheduling. The facts that mattered:

- **Water value is an opportunity cost.** Water released now cannot be sold later. A unit is worth running when the price exceeds the value of the water it uses, the reservoir's *water value* (EUR/MWh). The dispatch condition is therefore the *spread* between price and water value, not the price alone.
- **The week has a boundary condition.** A weekly schedule values the water left at the end of the week at the *terminal* water value; the documentation states which day's value that is.
- **Starting a unit costs money** (start/stop costs in the topology file), so units run in contiguous blocks rather than hour by hour.
- **The system is a cascade.** Water released upstream flows into downstream reservoirs and plants (Figure 1); a plant's decision depends on the reservoirs above and below it.
- **Identical units share a penstock** at Tokke (four units) and Vinje (three), so which one starts first is a matter of hydraulic losses and tie-breaking, not of different machines.
- **Norwegian inflow is dominated by snowmelt** in May–June; reservoirs are drawn down through winter and refilled in summer.

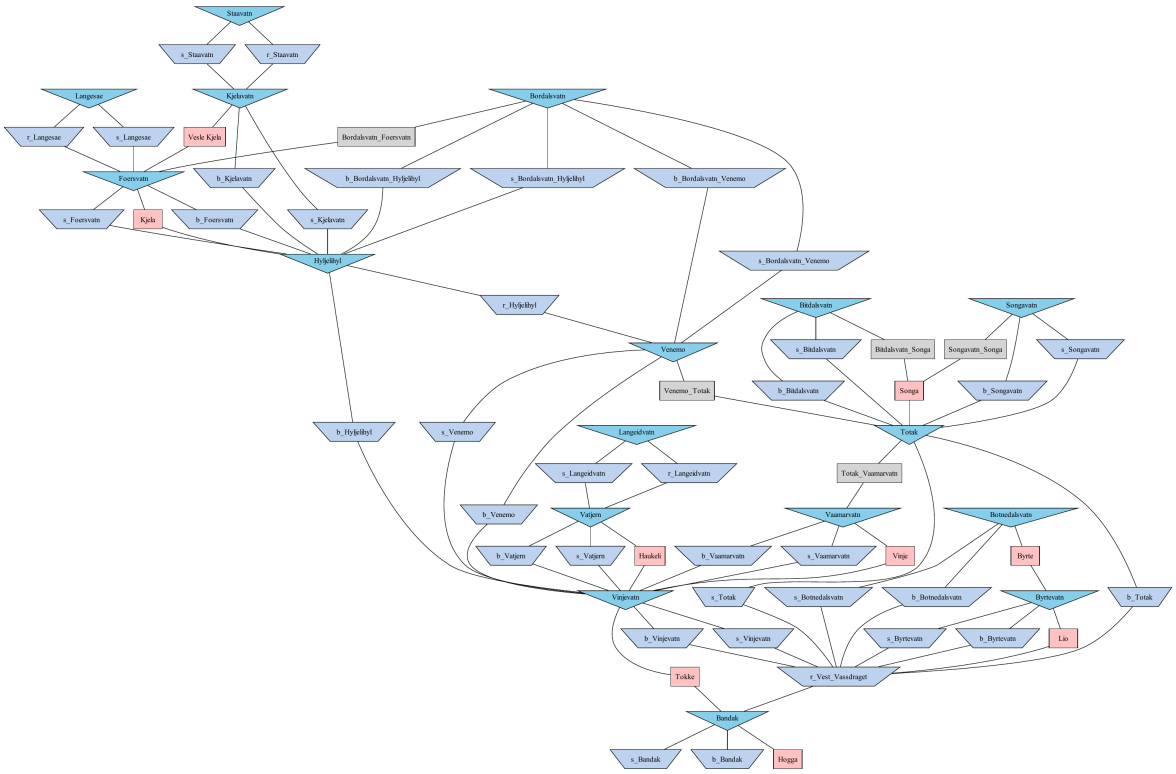


Figure 1: The Tokke–Vinje watercourse, as provided with the data (`Tokke_topology.pdf`): reservoirs (triangles), rivers and spillways (trapezoids), tunnels (grey) and the 9 plants (red). Water flows down the page to Bandak.

plant	units	$p_{\min}$ (MW)	$p_{\max}$ (MW)	start/stop cost (EUR)
Tokke	G1–G4 (identical)	36.9	107.5	214
Vinje	G1–G3 (identical)	34.4	100.0	198
Songa	G1	41.2	120.0	222
Kjela	G1	20.6	60.0	171
Lio	G1	14.8	43.0	163
Byrte	G1	6.9	20.0	135
Hogga	G1	3.0	17.0	94
Vesle Kjela	G1	2.9	8.5	92
Haukeli	G1	0.3	4.9	143

Table 2: System configuration from `Tokke_Vinje_topology.yaml`: 14 units, 1,003 MW in total. Tokke and Vinje hold 73% of the capacity.

2.2 Understand how the data was generated

- **The labels are optimiser output.** Every case was solved by SHOP v17.17 on the Tokke–Vinje model described by the topology file. The labels are therefore a deterministic function of the inputs we hold, not noisy observations of the real plant.
- **Cases are rolling.** A new case starts every day, so consecutive cases share 144 wall-clock hours, solved from almost the same state. The same hour solved one day later gets a *different* decision 12% of the time (18% for Tokke and Vinje; Figure 2): the decision depends on the whole horizon, including the end-of-week boundary, not only on the hour. This label self-disagreement explains 78% of the variance in per-unit difficulty of our first baseline. It also means random row splits leak (Section 3).
- **The water values are synthetic** (file name and documentation), one per reservoir per day. For a case starting on day d the documentation defines the initial value at d and the final (terminal) value at $d + 7$ 00:00. We first read the terminal value one day late (log 009) and found the off-by-one by rereading the documentation (log 033).
- **Two storage series are invented:** Hyljelihyl and Vatjern have no historical records and are set to 50% full throughout.
- **The storage series is observed, not planned:** it is the reservoir level at the start of each day, so the level at the end of a case reflects decisions taken during it ("post-decision" information, Section 2.6).
- **SHOP's conventions can be recovered from the labels.** Rebuilding the generator (Track B, Section 4.2) required three conventions the documentation does not state — how water values are converted to a value per cubic metre, how river costs are read, how identical units are ordered — and we measured each of them from the labels (Figure 16).

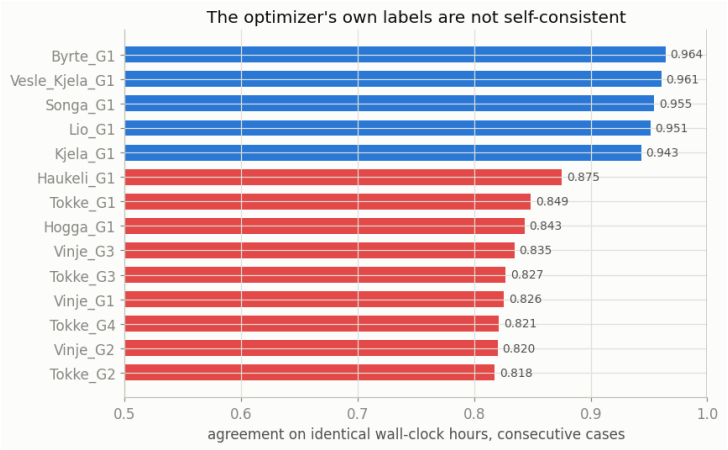


Figure 2: Agreement between consecutive cases on identical wall-clock hours, per unit. (Log 008.)

2.3 Check if the data is intuitive

Most series behave as domain knowledge predicts. Inflow peaks in May–June at about 8 times the winter level, and storage follows the expected annual fill and drawdown (Figure 3). The system water value tracks price but is much smoother, as an expected *future* value should be. Commitment has a double-hump daily pattern (trough 0.46 at night, peaks 0.55 at the morning and evening ramps) and almost no weekday pattern (Figure 4).

One fact is counter-intuitive: **2022 has the highest prices (∼211 EUR/MWh) and the lowest on-rate (0.34)** (Figure 5, left), and raw price correlates *negatively* with commitment (−0.19). The resolution is in the right panel: water values rose with prices, so high prices did not mean a high *spread*; 2022 was a scarce-water year in which SHOP conserved water. Measured within each week, price correlates positively (+0.26 for the within-week z-score). The data is consistent once read through the opportunity cost of water.

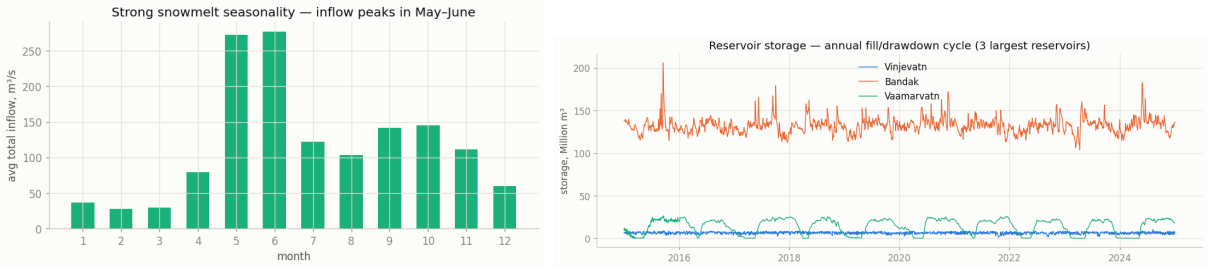


Figure 3: Left: mean total inflow by month (snowmelt in May–June). Right: daily storage of the three largest reservoirs. (Log 002.)

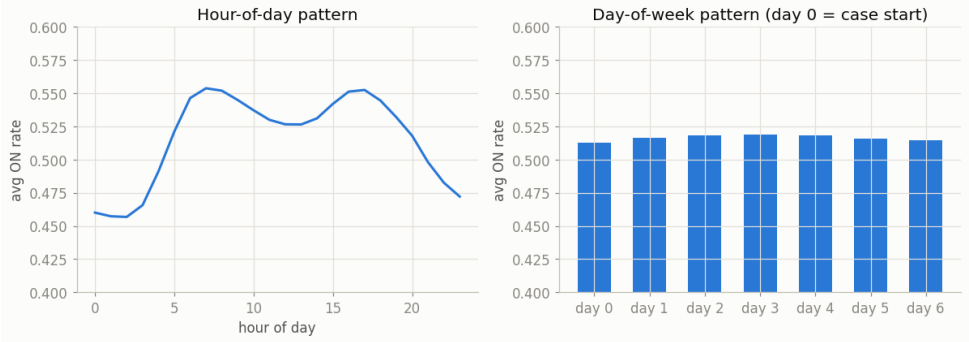


Figure 4: On-rate by hour of day and by day of week. (Log 002.)

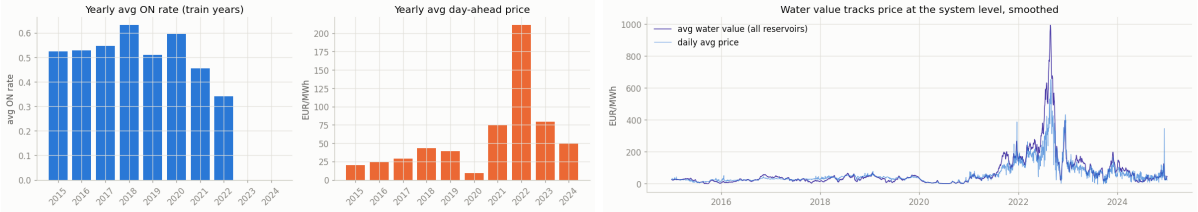


Figure 5: Left: yearly on-rate against yearly mean price — the highest-price year has the lowest on-rate. Right: system water value against price; the two rise together, so the spread does not. (Log 002.)

2.4 Explore individual features

Target. On-rates range from 26% (Vinje G3) to 75% (Lio G1): fourteen differently balanced problems pooled by one metric (Figure 6, left). Most unit-weeks are flat: Haukeli never switches in 81% of weeks, Vesle Kjela in 75%, Byrte in 74% (right); within a week commitments come in 1.2–2.7 contiguous blocks, $P(\text{on}_t \mid \text{on}_{t-1}) = 0.995$. The units are not stationary: Haukeli's on-rate falls from 0.87 (2015) to 0.14 (2022) and Songa swings $0.30 \to 0.71 \to 0.12$ (Figure 7).

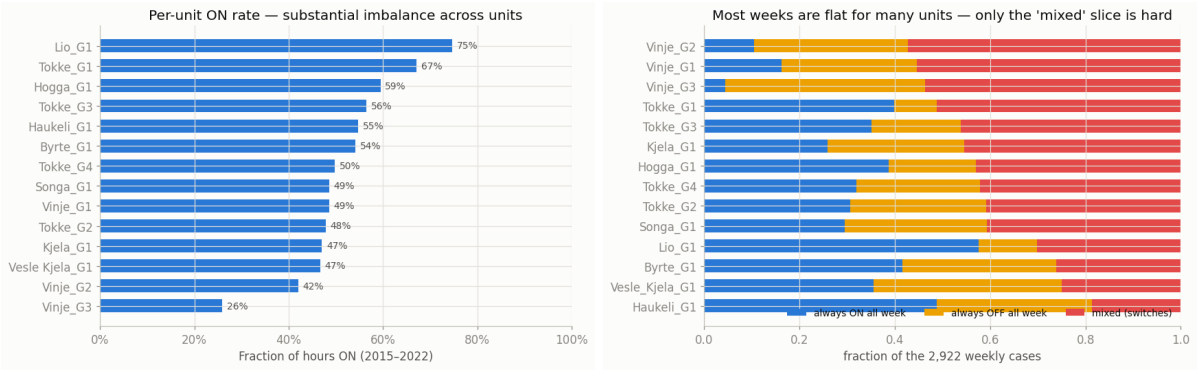


Figure 6: Left: on-rate per unit, 2015–2022. Right: share of unit-weeks that never switch. (Logs 002, 008.)

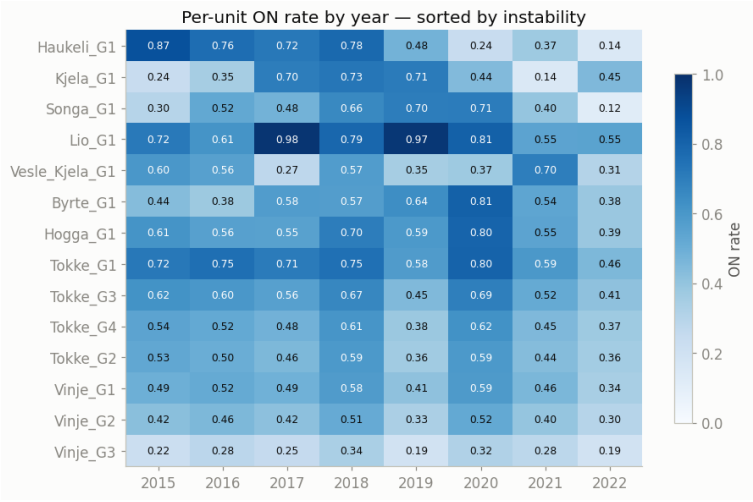


Figure 7: On-rate per unit by year, sorted by instability. (Log 008.)

Price. Stable at 9–43 EUR/MWh until 2020, then the 2021–22 energy crisis; the test period 2023–24 sits between the two regimes and contains negative prices in 2.2% of hours (Figure 8).

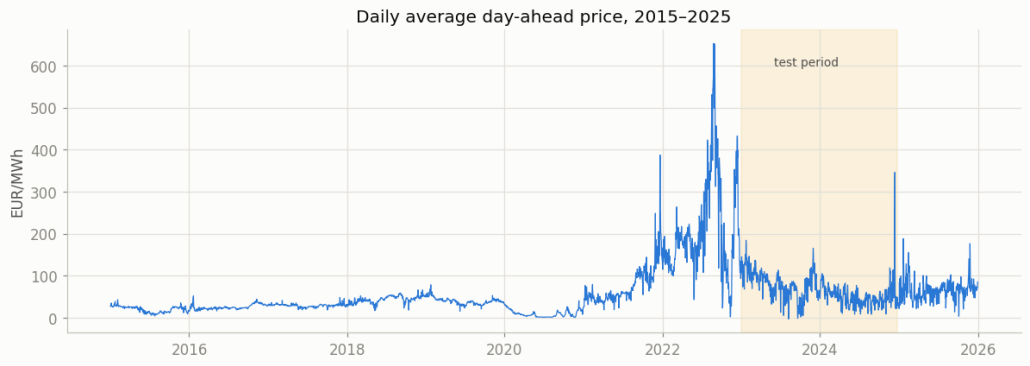


Figure 8: Daily mean day-ahead price, test window shaded. (Log 002.)

Inflow, storage, water value. Strongly seasonal (Figure 3). Raw levels are weak predictors because they confound the season: every reservoir is full in June. Normalised against the same time of year, they become the strongest univariate correlates in the project — storage's seasonal percentile +0.54, inflow's trailing-year mean +0.33 (Figure 13, Section 2.5).

Constraints. Minimum-volume constraints bind for exactly three reservoirs and only from July to October. Minimum flow is positive in every hour for five of the six constrained rivers, so a binary "constrained" flag would be constant; the information is in the level and its changes.

2.5 Explore pairs and groups of features

Units together. Units at one plant correlate 0.8–0.96, and Tokke and Vinje correlate 0.8–0.9 with each other because Vinje's discharge feeds the reservoir Tokke draws from (Figure 9, left). Identical units follow a near-strict merit order (right): one Tokke unit on is G1 99% of the time, two are G1+G3 94%; knowing how many units are on predicts exactly which ones with 97.8% (Tokke) and 99.9% (Vinje) accuracy.

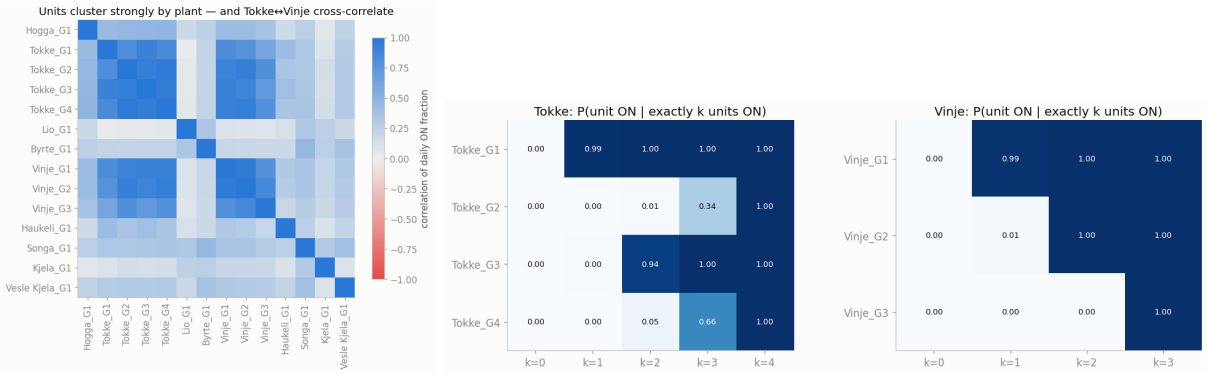


Figure 9: Left: correlation of the units' daily on-fraction, blocks by plant. Right: P(unit on | k units on at the plant) for Tokke and Vinje. (Logs 002, 008.)

Price with water value. The spread price − water value of the feeding reservoir correlates with commitment more strongly than either component, for every unit (Figure 10, left). Price helps only relative to its week: subtracting the case mean flips its correlation from −0.185 to +0.183 and dividing by the case's standard deviation gives +0.263 (right). The spread must *not* be normalised the same way (+0.241 raw, +0.114 z-scored): it is already relative.

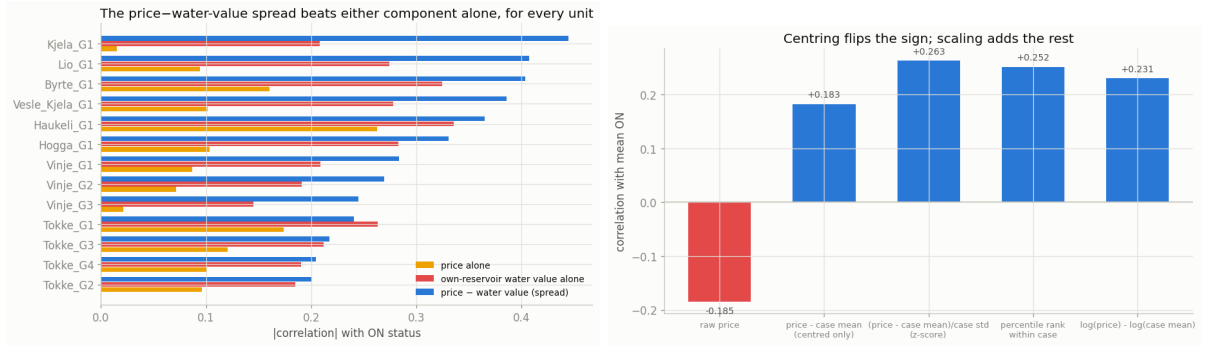


Figure 10: Left: |correlation| with commitment of price, own water value and their spread, per unit. Right: correlation of five price transformations with commitment. (Logs 008, 009.)

Groups of series. The 17 water-value columns share one factor (PC1 explains 91% of their variance); inflow needs 3 components for 90%, storage 7 (Figure 11). The useful decomposition is a system-wide level plus each reservoir's deviation from it; the deviation is the strongest of the three for five units but useless for Tokke, whose reservoir tracks the system.

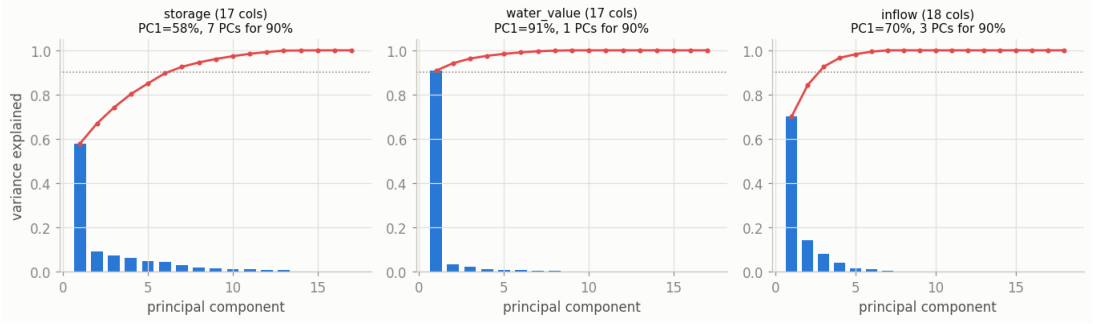


Figure 11: Variance explained by principal components within each family of series. (Log 009.)

Across time. The terminal water value correlates −0.41 with case-level commitment against −0.26 for the initial value, and the two differ in 93% of case/reservoir pairs (Figure 12, left). Price correlates with commitment most weakly at the current hour and more strongly at leads and lags of several days (right): SHOP plans the whole week. Normalising the slow series against the same time of year raises their correlation 1.3–2.6× (Figure 13).

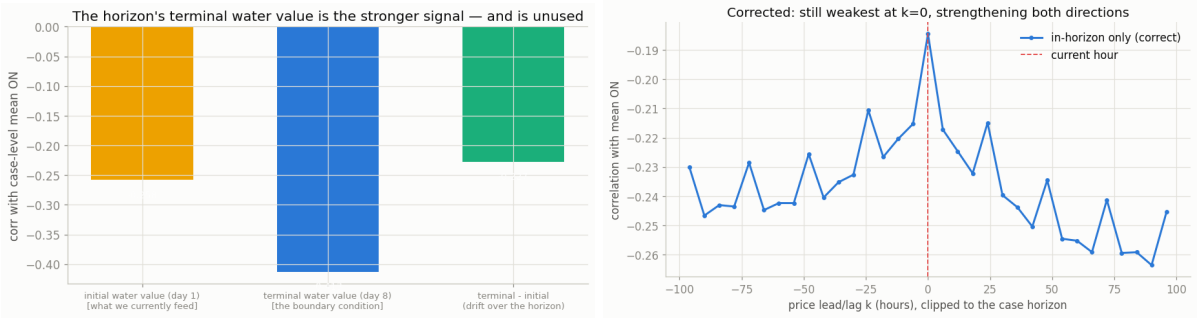


Figure 12: Left: initial and terminal water value against case-level commitment. Right: correlation of commitment with price k hours earlier or later, within the case horizon. (Logs 009, 011.)

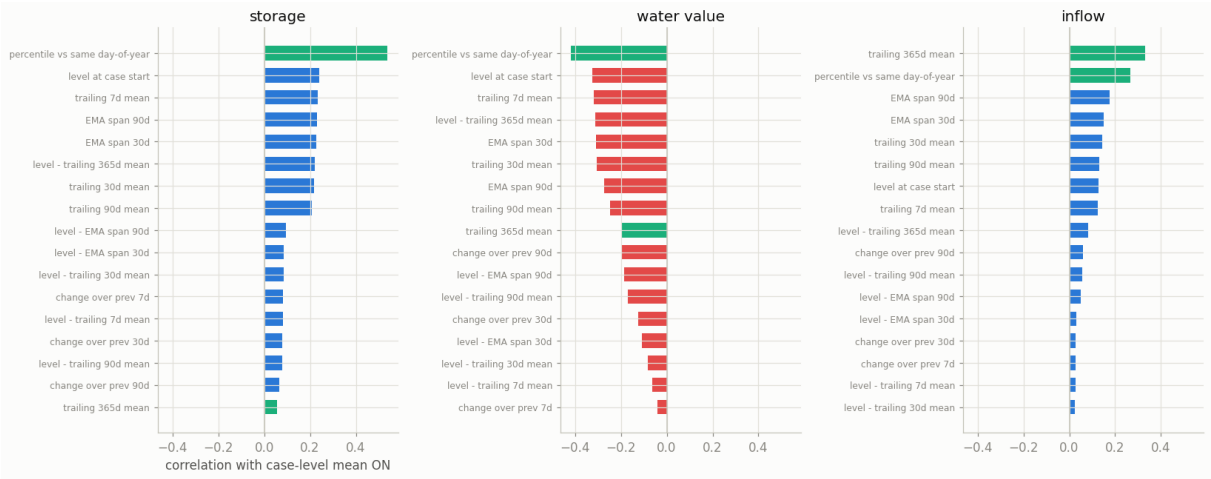


Figure 13: Seventeen backward-looking transformations each of storage, water value and inflow, ranked by correlation with commitment. (Log 011.)

2.6 Clean up features

check	action
missing values	none in any core or extended file; reservoir names consistent across files
constant series	storage of Hyljelihyl and Vatjern is fixed at 50% (no records): dropped
leakage	`total_calculation_time` (SHOP's solve time) exists only for labelled cases and describes the solution: excluded from all features
availability	every feature file covers all 725 test cases (price to 2025-12, inflow to 2025-10, storage and water value to 2025-01-01); labels end in 2022, so no feature may use past labels. The terminal water value of the last test case (2024-12-25) is the file's final row
redundant scaling	normalising a single column (e.g. storage by reservoir size) is a no-op for tree models; scaling was only applied where columns are combined
uninformative encodings	a binary "min-flow active" flag is constant for five of six rivers: the level is used instead
post-decision information	end-of-week storage reflects the decisions being predicted. It is legal (the files cover the test period) and was used in Track A; Track B shows it is not needed (Section 4.4)
unit identity	plant names are sanitised for column names (`Vesle Kjela`) and mapped back from the topology for the submission header

Table 3: Data cleaning. (Logs 001, 008, 011.)

2.7 Training versus test period

We measured which past period resembles the test set instead of assuming it: a classifier separates each half-year's cases from the 2023–24 test cases using case-level inputs (price level, volatility, negative-price share, water value, spread, fill, inflow). 2021-H2 is the hardest to separate (AUC 0.785), then 2022-H2 (0.879); every other half-year is above 0.95 (Figure 14). We report these two *test-like folds* separately in validation.

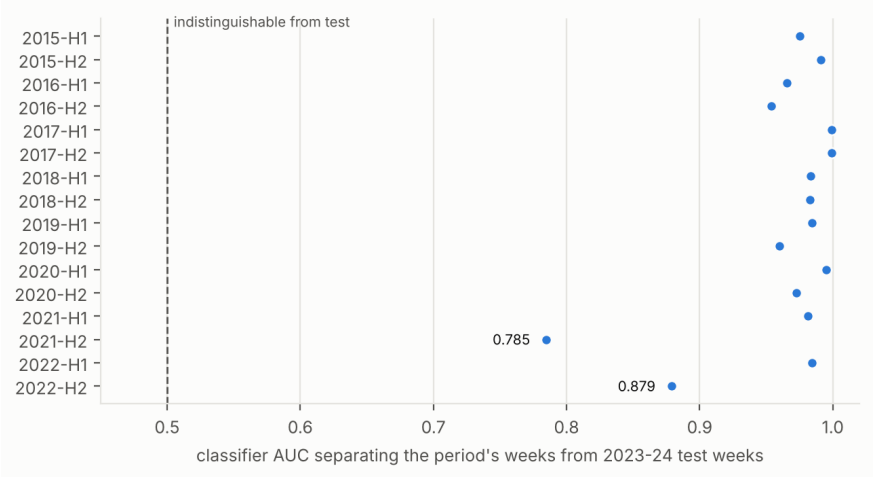


Figure 14: Adversarial validation: how easily each half-year's cases are told apart from the test cases. Lower is more test-like; 0.5 would be indistinguishable.

2.8 What the analysis implied for modelling

finding	consequence
the spread price − water value drives decisions	spread features in Track A; LP margin in Track B
terminal water value is the week's boundary condition	terminal features; the LP's end-of-week value
price matters relative to its week	case-relative price features, never raw levels
slow series confound season	seasonal normalisation (Track A)
units are coupled through the cascade	one joint model for all units; long format
identical units follow a merit order	unit identity and merit rank as features; order constraints in the LP
consecutive cases share 144 hours	temporal validation only (Section 3)
regimes shift; 2021-H2 is most test-like	walk-forward folds with test-like folds reported separately
labels are the output of a known optimiser	Track B: rebuild the optimiser as a feature generator

3 Validation methodology

Random splits leak. A random row split scores 0.957 with the same model that scores 0.835 on a held-out year: rows from one week are near-duplicates, and consecutive weeks share 144 hours at 88% label agreement. Random *case* splits still inflate by +0.105 (log 017). Every number in this report therefore comes from a temporal split.

Walk-forward. The reference protocol (logs 024, 028) has six half-year folds from 2020-H1 to 2022-H2; each trains on every case starting before the fold and scores the six months after. Earlier stages used a single held-out year (2021, the most test-like year, log 020) or four-fold rolling-origin CV; we judged changes on the per-fold sign pattern, since fold-to-fold variation (SD up to 0.06) dwarfs most feature effects. A rotating-block scheme was built and rejected: training on both sides of the held-out block measures interpolation, while the test set requires extrapolation (log 035). Experiments train on every third case ("stride 3"); submissions train on all cases.

4 Feature engineering

We used both kinds of feature engineering the course names: *feature extraction* (new features built from the raw series, Sections 4.1, 4.2 and 4.3) and *feature selection* (Section 4.4).

4.1 Feature extraction I: economic and structural features (Track A)

Twenty feature themes were built and screened (logs 012–014), following the EDA's rules: case-relative price, raw spreads, seasonally normalised slow series, a system factor plus reservoir deviations, multi-hop topology for unit-to-reservoir mapping.

family	what it computes	effect
spread	price − water value of the reservoir feeding each plant, positive part, system spread	+0.075
terminal water value	value at the end of the week; price − terminal value; per-plant terminal spreads; slope	+0.034; day fixed: +0.006
merit and capacity	units a spread justifies ($p_{\max}/p_{\min}$), capacity share needed, break-even hours from start costs	in top 15
cascade graph	upstream spreads and water values via multi-hop traversal of the topology	in top 15
count table	per (case, unit): trailing and forward windows of price, inflow, storage; water balance and implied production; unit-relative views	+0.078 (water balance)
hourly structure	the spread summed over windows of several lengths around each hour: a unit on for c hours runs in a high-spread stretch	+0.003
long format	one row per (case, hour, unit): unit-specific features without 14× column cost	+0.006

Table 4: Track A feature families. Effects are the gains measured when each was added (logs 013, 022, 031–033; protocols differ between rows).

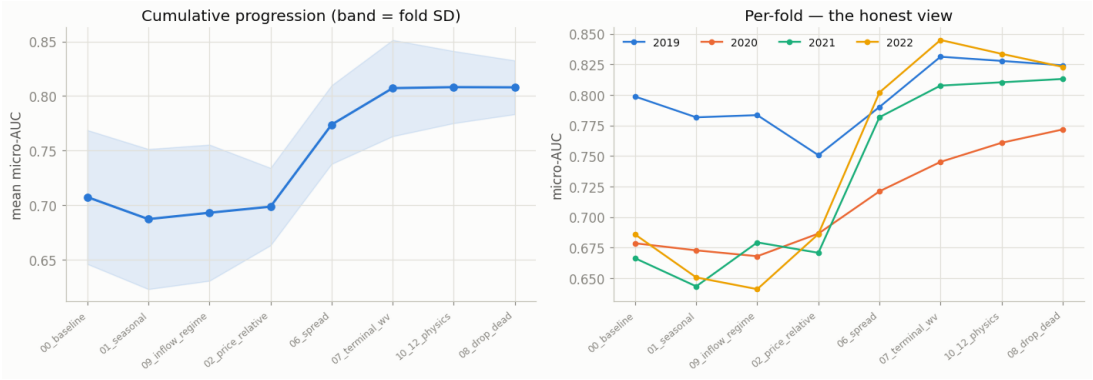


Figure 15: Feature iteration 1: cumulative micro-AUC as stages are added, with the four validation folds (right). The spread and terminal-water-value stages carry the gain; the seasonal stage hurts. (Log 013.)

Features that did not help are as informative as the ones that did (Table 5): the strongest univariate correlate in the project, storage's seasonal percentile (+0.54), made the model *worse*.

rejected feature idea	effect	log
expanding-window seasonal percentile (strongest univariate signal)	−0.020	013
"physics" stage (heads, MWh per m^3) as features	−0.012	014
corrected water balance for the four worst units	−0.003	027
one contiguous block per unit-week (oracle counts)	−0.043	031
67-year inflow climatology	−0.000	031
min-flow obligation per plant	−0.003	031
287 slope (rate-of-change) features	−0.002	035
full count distribution forwarded; hydraulic neighbour counts	−0.001	035

Table 5: Track A feature families that were built, measured and rejected.

4.2 Feature extraction II: a first-principles optimiser as a feature generator (Track B)

Track A approximated SHOP with hand-made features. But the labels are the output of one deterministic optimiser, and every input it used is in our data — the topology file is SHOP-ready. So we rebuilt the optimisation problem itself (`src/shoplite.py`; Short notebook 1) and use its solution as features. The LP is a **feature extractor only**: its output is never submitted; every prediction comes from the learned model.

The linear programme. One per case, the LP relaxation of SHOP's weekly unit-commitment problem, solved with HiGHS (dual simplex; building and solving take 8 s per case on one core on average, 2–25 s for individual cases; SHOP's own recorded solve times are a median 52 s, mean 147 s):

$$\max \sum_t \lambda_t (P_t - L_t) \; - \; \text{start/stop costs} \; - \; \text{river, bypass, spill costs} \; - \; \text{penalties} \; + \sum_r \omega_r V_{r,T}$$

subject to hourly water balance in all 17 reservoirs, the Vest-Vassdraget river junction and the Songa tunnel junction; reservoir bounds; minimum volume and minimum flow (soft, heavily penalised); for every unit an on/off variable $u \in [0, 1]$ carrying the minimum block ($q_{\min}$, $p_{\min}$) plus segments of the concave envelope of its PQ curve at the case's initial head (turbine efficiency interpolated between the reference-head curves, times generator efficiency); start/stop variables with the topology's costs; and main-tunnel and shared-penstock friction L_t as a convex piecewise-linear cubic in plant flow. λ_t is the price and ω_r the terminal value of water. The LP's duals give the hourly shadow price of water in each reservoir, from which each unit's *local water value* (EUR/MWh) and its margin, price − local water value, follow. Simplifications against SHOP: heads frozen at their initial levels, lossless tunnels between reservoirs, and an unknown initial on/off state.

Three conventions measured from the labels. **Terminal water value:** in weeks where a unit switches, its on-hour and off-hour prices bracket the threshold SHOP used; regressing that threshold on terminal water values gives $\approx 1.0\times$ the feeding reservoir's value and $\approx 0\times$ the downstream value (Figure 16; Byrte slope 0.98, R^2 0.99), so one Mm^3 in reservoir r is valued as $\omega_r = \sum_k \text{WV}(\text{reservoir feeding } k) \cdot \text{conv}_k$ over the plants k on its way to the sea. **River flow cost:** `flow_cost_curve` read as total cost per hour (marginal cost = slope) rather than as a marginal cost lifted Kjela's raw AUC from 0.70 to 0.96. **Sibling order:** the merit order from the EDA is imposed as $u_{\text{G1}} \geq u_{\text{G3}} \geq u_{\text{G4}} \geq u_{\text{G2}}$ at Tokke and $u_{\text{G1}} \geq u_{\text{G2}} \geq u_{\text{G3}}$ at Vinje.

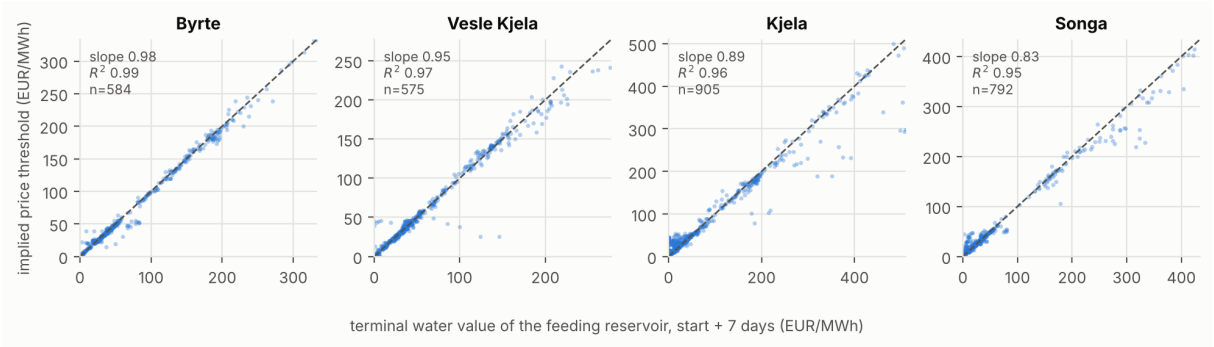


Figure 16: Implied price threshold in every mixed week against the terminal water value of the reservoir feeding the plant. Points lie on $y = x$: each plant breaks even at its own reservoir's water value.

LP versus MILP as a feature. Solving with binary u gave a raw AUC of 0.941 against the relaxation's 0.967 on the same 59-case sample: a 0/1 schedule loses the graded information about how close each hour is to the margin. The relaxation is the better feature.

The 61 features. One row per (case, hour, unit): `lp_` — the LP solution (commitment u, power, flow), its temporal context (rolling windows, run lengths, weekly and daily means, number of units on at the plant), the local water value and margin, and the reservoir trajectories; `ec_` — price level, rank and z-score within the week, spread to the terminal water value, season, inflow; `cn_` — the constraint series; `st_` — the observed storage trajectory; `s_` — unit identity and static attributes.

4.3 Feature extraction III: closed-form physics features for a neural network (Track C)

Track C builds 99 features per (case, unit, hour) for a sequence model (Short notebook 2). The 83 base features follow the same EDA findings as Track A:

group	n	content
economic core	9	margin = price − terminal water value of the intake reservoirs; at the initial value; share of positive hours; daily and rolling (12 h, 24 h min/max) views
price	18	level, within-week z-score and rank, within-day rank and position, daily spread, rolling means, lags and leads of 1/3/6 h, price minus the mean of the remaining hours
water value	9	intake and downstream values, change over the week, daily path and its volatility, a margin against the whole cascade's water value
state	9	fill of the intake, storage and downstream reservoirs, system fill, five large reservoirs
inflow	5	weekly inflow to the plant's storage, relative to its size, system inflow, its anomaly against the 1958–2014 climatology, hourly system inflow
constraints	9	minimum flow on six rivers; distance of three reservoirs to their minimum volume
volume path	8	observed storage changes over the week and implied releases (post-decision information)
unit	8	identity, $p_{\min}$, $p_{\max}$, start cost, position in the plant, number of siblings, head, hour index
calendar	5	local hour and weekday, season, Norwegian holidays
plant context	3	system-mean margin, the unit's margin relative to it, rank across units

Table 6: Track C base features.

Physics features without a solver. Sixteen features are derived in closed form from each generator's efficiency curves at the plant head (minus friction losses, times generator efficiency, reduced to a concave envelope):

- **break-even prices** at minimum, full and marginal load — the intake water value scaled by how much less efficient that operating point is than the plant's best one — and the margins of price over them;
- a **weekly water budget**: hours of full-plant flow the reservoirs feeding the plant can afford (storage above the minimum volume plus inflow), and hours it is *forced* to run because the intake would otherwise overflow;
- a **greedy schedule**: the plant runs in its k best-margin hours, k the number of positive-margin hours clipped between the forced and affordable hours; whether each hour is in it and how far from the cut-off;

- **block economics**: for each run of positive-margin hours, its length and profit at full load against the start cost, and for each negative run, whether idling at $p_{\min}$ is cheaper than stopping and restarting.

These approximate, hour by hour, what Track B's LP computes jointly. They add +0.004 to the yearly cross-validated AUC of the network (0.9697 → 0.9740), most of it on 2021 (0.9514 → 0.9658), the most test-like year (Section 2.7).

4.4 Feature selection

Pruning by gain (log 015). All 299 Track A features were ranked by gain from one fit and the feature-set size swept: the top 15 beat the full set (0.8157 vs 0.8144 over four folds, with half the fold SD). All 15 survivors are economic quantities; no raw price, storage, inflow or calendar column survived. They are the 15 `PRUNED_FEATURES` of the Track A notebook.

feature set	mean micro-AUC (4 folds)	SD
all 299	0.8144	0.0304
top 15 by gain	**0.8157**	**0.0158**
top 15, 900 trees	0.8097	0.0151

Table 7: Pruning: 15 features beat 299. (Log 015.)

Model-based ranking (logs 032–036). The count table holds 446 candidate (case, unit) features. A 6-class XGBoost model over hours-on bins is fitted *only to rank them*; the top 120 by importance, plus up to 30 unit-relative features ranked below them, are attached per unit (141 in the submission). Three selection methods were compared; every one that used validation data looked better on the validation year and worse on the folds:

selection method	single year 2021	walk-forward
count-model ranking (chosen without validation data)	0.9651	**0.9638**
permutation importance + unit-relative trimming	**0.9671**	∼0.9609 (3 folds, all worse)
permutation top-60 (circular)	0.9668	not run
held-out inner-split ranking	0.9631	not run

Table 8: Feature-selection methods. (Log 036.)

Group selection in Track B. Removing one feature group at a time on the walk-forward folds (Figure 20, Section 7) shows the observed storage trace can be dropped at no cost (0.9910 vs 0.9912): the model needs only information available before the week starts. Without the LP features the same learner falls to 0.9483.

4.5 Inputs used by each final model

input	notebook 1: LP + LightGBM	notebook 2: neural network
price (hourly)	✓	✓
inflow (hourly)	✓	✓ (and its 1958–2014 climatology)
storage (daily)	✓	✓
water value, initial and terminal (daily)	✓	✓ (and the daily path)
minimum volume, minimum flow (extended)	✓	✓
topology and system configuration (extended)	✓ (the full LP)	✓ (efficiency curves, heads, costs)
calendar	season	local hour, weekday, season, holidays

5 Predictors: models and algorithms

5.1 Four predictor types on identical features

To separate the effect of the learner from the effect of the features, four types of predictor were trained on the same 61 features (Section 4.2) and scored on the same six walk-forward folds (stride 3; `src/predictor_comparison.py`):

- **logistic regression** — a linear model: median imputation, standardisation, one-hot unit and plant;
- **random forest** — 200 bagged trees, `min_samples_leaf` 100, $\sqrt{p}$ features per split, a quarter of the rows per tree;
- **XGBoost** — boosted trees with Track A's tuned settings (Section 6);
- **LightGBM** — boosted trees with the submitted model's settings.

fold	logistic regression	random forest	XGBoost	LightGBM
2020-H1	0.9864	0.9915	0.9922	0.9925
2020-H2	0.9776	0.9854	0.9869	0.9866
2021-H1	0.9906	0.9939	0.9947	0.9950
2021-H2 (test-like)	0.9810	0.9893	0.9909	0.9914
2022-H1	0.9754	0.9835	0.9871	0.9862
2022-H2 (test-like)	0.9885	0.9934	0.9952	0.9953
mean	0.9833	0.9895	**0.9912**	**0.9912**
test-like mean	0.9848	0.9913	0.9930	**0.9934**

Table 9: Four predictor types on the same 61 features and the same walk-forward folds (micro ROC-AUC). `report/runs/predictor_comparison_s3.csv`.

The ranking is the same on every fold: the linear model below the bagged forest, the forest below the two boosted ensembles. XGBoost and LightGBM tie (0.9912 each; 0.9930 and 0.9934 on the test-like folds), so the choice of boosting library does not matter; we kept LightGBM, the submitted model. More important, every type scores far above Track A's XGBoost on its own engineered features (0.9619): even logistic regression on the LP features reaches 0.9833. The gain of Track B is in the features the optimiser extracts, not in the learner.

5.2 Every model we trained, helpful or not

Table 10 lists every model in `report/leaderboard.md`, in order. Protocols changed as the project matured (single year 2022 → four-fold rolling origin → single test-like year 2021 → six-fold walk-forward), so scores are comparable only within a protocol.

log	model	protocol	AUC	verdict
004	14 independent XGBoost classifiers, raw features	year 2022	0.647	baseline
006	one multi-output XGBoost tree (shared splits across units)	year 2022	0.687	kept
007	multi-output tree, 300 / 700 trees	year 2022	0.686 / 0.695	300 kept (plateau)
013	multi-output tree + engineered features	4-fold CV	0.808	kept
015	pruned to top 15 features (900 trees: 0.810)	4-fold CV	0.816	kept
016	stacked two-stage model (uniform / gated)	year 2022	0.839 / 0.843	superseded
018	oracle: true hours-on, allocated by spread (upper bound)	year 2022	0.977	diagnostic
018	count model, then allocate hours by spread	year 2022	0.780	rejected
019	three-class count model + allocation	year 2022	0.826	rejected
019	pointwise model + count-budget augmentation	year 2022	0.855	kept
021	+ multi-horizon count features	year 2021	0.856	kept
022	+ water-balance (post-decision) features	year 2021	0.912	kept
022	same without post-decision information	year 2021	0.855	diagnostic
023	+ unit-relative and absolute count features	year 2021	0.917	kept
024	walk-forward estimate of that model	walk-forward	0.902	
025	count features into the pointwise model	year 2021	0.929	kept
026	compact distilled architecture	year 2021	0.940	kept
027	water-balance fixes (v2)	year 2021	0.937	rejected
028	walk-forward estimate, compact model	walk-forward	0.935	
030	binned 6-class count head	year 2021	0.942	kept
030	unit-resolved count model / stationarised variant	year 2021	0.912 / 0.940	rejected
031	+ hourly temporal structure	year 2021	0.944	kept
031	block allocation; per-unit calibration; climatology; min-flow	year 2021	0.906–0.942	rejected
032	long format, one row per (case, hour, unit)	year 2021	0.950	kept
032	long format with Lio up-weighted	year 2021	0.948	rejected
033	terminal water value corrected to start +7 days	year 2021	0.960	kept
034	tuned pointwise model; tuned count model (no change)	year 2021	0.963 / 0.963	kept
035	slope features; richer count outputs; unit masking	year 2021	0.961–0.963	rejected
036	**submission 1**: count model only ranks features	walk-forward	0.9638	submitted
036	**submission 1** on the public leaderboard	test set	**0.98058**	
037	MILP relaxation's schedule as a feature (59 cases)	sample	0.941	rejected
037	LP relaxation's commitment feature alone, no learning	walk-forward	0.9710	diagnostic
037	**LightGBM on LP features (submission 2)**	walk-forward	**0.9912**	selected
037	LightGBM on LP features, no storage trace / LP only	walk-forward	0.9910 / 0.9897	ablation
037	LightGBM without the LP features	walk-forward	0.9483	ablation
B	LP features: logistic regression / random forest / XGBoost	walk-forward	0.983 / 0.990 / 0.991	comparison
C	UCNet on the 83 base features	yearly CV	0.9697	superseded
C	LightGBM on the 99 base + physics features	yearly CV	0.9722	rejected
C	**UCNet on base + physics features (submission 3)**	yearly CV	**0.9740**	submitted
C	**submission 3** on the public leaderboard	test set	**0.99013**	selected

Table 10: All models, in the order they were trained ("C": Track C, documented in its own specification rather than the lab log). "Year 2022" and "year 2021" hold out one year; 4-fold CV is rolling-origin; walk-forward is six half-year folds; yearly CV is Track C's expanding window (2019–2022).

Three architectural ideas carried Track A. Joint modelling of the correlated units (0.647 → 0.687). A count model over hours-on per unit-week: given the true count, allocating hours by spread reaches 0.977, so the hard part is one number per unit-week; the count model first augmented and later only ranked features, because forwarding its predictions cost 0.008 (log 035). And the long format, which made unit-specific features affordable.

5.3 Track B: LightGBM on LP features

Both submitted models were run on the same six walk-forward folds on the same machine, each training on every third case; the submitted XGBoost model re-run this way reproduces its recorded 0.9638 to within 0.002.

fold	XGBoost (Track A)	LP feature alone	LP + LightGBM	Δ
2020-H1	0.9535	0.9720	**0.9925**	+0.039
2020-H2	0.9455	0.9671	**0.9866**	+0.041
2021-H1	0.9765	0.9787	**0.9950**	+0.019
2021-H2 (test-like)	0.9504	0.9695	**0.9914**	+0.041
2022-H1	0.9672	0.9586	**0.9862**	+0.019
2022-H2 (test-like)	0.9784	0.9802	**0.9953**	+0.017
mean (SD)	0.9619 (0.014)	0.9710 (0.008)	**0.9912 (0.004)**	+0.029

Table 11: Walk-forward micro ROC-AUC. "LP feature alone" scores the LP's commitment feature u directly, with no learning: it measures the feature, it is not a submitted predictor.

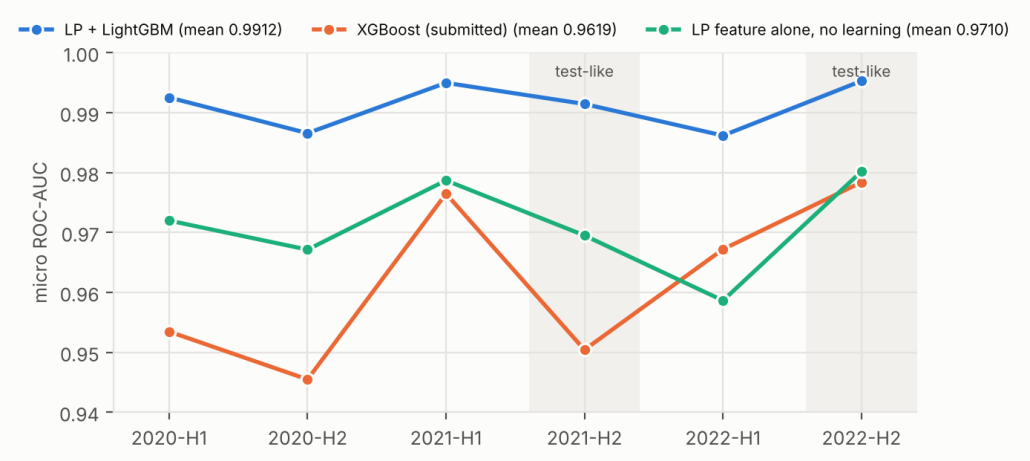


Figure 17: Walk-forward results per fold. The LP commitment feature on its own already ranks hours better than the trained Track A model on five of six folds.

Every fold and every unit improves (Figure 18); the two units Track A never fixed move most (Lio 0.864 → 0.979, Haukeli 0.920 → 0.973), and the fold SD falls from 0.014 to 0.004.

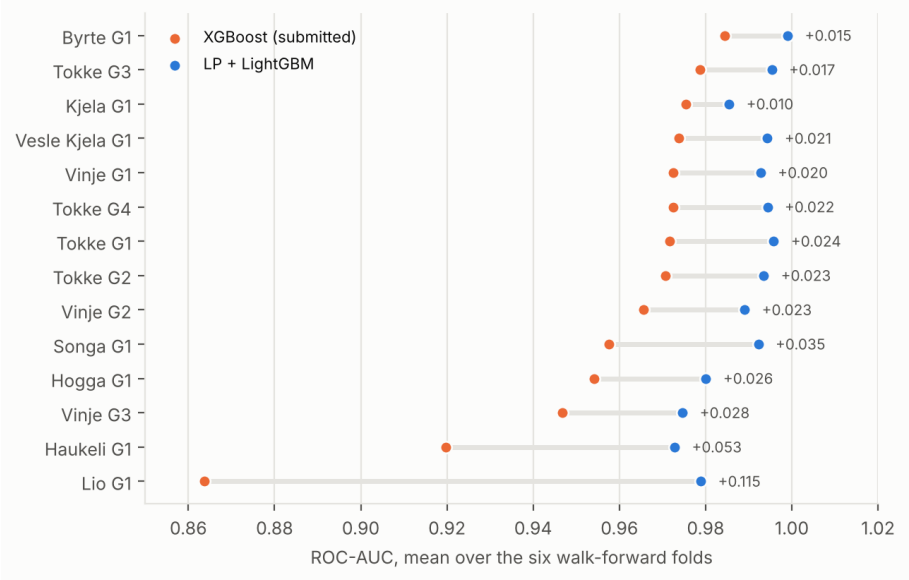


Figure 18: Per-unit ROC-AUC, mean over the six folds, Track A (orange) and Track B (blue).

5.4 Track C: a sequence neural network

Each case is one sample: an input of 14 units × 168 hours × 99 features and a target of 14 × 168 commitments. The network, UCNet (244,161 parameters), mirrors the structure the EDA found:

- a two-layer input projection to width 96, concatenated with a learned 16-dimensional embedding of the unit;
- a residual temporal convolution (two layers, kernel 5, the second dilated) and a residual two-layer bidirectional GRU along the 168 hours of each unit — commitments come in blocks;
- residual multi-head self-attention (4 heads) across the 14 units at each hour — units share water and are coupled through the cascade;
- a two-layer head giving one logit per unit-hour.

Inputs are scaled per feature (median, 10–90% range, clipped at ±8). Training minimises binary cross-entropy over all unit-hours with AdamW and a one-cycle learning-rate schedule, 20 epochs, batches of 32 cases; the submission averages three seeds' probabilities.

Validation. Track C used an expanding window by year: train on cases starting more than 7 days before 1 January of year Y (consecutive cases overlap, so the gap prevents leakage), score year Y. One seed:

validation year	AUC	validation AUC after epoch				
		4	8	12	16	20
2019	0.9837	0.9783	0.9790	0.9817	0.9833	0.9837
2020	0.9685	0.9643	0.9649	0.9641	0.9696	0.9685
2021	0.9658	0.9637	0.9648	0.9626	0.9670	0.9658
2022	0.9779	0.9748	0.9710	0.9772	0.9781	0.9779
mean	**0.9740**					

Table 12: Track C, yearly cross-validation (micro ROC-AUC).

On the same yearly folds, the same network without the 16 physics features scores 0.9697, and LightGBM on the same 99 features 0.9722: the sequence model adds +0.002 over boosted trees on these features, and the physics features +0.004 over the base set. On the public leaderboard the network scores **0.99013**, far above its cross-validation — the same direction as Tracks A and B, whose walk-forward estimates also understated their leaderboard scores (the test period is post-crisis, the folds are not).

The two selected models on the same folds. Track B's walk-forward and Track C's yearly cross-validation use different folds, so their figures are not comparable. We therefore also scored LP + LightGBM, with Short notebook 1's settings, on Track C's protocol: the same yearly folds, the same 7-day gap, and all training cases with no stride (`src/lp_yearly_cv.py`).

validation year	Track C (network)	Track B (LP + LightGBM)	difference
2019	0.9837	**0.9927**	+0.009
2020	0.9685	**0.9905**	+0.022
2021	0.9658	**0.9939**	+0.028
2022	0.9779	**0.9914**	+0.013
mean	0.9740	**0.9921**	+0.018

Table 13: The two selected models on Track C's yearly folds (micro ROC-AUC). Track C's figures are from a single seed.

Track B is ahead in every year. Its lead is largest in 2021 (+0.028), the year that most resembles the test period (Section 2.7), and its scores vary less across years (0.9905–0.9939, against 0.9658–0.9837). The public leaderboard points the same way, by a smaller margin (0.99522 against 0.99013), which is why Short notebook 1 holds our best score.

6 Hyperparameters: how the values in the notebooks were obtained

The course allows a few hyperparameter values to be set directly in the short notebooks, provided the report says how they were obtained. Every value set by hand in either notebook is listed here.

6.1 Track A: XGBoost (first submission, not selected)

Track A's notebook is no longer one of the two selected predictions, but its values are documented for completeness. The point model's and count model's settings were checked by coordinate descent, one knob at a time, on the most test-like validation year (2021) at stride 3 (log 034).

parameter	value	how it was obtained
`min_child_weight`	500	raised from 1: +0.0018 (0.9604 → 0.9622). Predicted from the data: hourly rows are autocorrelated at 0.995, so the effective sample size is ∼50× below the row count and small leaves fit single blocks
learning rate × rounds	0.05 × 600	from 0.10 × 300: +0.0010 (0.9622 → 0.9632)
`max_depth`	8	inherited; 10 and 12 gave 0.9621 and 0.9620: no gain
`colsample_bytree`, `subsample`	0.8, 0.8	inherited; column sampling 0.5 and 0.3 gave 0.9625 and 0.9626: no gain
`max_bin`	256	library default
count model	400 trees, rate 0.05, depth 5, `colsample` 0.6	set when the count model was built (log 018); screened on log-loss in log 034 (`min_child_weight` 10/50/200, depth 4/6/8, 800 and 1,200 trees): nothing beat it
forwarded count features	top 120	fixed when the long format was built (log 032); 180 and 200 gave 0.9609 and 0.9618 against 0.9632 (log 034)
unit-relative extras	up to 30	set by hand, not tuned: unit-relative and absolute features together beat either alone (log 023)
`PRUNED_FEATURES`	15 names	gain ranking and size sweep (log 015; Section 4.4)
seed	0	fixed for reproducibility

Table 14: Hyperparameters of the Track A notebook.

6.2 Short notebook 1: LightGBM on LP features (Track B)

These settings were *not* tuned for this model before it was submitted: they were carried over from Track A's tuned XGBoost, translated to LightGBM's parameter names.

parameter	value	how it was obtained
learning rate, rounds	0.05, 600	as Track A (log 034)
`min_data_in_leaf`	500	as Track A's `min_child_weight`, for the same autocorrelation reason
`num_leaves`	127	LightGBM grows trees leaf-wise; 127 leaves is one fewer than a full depth-7 tree, the capacity range Track A found sufficient (depth 8; deeper gave nothing)
`feature_fraction`, `bagging_fraction`	0.8, 0.8	as Track A's column and row sampling; bagging every iteration
`lambda_l2`	1	XGBoost's default L2 penalty, kept for parity
`max_bin`	255	library default
categorical features	unit, plant	native categorical splits instead of one-hot
seed, threads	0, 4	reproducibility; the course's 4-core limit

Table 15: Hyperparameters of Short notebook 1.

Sensitivity, checked after submission. Because these values were inherited rather than tuned, we varied each one around its submitted value, one at a time, on the two test-like walk-forward folds (stride 3, as in Table 11; `src/lgbm_sensitivity.py`):

variant	2021-H2	2022-H2	mean	vs. submitted
submitted	0.9914	0.9955	**0.9935**	—
`num_leaves` 31	0.9912	0.9951	0.9932	−0.00029
`num_leaves` 255	0.9915	0.9954	0.9934	−0.00001
`min_data_in_leaf` 100	0.9910	0.9956	0.9933	−0.00017
`min_data_in_leaf` 2000	0.9914	0.9953	0.9933	−0.00012
learning rate 0.1, 300 rounds	0.9909	0.9953	0.9931	−0.00035
learning rate 0.025, 1,200 rounds	0.9916	0.9954	0.9935	+0.00004
`feature_fraction` 0.5	0.9917	0.9953	0.9935	+0.00005
`feature_fraction` 1.0	0.9910	0.9953	0.9931	−0.00032

Table 16: One-at-a-time sensitivity of Short notebook 1's LightGBM settings (micro ROC-AUC). This run was on a different machine from Table 11, which is why the submitted settings score 0.9955 rather than 0.9953 on 2022-H2.

No change moves the mean by more than 0.0004. Two variants score above the submitted settings, `feature_fraction` 0.5 and the slower learning rate, both by less than 0.0001 and on only one of the two folds (2021-H2). The submitted model is therefore not sitting on a sharp optimum that it happened to inherit from Track A: within this range, LightGBM's settings hardly matter, and the score comes from the LP features (Section 4.4). We did not change the submitted model after seeing these results.

6.3 Short notebook 2: UCNet (Track C)

The network's architecture and optimiser settings were set by hand during development; no record of a systematic search over them was kept. The table gives each value and the reasoning it follows; the number of epochs is the one value checked against validation curves.

parameter	value	how it was obtained
epochs	20, last-epoch weights	the per-epoch validation curves (Table 12) still rise or flatten at epochs 16–20 in every year, so training runs a fixed 20 epochs with no early stopping
hidden width, unit embedding, attention heads	96, 16, 4	set by hand during development; keeps the network small (244,161 parameters) next to ∼41,000 labelled unit-weeks
learning rate and schedule	$2 \cdot 10^{-3}$, one-cycle, 15% warm-up	set by hand; standard settings for AdamW on a model of this size
weight decay, dropout, gradient clipping	10^{-2}, 0.1, norm 1.0	set by hand; conventional regularisation for a recurrent network
batch size	32 cases	set by hand; one case is already 14×168 outputs
input scaling	median, 10–90% range, clip ±8	robust to heavy tails: prices reach 898 EUR/MWh and margins swing by hundreds within a week
seeds	0, 1, 2; mean of probabilities	averaging seeds reduces the variance of a single network

Table 17: Hyperparameters of Short notebook 2.

Constants of the LP. The optimiser in Short notebook 1 also contains fixed numbers that are model constants rather than learning hyperparameters: unit conversions ($K = 0.0036\,\mathrm{Mm^3}$ per $\mathrm{m^3/s}$ for one hour; $P = 9.81 \cdot 10^{-3}\,HQ\eta\,\mathrm{MW}$); deficit penalties of 5,000 EUR per $\mathrm{m^3/s{\cdot}h}$ of missing minimum flow and 10^5 EUR per $\mathrm{Mm^3}$ of missing minimum volume, set by hand so that

the constraints are expensive to break but every case stays feasible (not tuned); and the three conventions measured from the labels (Section 4.2). All physical data (curves, costs, limits) is read from the topology file.

7 Model interpretation

7.1 Track A: what the trees learned

The pruned 15 features are all economic quantities — spreads, margins, break-even hours, the share of capacity a spread justifies — with topology used only to pick *which* water value and *which* capacity; no raw price, storage, inflow or calendar column survived (log 017). The single most important feature of the early model, by a factor of 14, was the binary "price > system water value": SHOP's dispatch condition, rediscovered (Figure 19). In the count model the top feature was the system storage change over the week — released water, i.e. production — and the binding-constraint term min(hours of water, hours profitable) (logs 021–022). Hourly-varying features held 96% of the gain: micro-AUC rewards ranking hours *within* a week, which a case-level constant cannot do (log 013).

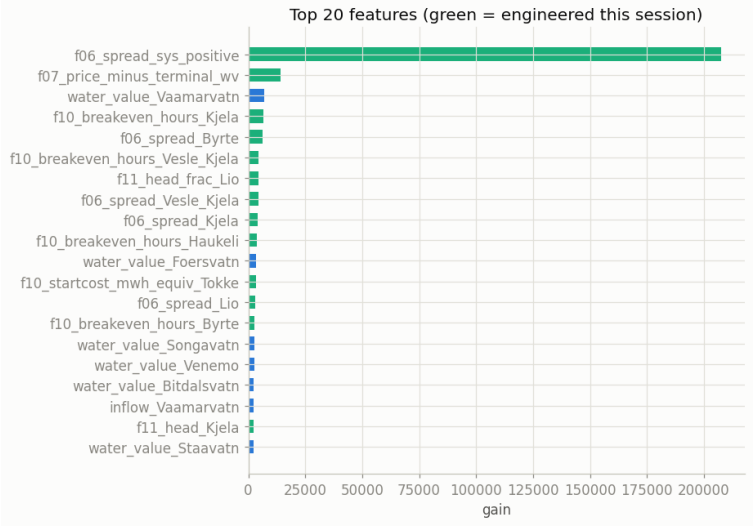


Figure 19: Track A, first feature iteration: gain importance. The binary spread sign — SHOP's own dispatch condition — dominates. (Log 013.)

7.2 Track B: the model leans on the optimiser

Ablation. Removing one feature group at a time on the same folds (Figure 20): without the LP features the same learner scores 0.9483 — *below* Track A — so the gain is neither LightGBM nor the feature engineering. The LP features plus unit identity alone reach 0.9897. Removing the observed storage trajectory changes nothing (0.9910 vs 0.9912): the model uses only information available before the week starts, unlike Track A's best features.

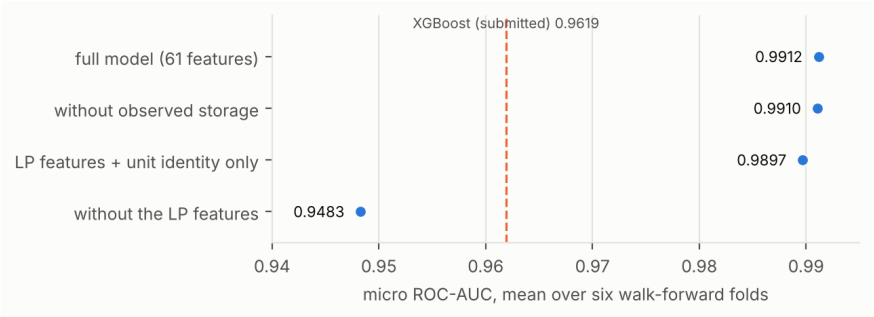


Figure 20: Ablation: mean walk-forward micro-AUC with one feature group removed.

Importance. The LP commitment and flow alone carry 71% of the split gain, LP features 86% in total (Figure 21). Unit identity (5%) is how the model learns per-unit corrections — e.g. that SHOP runs Lio harder than our LP does.

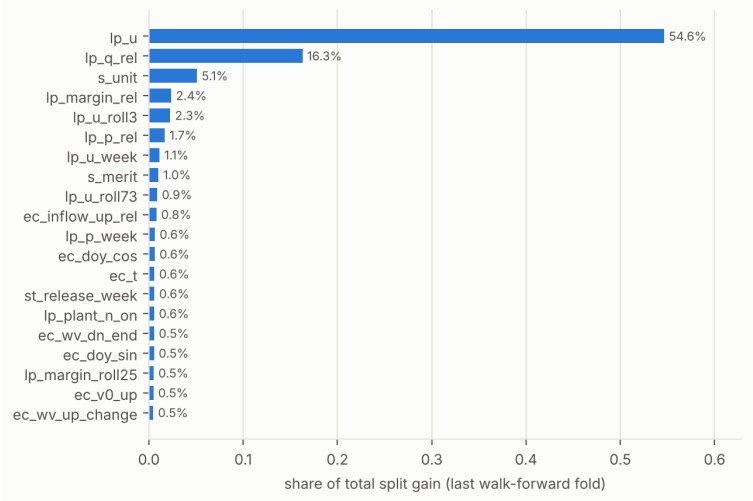


Figure 21: LP + LightGBM: share of split gain, top 20 features (last walk-forward fold).

Contributions. Split gain counts how much a feature is used in training, not how much it moves a prediction. TreeSHAP contributions (LightGBM's `pred_contrib`) on 20,000 validation rows of the test-like 2021-H2 fold (fold AUC 0.9914) give the per-prediction view (Figure 22). The LP commitment u moves the log-odds by 2.05 on average, nearly twice any other feature. Unit identity is second (1.14): it is the channel for per-unit corrections. Third is the margin, price minus the local water value from the LP duals (0.93), with its 7-hour rolling mean close behind: the learner reads how far each hour sits from the dispatch threshold, exactly the graded information a binary schedule discards (Section 4.2). By group, LP features carry 70% of the total absolute contribution, economics 15%, unit identity and static attributes 13%, the observed storage trace 2% and the constraint series 1%. Economics weigh more here than in split gain (15% against 6% of the gain in the last fold): many small adjustments rather than a few decisive splits.

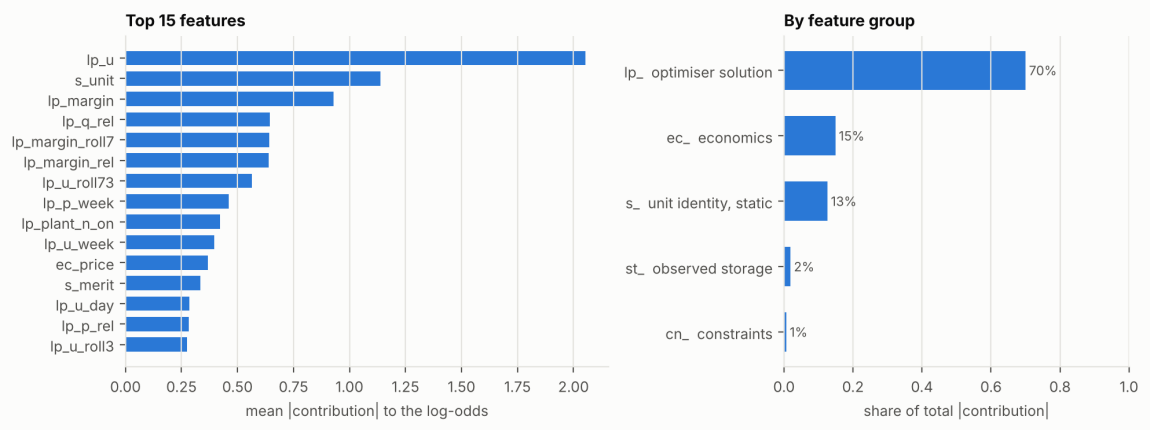


Figure 22: LP + LightGBM, 2021-H2 fold: mean absolute TreeSHAP contribution to the log-odds, top 15 features (left) and share by feature group (right).

How the learned layer reads the LP. The relaxation's u is already roughly a probability (Figure 23): SHOP commits in about 1% of hours where $u = 0$ and 94% where $u = 1$, and the on-rate rises close to the diagonal in between. The departures are unit-specific, which is why unit identity ranks second. Lio, which SHOP runs to meet the Vest-Vassdraget minimum flow, is committed in every hour where the LP gives it $u \geq 0.6$ and in three quarters of hours at 0.4–0.6. Hogga, whose timing is degenerate in our LP when Bandak must fill, is committed in only 45% of hours at u between 0.6 and 0.8.

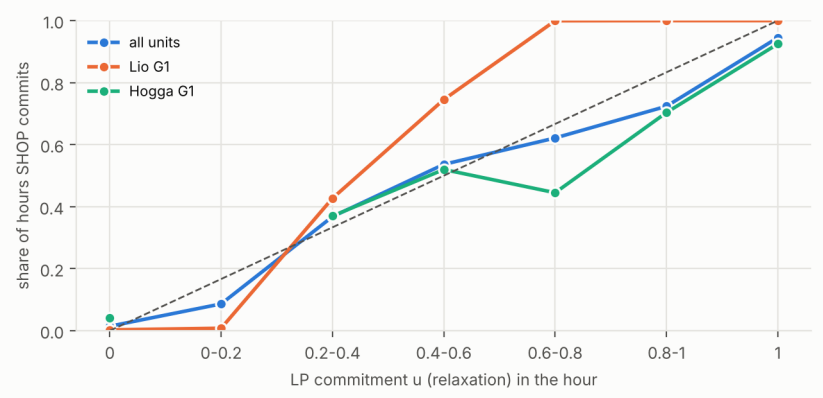


Figure 23: Share of hours SHOP commits, by the LP relaxation's u in that hour (2021-H2 fold). Dashed: perfect calibration.

One week. Figure 24 shows a test-like case the LP finds harder than three quarters of its fold. The relaxation spreads Tokke's commitment fractionally over four identical units and holds Lio at $u \approx 0.45$ until day 3.5. The learned layer concentrates Tokke on G1, the unit SHOP picks first, and sharpens Lio into the on-block SHOP chose; its within-case AUC is 0.999 against the LP's 0.974. Its remaining error is inherited from the LP: Kjela, on all week in SHOP's schedule, dips where the relaxation dips (days 4.4–5.6).

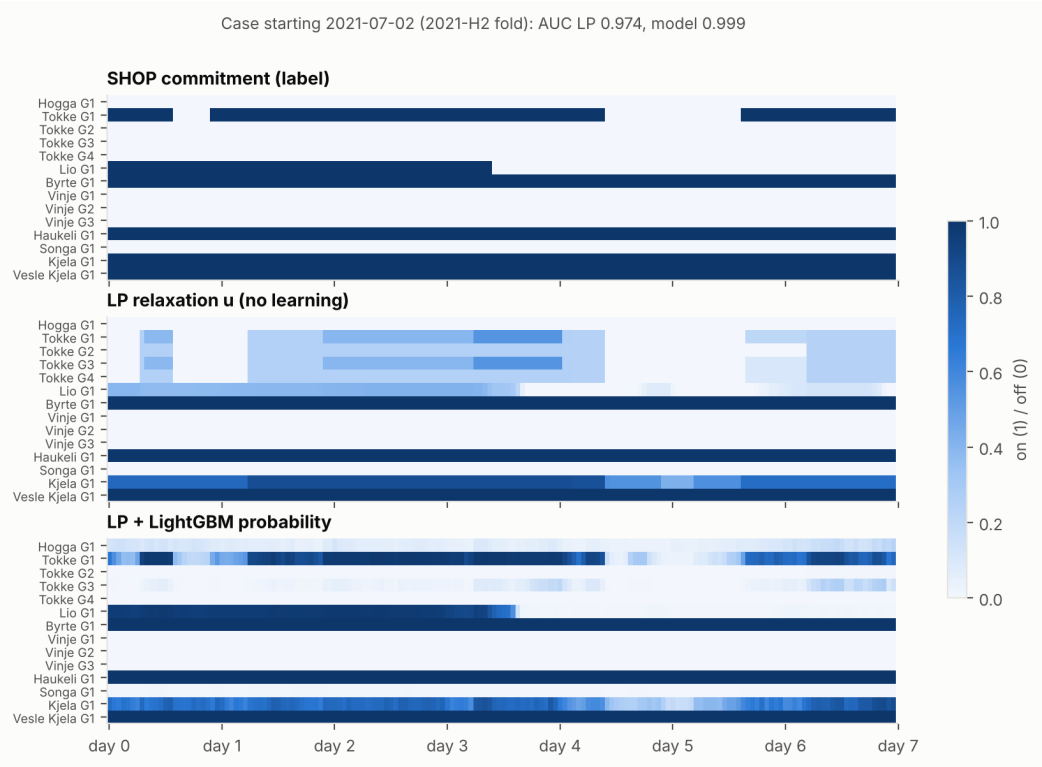


Figure 24: Case starting 2021-07-02: SHOP's commitment (top), the LP relaxation with no learning (middle) and the LP + LightGBM probability (bottom), one row per unit, one column per hour.

The optimiser itself is interpretable. Its decisions are explained by quantities with physical meaning: each unit runs when the price exceeds its local water value, which equals its reservoir's terminal water value unless a constraint binds — a full reservoir lowers it (water would otherwise spill), a minimum-flow obligation can force a unit on below it, an empty reservoir raises it. The three conventions of Section 4.2 are themselves findings about how the labels were produced.

7.3 Track C: what the physics features contribute

The network's own ablation (Section 4.3) is its interpretation result: removing the 16 physics features costs 0.004 AUC overall and 0.014 on 2021, the year that most resembles the test period. The features that carry the gain are the ones that encode the same economics as Track B's LP — break-even prices from the efficiency curves, the weekly water budget and block profit against start costs — computed per plant in closed form instead of jointly by a solver.

8 The two selected predictions

	Short notebook 1	Short notebook 2
model	LightGBM on LP features (Track B)	sequence neural network (Track C)
Kaggle public score	0.99522	0.99013
validation	walk-forward 0.9912 (6 half-year folds)	yearly CV 0.9740 (2019–2022)
trains on	all 2,922 labelled cases	all 2,922 labelled cases, 3 seeds
runtime, 4 CPU cores	∼2 h (3,647 LPs) + 6 min	∼2 h (1.9 min per epoch); 32 min measured on an Apple M4 GPU
peak memory	∼5 GB	∼8 GB
reproduces the uploaded file	to $2 \cdot 10^{-13}$	features exactly; re-run correlates 0.9993 with it (training is not bit-reproducible on a GPU)

Table 18: The two predictions selected on Kaggle. Both notebooks start from the raw data, run offline and stay within the course limits (12 hours, 4 cores, 32 GB).

9 Discussion

Why Track B works. Track A spent most of its effort recovering, through features, a decision process that is written down: the spread is the dispatch condition, the count is the weekly water budget, the block structure comes from start costs, the merit order from penstocks. Solving the actual problem gets all of these at once, consistently across units and reservoirs, and extrapolates to unseen price regimes (negative prices, the post-crisis level) from physics rather than from training data. The learner is left with a smaller, more stationary job: the residual between our simplified replica and SHOP.

Relation to the task's purpose. InterOpt wants ML to speed up the optimiser. Our best model still solves an LP, but one roughly 6–18× cheaper than SHOP's recorded solve times, and its output is a calibrated commitment probability that could warm-start SHOP's MIP. Unlike Track A's best features, it needs no post-decision information.

Limitations and next steps. The remaining errors are fidelity gaps of the replica: heads frozen at their initial level, Lio's min-flow behaviour, weeks where SHOP runs units below their water value (e.g. 2019-07-09), and degenerate weeks where a reservoir must fill and the timing of release is arbitrary. Each fix to the replica feeds the learned layer directly. Three conventions were read off label statistics that include the validation years; they are structural and stable across years, but the protocol is not blind to them.

What we would do differently. Read the topology file on day one. Track A's three largest late gains were corrections found by reading the documentation; Track B is the same lesson taken to its conclusion.

A Reproducibility

- **Configuration.** `requirements.txt` next to the notebooks pins the package versions both notebooks were run with (Short notebook 2 adds PyTorch). The notebooks contain all code, start from the raw competition files (`data/kernel`, `data/extended`,

`sample_submission.csv`, `prediction_mapping.csv`), need no network and write only intermediate results (the LP cache) and the submission file.

- **Short notebook 1** (LP + LightGBM): solves the 3,647 LPs in parallel (∼2 h on 4 cores, cached after the first run), builds features, trains on all labelled cases and writes the submission (∼6 min, peak ∼5 GB RAM). Executed end to end from the raw data, it reproduces the uploaded file to $2 \cdot 10^{-13}$, and its LP solutions are byte-identical to the stored ones.
- **Short notebook 2** (neural network): builds the 99 features from the raw files (about 1–2 minutes), checks them against values recorded from the original run, and trains three seeds on all labelled cases. Its 99 features match the original run's checksums (mean and SD of every feature on train and test) to the printed precision. Training on a GPU is not bit-reproducible across machines, so a re-run gives a prediction close to, not identical with, the uploaded file; the notebook's last cell measures the difference when that file is present. Executed end to end on an Apple M4 GPU (MPS), it built the features in 16 s and trained the three seeds in 32 min (27–54 s per epoch); its output correlates 0.9993 with the uploaded file, 99.4% of cells fall on the same side of 0.5, and the largest single difference is 0.33. The total of 84 min printed in the executed notebook includes about 52 min during which the machine was asleep, visible as three slow epochs of seed 1. The notebook uses CUDA, then Apple MPS, then the CPU; no GPU is needed: on 4 CPU cores training takes about 2 h (1.9 min per epoch for 3 seeds × 20 epochs).
- **Track A notebook** (`notebooks/Track A XGBoost (0.98058).ipynb`, not selected): rebuilds our first submission from the raw files (about 40 min on 4 cores; peak ∼24 GB of memory including swap). The uploaded file was produced on an Apple Silicon Mac; on x86 Linux with the same package versions the count model selects 143 count features instead of 141, and the output correlates 0.995 with the uploaded file.
- Source modules, every experiment and its result: `src/`, `report/leaderboard.md`, `report/runs/`.

B Full lab log

The chronological record of every session, including all experiments and negative results, exactly as written during the project.

B.1 2026-09-21 — Problem framing and data overview

Task.

SINTEF Energy's InterOpt project: predict, for each of 14 generating units in the Tokke–Vinje hydropower system (17 reservoirs, 9 plants, southern Norway), the probability that the unit is ON at every hour of a 168-hour (one-week) rolling scheduling horizon. Ground truth comes from a SHOP optimization run. Submissions are scored on Kaggle with **micro-average ROC-AUC**. The intent (per the task PDF) is for a trained ML model to approximate what the optimizer would decide, to speed up real hydropower scheduling.

Case structure.

The full problem spans 3,647 daily rolling cases (2015-01-01 to 2024-12-25). Labels only exist for the first 2,922 cases (2015-01-01 to 2022-12-31, `data/kernel/Unit_commitment_decisions.csv`); the remaining 725 cases (2023-01-01 to 2024-12-25, `prediction_mapping.csv`) are the held-out Kaggle test set.

Data layout.

Two source folders, kept as raw, unmodified files (see Section B.3 for why we don't leave them scattered when actually modeling):

- `data/kernel/` (core, required): hourly day-ahead price, hourly reservoir/river inflow (17 reservoirs + 1 river), daily reservoir storage, daily marginal water value, and the ground-truth unit commitment decisions.
- `data/extended/` (optional context): watercourse topology (`Tokke_Vinje_topology.yaml`, SHOP-format graph of reservoirs / rivers / tunnels / plants / generators), a topology diagram PDF, and hourly environmental constraints (minimum reservoir volume, minimum river flow).

Finding. No missing values in any core or extended file; reservoir naming is consistent across files; `Constraint_min_volume/min_flow` reservoirs/rivers are a strict subset of the full 17. The data is clean going in.

Finding. `total_calculation_time` (SHOP solver runtime) is present in the training labels file but is solver metadata, not a feature — it does not exist for the test cases we need to predict. It must be excluded from any feature pipeline to avoid leakage.

B.2 2026-09-21 — Exploratory data analysis

Full notebook: `src/EDA/eda.ipynb`. Figures below are extracted from its executed outputs; see `figures.md` for exact provenance per figure.

Target: per-unit imbalance and correlation structure.

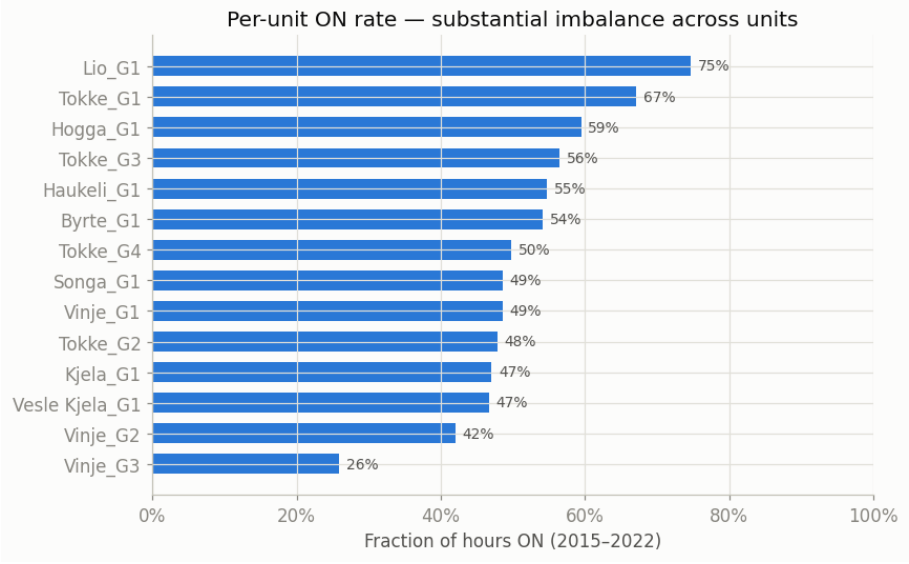


Figure 25: Per-unit ON rate, 2015–2022.

Finding. ON rate varies from 26% (Vinje_G3) to 75% (Lio_G1) across units (Figure 25) — this is not one balanced binary problem, and per-unit performance should be tracked separately, not just the pooled micro-AUC.

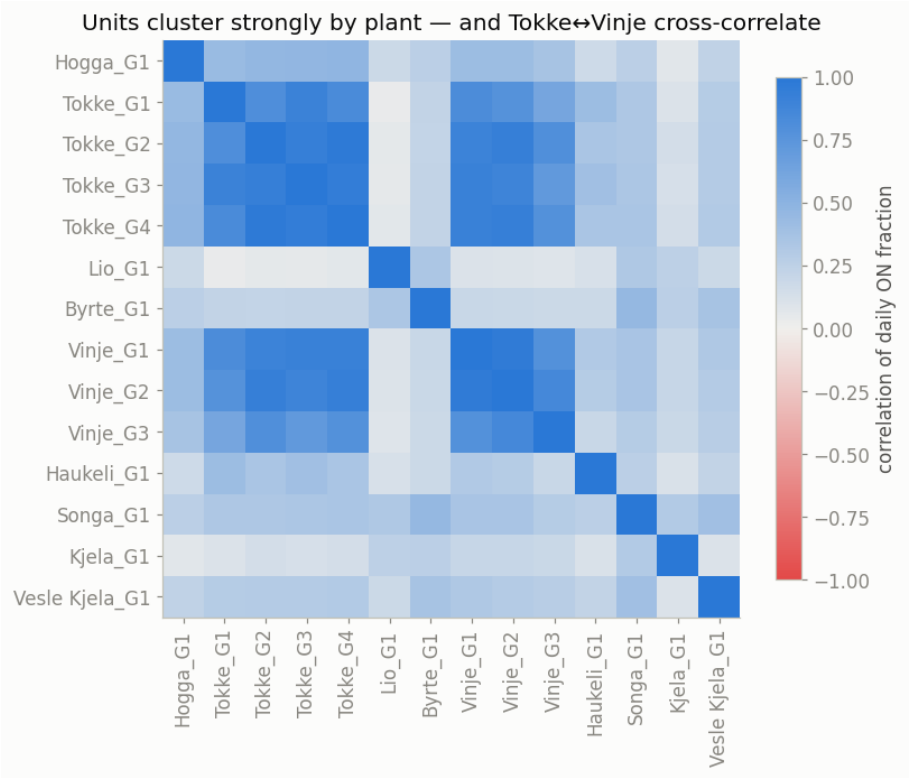


Figure 26: Correlation of units' daily ON fraction, ordered by plant.

Finding. Units at the same plant correlate strongly (0.8–0.96). More notably, Tokke and Vinje — different plants, different immediate reservoirs — also cross-correlate at 0.8–0.9 (Figure 26). This turned out to be explainable by the watercourse topology: Vinje's reservoir (Vaamarvatn) cascades downstream into Vinjevatn, which is exactly the reservoir Tokke draws from. This motivates modeling units jointly rather than as 14 independent problems.

Seasonality.

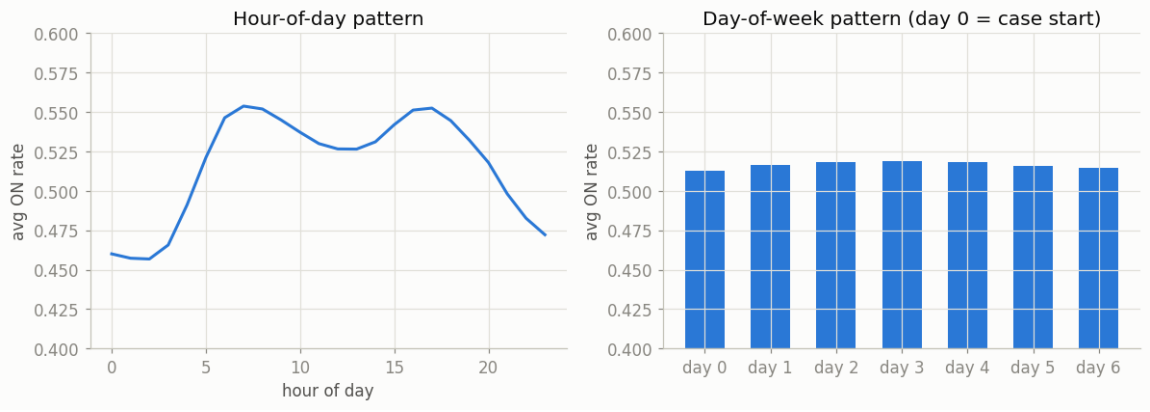


Figure 27: ON rate by hour-of-day and by day-of-week within a case.

Finding. Strong intraday seasonality (double-hump pattern, trough ≈0.46 at night, peaks ≈0.55 at the morning/evening demand ramps) but weak day-of-week seasonality (0.51–0.52 across all 7 days) — see Figure 27. `hour_of_day` is a much more informative time feature than `day_of_week` here.

Yearly trend and the 2022 outlier.

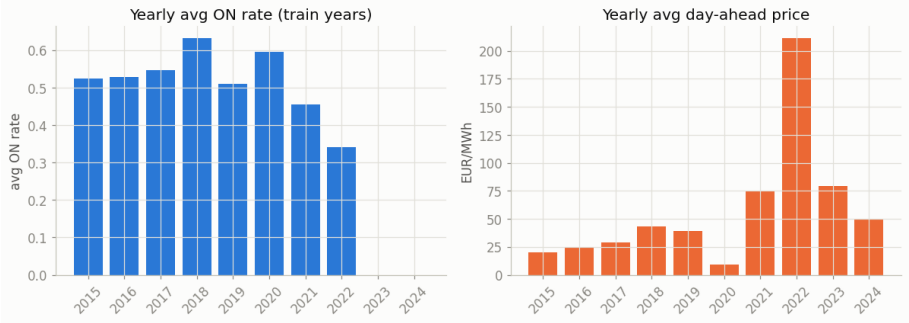


Figure 28: Yearly average ON rate vs. yearly average day-ahead price.

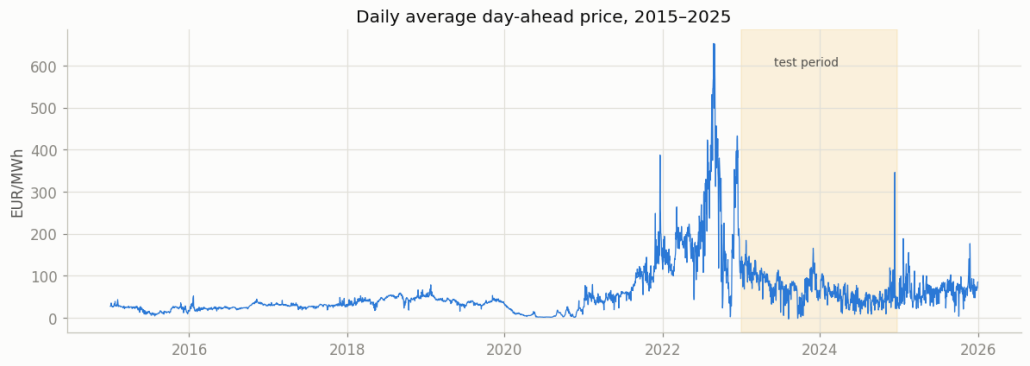


Figure 29: Daily average price, 2015–2025, with the 2023–2024 test window shaded.

Finding. 2022 (European energy crisis) has the *highest* average price in the series (∼211 EUR/MWh) but the *lowest* average ON rate (0.34) — see Figure 28. The naive "high price ⇒ more generation" intuition does not hold at the year level (explained further below). The 2022 spike sits entirely in the training window; the test window (2023–2024, shaded in Figure 29) is elevated relative to 2015–2020 but nowhere near the 2022 extreme — a distribution-shift risk that motivated a chronological (not random) validation split for modeling.

Physical drivers: inflow, storage, water value.

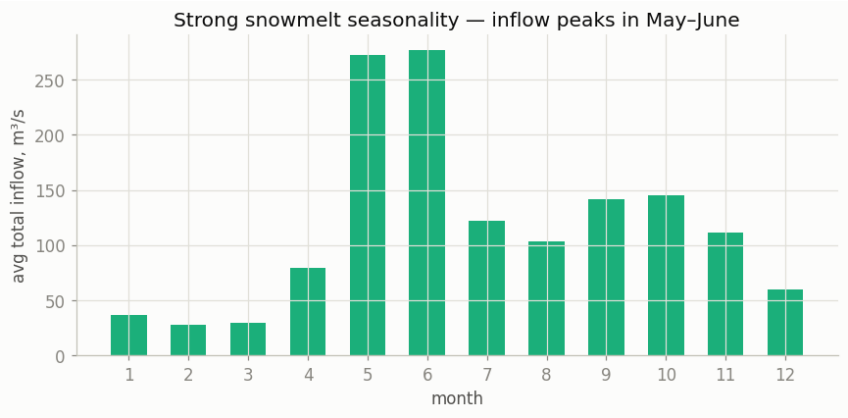


Figure 30: Average total reservoir inflow by calendar month.

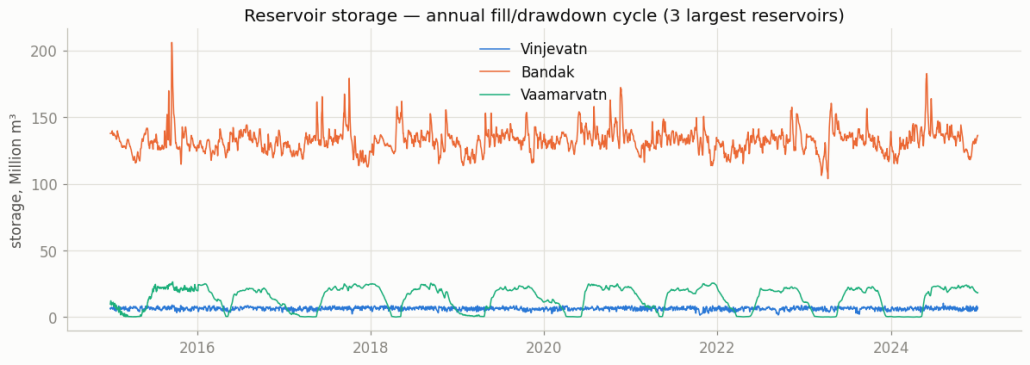


Figure 31: Daily storage, three largest reservoirs.



Figure 32: System-average water value vs. daily average price.

Finding. Classic Norwegian snowmelt seasonality: total inflow across all 17 reservoirs is ∼8× higher in May–June than in the Jan–Feb trough (Figure 30), driving the annual storage fill/drawdown cycle (Figure 31). System-average water value tracks price but is much smoother (Figure 32) — it reflects the expected *future* value of stored water, not the instantaneous spot price.

The headline finding: relative price, not absolute price.

Finding. For every single unit, correlation with ON status flips from weak/negative (raw price level) to consistently positive (∼0.15–0.25) when price is normalized *within each case's own 168-hour window* (Figure 33). The optimizer reacts to "is this hour relatively good within this week's water/price outlook," not to absolute price. High-price years like 2022 also tend to be scarce-water years, where the optimizer conserves water and runs fewer units despite high prices — explaining the year-level negative relationship in Figure 28. Implication: within-case normalized price and a `price − water_value` spread are promising engineered features, likely more robust to the 2022-style regime shift than raw levels.

Extended data.

The topology YAML gives an explicit plant → upstream-reservoir map (used above to explain the Tokke/Vinje correlation) and defines the wider reservoir/river/ tunnel graph. The environmental constraint files are sparse and seasonal — only a handful of reservoirs/rivers are ever constrained, and only part of the year — suggesting a binary "currently constrained" flag would be a more useful encoding than the raw (mostly-zero) values, if we decide to use the extended data at all.

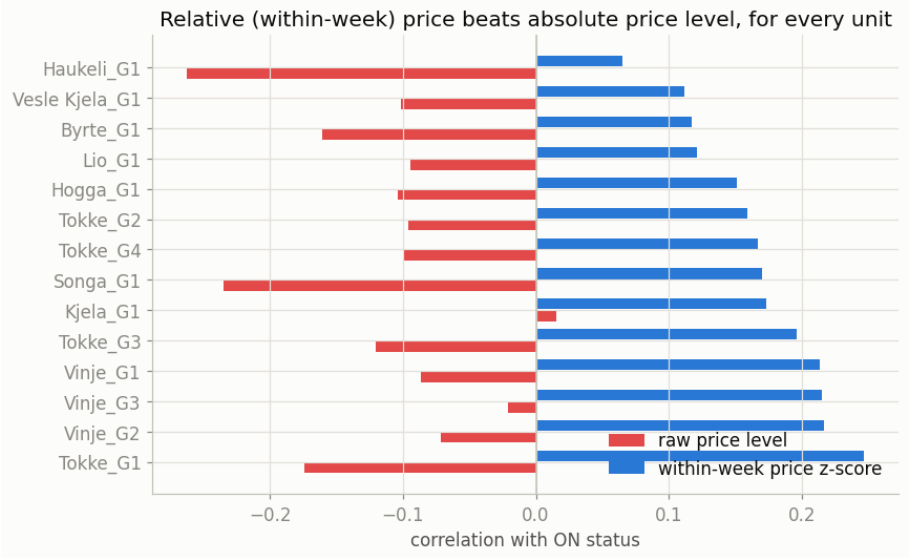


Figure 33: Per-unit correlation of ON status with raw price vs. within-case price z-score.

B.3 2026-09-21 — Centralizing the scattered data into a modeling table

Motivation.

The raw source files are scattered across several CSVs at different native granularities (hourly price/inflow, daily storage/water value, one row per 168-hour case for labels). Keeping them separate is fine, and good practice, as the unmodified source of truth. But a supervised model needs one row = one example, features..., target at the end — so before any modeling, everything is joined once into a flat table, which also forces explicit, one-time decisions about how sources at different granularities align to a row and where the train/test boundary falls.

Design decisions (discussed and agreed before building).

- **Row granularity: one row per (case, hour).** Considered alternatives: one row per (case, hour, unit) (long format, ∼6.9M rows, simplest per-row model but duplicates case/hour features 14× and drops the multi-unit correlation structure from Section B.2), and one row per case (fully wide, 2,922 rows but 2,352 target columns, awkward for standard tabular models). Case+hour keeps a manageable row count while keeping all 14 units' targets on one row, so a model can see them jointly if we want it to.
- **Raw joins only, no engineered features.** No normalization, no lags, no derived columns at this stage — feature engineering (e.g. the within-case price normalization from Section B.2) is treated as a separate, explicit step.
- **Core dataset (`data/kernel/`) only.** Extended data (topology, constraints) intentionally left out until there's a concrete reason to use it.
- **Per-reservoir columns kept separate**, not pre-aggregated (17 inflow + 17 storage + 17 water-value columns, individually named), so nothing is thrown away before we've decided how to use it.

Implementation.

`src/build_modeling_table.py`. Daily-native features (storage, water value) are repeated across each day's 24 hours since they're only observed once per day. Targets are reshaped

from the labels file's wide format (`result_committed_<unit>_t<hour>`) into one `on_<unit>` column per unit. Output:

- `data/processed/train_hourly.parquet` — 490,896 rows (2,922 cases × 168 hours) × 72 columns (5 id/time columns, 53 raw features, 14 targets).
- `data/processed/test_hourly.parquet` — 121,800 rows (725 cases × 168 hours) × 58 columns (same features, no targets).

Finding. Spot-checked the join against raw source files (a mid-case UC label, a price value, a storage value at a specific timestamp) — all matched exactly. No missing values after joining any feature column.

B.4 2026-09-21 — Baseline: 14 independent XGBoost models

Goal.

Get a first read on how hard the problem is, and where the weaknesses are, before investing in feature engineering or a more complex architecture. Full notebook: `src/modeling/baseline_xgboost.ipynb`.

Design decisions (discussed and agreed before running).

- **Chronological validation split**, not random: train on 2015–2021, validate on held-out **2022** in full. A random split would let the model see neighboring hours from the same case/week, inflating the score; holding out 2022 whole also directly tests robustness to the price-crisis regime shift identified in Section B.2.
- **One independent XGBoost binary classifier per unit** (14 models total), each trained on the same 54 raw features from `train_hourly.parquet` (`hour` + `price_eur_mwh` + 18 inflow + 17 storage + 17 water_value columns) — the simplest architecture that matches the table as built.
- **Metric**: the competition's own `kaggle_metric.py` (micro-average ROC-AUC), applied for real by reshaping validation predictions back into the competition's wide submission format, plus a per-unit AUC breakdown the pooled metric hides.

Split sizes.

Fit: 2,557 cases / 429,576 rows (2015–2021). Validation: 365 cases / 61,320 rows (2022).

Results.

Finding. Overall micro-AUC of 0.647 — a workable but mediocre first baseline. Per-unit AUC spans 0.50 (near-random, Tokke_G4) to 0.75 (Byrte_G1). The weakest units are almost exactly the multi-unit Tokke (4 units) and Vinje (3 units) group — the same units flagged as strongly cross-correlated in Section B.2. Hypothesis (not yet tested): at a multi-unit plant, *how many* units are on may be easier to predict from price/water data than *which specific* unit(s), and 14 independent per-unit models have no way to share that information.

Finding. Feature importance (Figure 35) is smeared across many similar reservoir columns rather than concentrated — consistent with the observation that every unit's model currently sees all 53 raw features (all 17 reservoirs), not just the ones plausibly relevant to that unit's plant.

Table 19: Per-unit ROC-AUC on the 2022 validation set, baseline model.

Unit	AUC
Byrte_G1	0.751
Vesle_Kjela_G1	0.724
Lio_G1	0.713
Songa_G1	0.677
Hogga_G1	0.655
Kjela_G1	0.619
Vinje_G3	0.580
Tokke_G1	0.577
Haukeli_G1	0.564
Tokke_G2	0.557
Tokke_G3	0.528
Vinje_G2	0.524
Vinje_G1	0.519
Tokke_G4	0.504
Overall (micro-average, official metric)	**0.647**

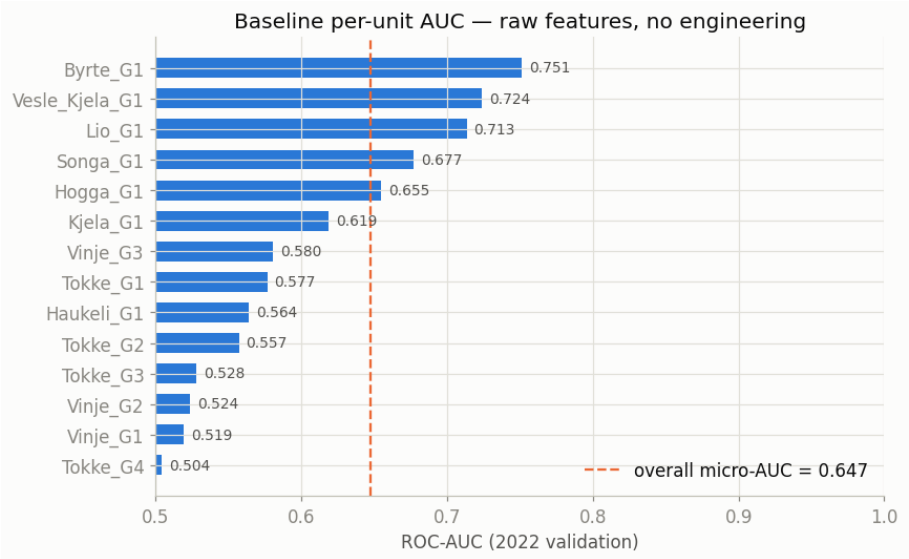


Figure 34: Per-unit AUC, baseline model, 2022 validation.

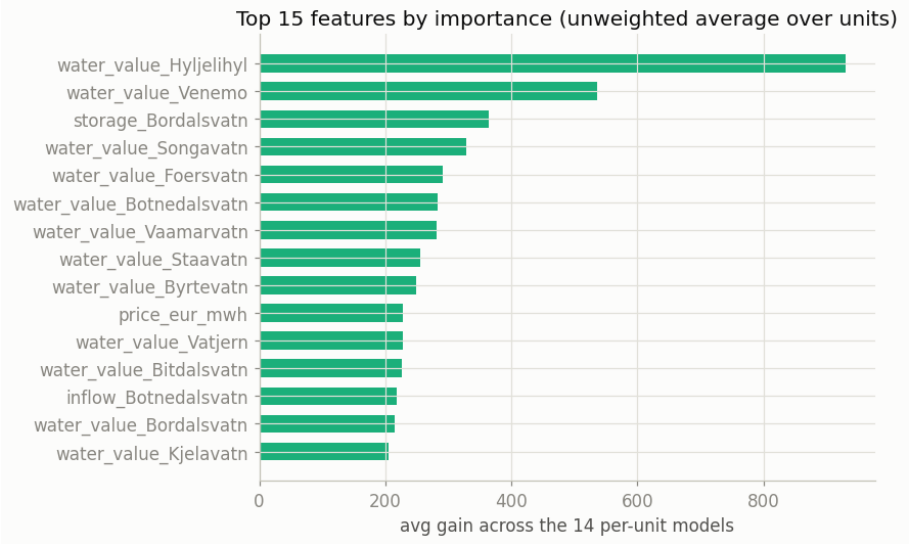


Figure 35: Top 15 features by average gain across the 14 per-unit models.

This result (and every future one, scored the same way) is tracked in `report/leaderboard.md` as it's produced, per `report_rules.md`.

B.5 2026-09-21 — Discussion: graph structure, joint modeling, and next baseline

The Tokke/Vinje weak spot in Section B.4 prompted a discussion of whether the watercourse topology (Section B.2) should be used more directly, e.g. via a graph neural network over the reservoir/plant graph.

GNN considered and set aside for now.

The topology does explain the Tokke/Vinje correlation physically (Vinje's reservoir cascades into Vinjevatn, which Tokke draws from), so a graph-based architecture is conceptually reasonable. But: the graph is small (∼22 nodes) and *identical* at train and test time, so a GNN's main advantage — generalizing to unseen graph structure — doesn't really apply here. It would also be a large implementation jump (new framework, per-case graph batching, a training loop) relative to what's been built so far.

A premise checked and corrected.

The natural cheaper alternative — adding each plant's upstream-reservoir features explicitly — turned out to already be moot: the current modeling table (Section B.3) already joins *all* 17 reservoirs' data into every row, so every unit's baseline model already had access to its neighbors' data. The real gap is therefore not missing input features, but that the 14 units are modeled **independently** and can't share information about each other's predictions — a correlated-*output* problem, not a missing-*input* problem.

Multi-output architectures considered.

To model 14 correlated binary outputs without exploding the output space (treating the full ON/OFF vector as one $2^{14} = 16{,}384$-way categorical label was explicitly ruled out — almost all combinations never occur, and it discards the partial-credit nature of AUC scoring):

- **Multi-output trees**: a single boosted model where each leaf stores a vector of 14 probabilities instead of one scalar, with splits chosen to jointly improve all 14 outputs (XGBoost's `multi_strategy="multi_output_tree"`). Leaf count stays linear in tree depth, not exponential in the number of units.
- **Classifier chains**: feed earlier units' predicted probabilities as extra input features to later units' models, in some order. Simple to bolt onto the existing per-unit XGBoost setup.
- Briefly noted: a shared-trunk neural net with 14 sigmoid output heads (bigger jump from the current XGBoost-based approach; not pursued yet).

Decision.

Explicitly *not* anchoring to the earlier plan of testing the cheapest hypothesis first (Section B.2's within-case price normalization) before trying anything else — this is an early stage of a multi-week project, and it's fine to resequence as better questions surface. Next step agreed: build a **multi-output tree** model as the next baseline and compare directly against the 14-independent-model baseline in Section B.4, using the same validation protocol (train 2015–2021, validate on 2022, score with `kaggle_metric.py`).

B.6 2026-09-21 — Baseline 2: joint multi-output tree

Goal.

Test directly whether the Tokke/Vinje weakness in the baseline (Section B.4) is a correlated-*output* problem, as hypothesized in Section B.5. Full notebook: `src/modeling/multioutput_tree.ipynb`.

Design.

Everything held identical to baseline 1 — same 54 raw features, same hyperparameters (`n_estimators=200`, `max_depth=6`, `learning_rate=0.1`, subsample/colsample 0.8), same chronological split (train 2015–2021, validate on held-out 2022), same `kaggle_metric.py` scoring — with exactly one change: a single `XGBClassifier` with `multi_strategy="multi_output_tree"`, trained on all 14 targets jointly, instead of 14 independent single-output models. Each leaf of each tree stores a vector of 14 probabilities; split choices are optimized jointly across units. This isolates the independent-vs-joint modeling question specifically, with no other confound.

Results.

Finding. Overall micro-AUC improves from 0.647 to **0.687** (+0.040), and confirms the hypothesis from Section B.4: the units that improve *most* are almost exactly the previously-weakest multi-unit-plant units — all four Tokke units and all three Vinje units gain +0.03 to +0.13 AUC, with Hogga_G1 (topologically adjacent to both, per Section B.2) also improving substantially. Modeling the units jointly does let the tree exploit the cross-unit correlation that 14 independent models couldn't see.

Table 20: Baseline 1 (independent) vs. baseline 2 (joint multi-output tree), 2022 validation.

Unit	Baseline 1 (independent models)	Baseline 2 (joint multi-output tree)	Δ
Tokke_G3	0.528	0.656	+0.129
Tokke_G4	0.504	0.616	+0.112
Tokke_G1	0.577	0.687	+0.111
Vinje_G1	0.519	0.622	+0.102
Hogga_G1	0.655	0.745	+0.090
Vinje_G2	0.524	0.609	+0.085
Vesle_Kjela_G1	0.724	0.773	+0.049
Tokke_G2	0.557	0.598	+0.041
Vinje_G3	0.580	0.615	+0.035
Byrte_G1	0.751	0.784	+0.033
Kjela_G1	0.619	0.636	+0.017
Haukeli_G1	0.564	0.554	−0.010
Lio_G1	0.713	0.656	−0.057
Songa_G1	0.677	0.597	−0.080
Overall (micro-average, official metric)	**0.647**	**0.687**	**+0.040**

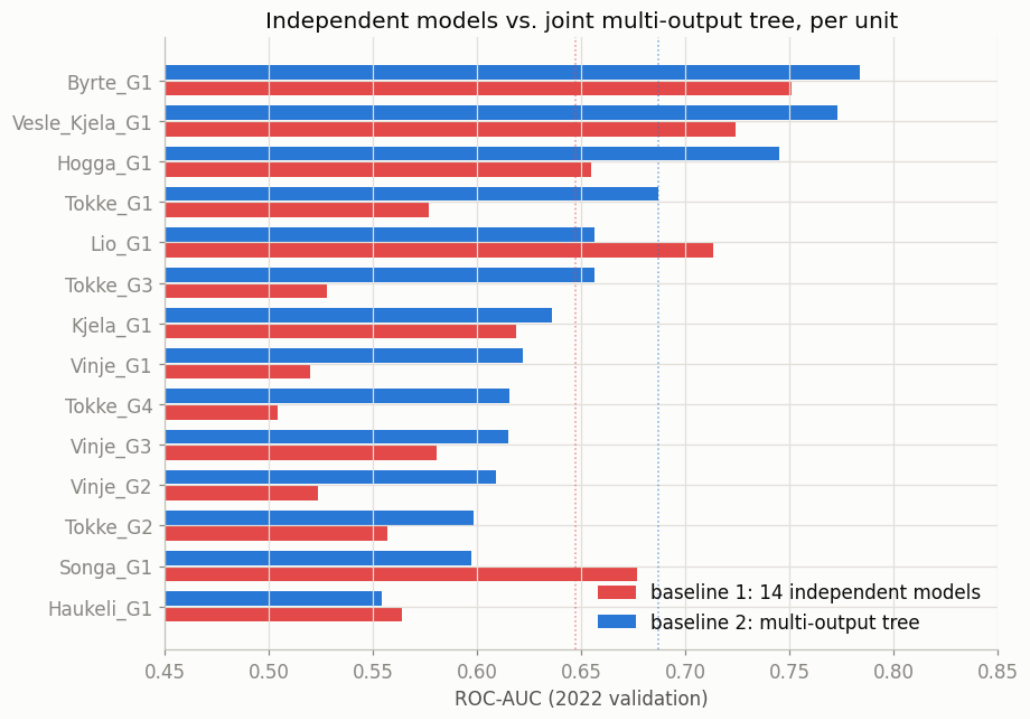


Figure 36: Per-unit AUC, baseline 1 vs. baseline 2, 2022 validation.

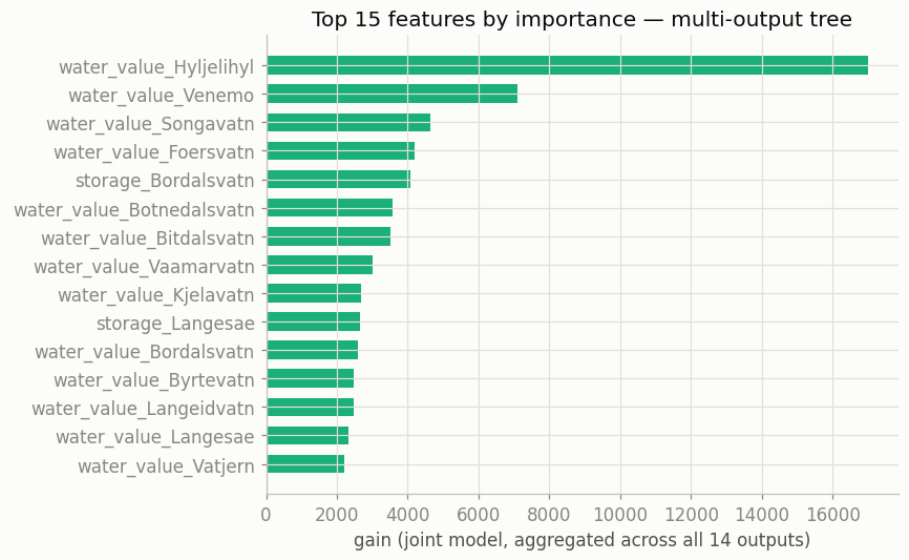


Figure 37: Top 15 features by gain, joint multi-output model.

Finding. Not a uniform win: **Songa_G1 and Lio_G1 got worse** (−0.080 and −0.057), and these were the two best-performing units in baseline 1. Working hypothesis, not yet tested: the joint model has the same total tree budget (200 trees) shared across all 14 outputs, i.e. roughly 14× less dedicated capacity per unit than baseline 1's 200-trees-*each* setup — units that didn't need cross-unit information to begin with may simply have lost modeling capacity to units that did. Worth revisiting (e.g. more estimators for the joint model) before treating this architecture as strictly better.

† **Superseded** — see Section B.7: this capacity hypothesis is tested directly, with a nuanced partial confirmation.

This result is tracked in `report/leaderboard.md` alongside baseline 1.

B.7 2026-09-21 — Does more capacity fix the multi-output model's regressed units?

Goal.

Section B.6 flagged, but did not test, the hypothesis that Songa_G1 and Lio_G1 regressed under the joint multi-output model because 200 trees shared across 14 outputs gives far less dedicated capacity per unit than baseline 1's 200-trees-*each*. Full notebook: `src/modeling/multioutput_tree_capacity.ipynb`.

Method.

Trained once to 3,000 trees (no early stopping; see below for why), identical features/hyperparameters to Section B.6 otherwise. Checked the official `kaggle_metric.py` micro-AUC at ∼45 checkpoints via `predict_proba(..., iteration_range=(0,k))`, which reuses the one trained model rather than retraining at every candidate tree count.

Finding. XGBoost's own multi-output `eval_metric="auc"` does *not* match the official micro-AUC — it tracks consistently lower and noisier (Figure 38). Almost certainly a macro-average across the 14 outputs rather than a pooled/micro calculation. Using it for early stopping in this multi-output setup would optimize the wrong target, which is why this run used a fixed generous cap and our own checkpointing instead.

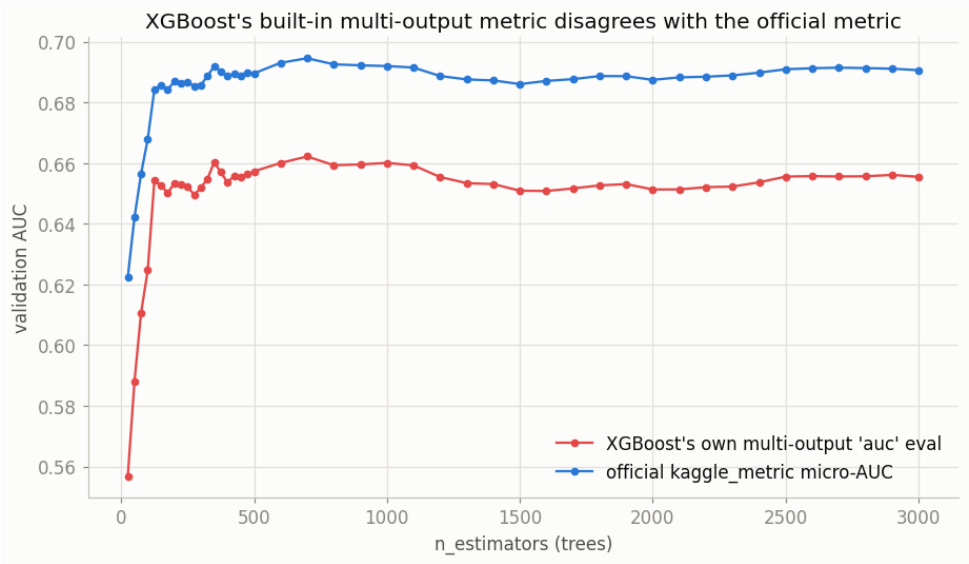


Figure 38: XGBoost's internal multi-output AUC vs. the official micro-AUC.

Results: a real plateau, then a real decline.

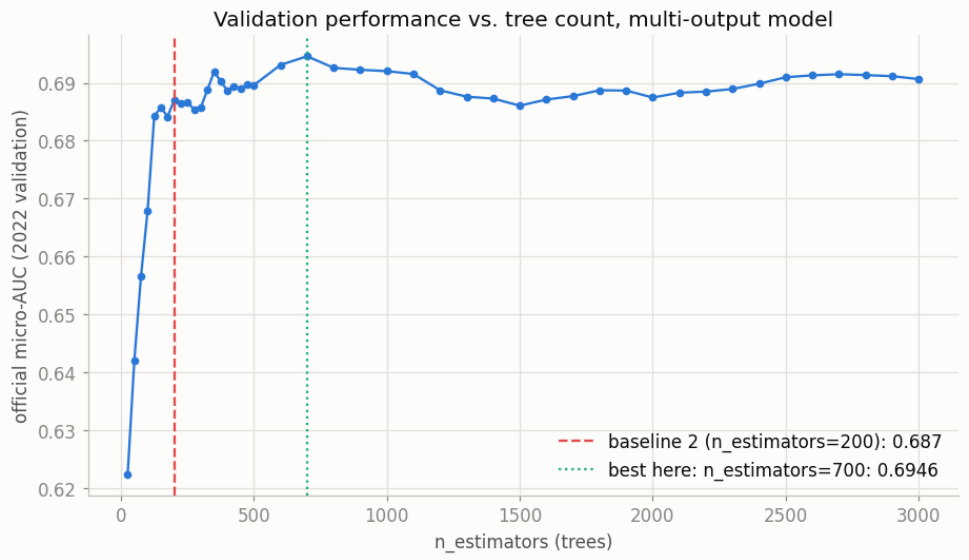


Figure 39: Official micro-AUC vs. tree count, up to 3,000 trees.

Finding. Validation performance rises sharply to ∼150 trees, keeps climbing gently to a peak at $n_{\textbf{estimators}} = 700$ **(micro-AUC 0.6946)**, then genuinely declines to ∼0.686 by 1,500 trees — a real ∼0.008 drop, not noise — before a mild, incomplete recovery out to 3,000 that never regains the 700-tree peak (Figure 39). $n_{\text{estimators}} = 700$ is a +0.0076 improvement over baseline 2's original $n_{\text{estimators}} = 200$ (0.687 → 0.6946), confirming the model was undertrained, not just differently structured.

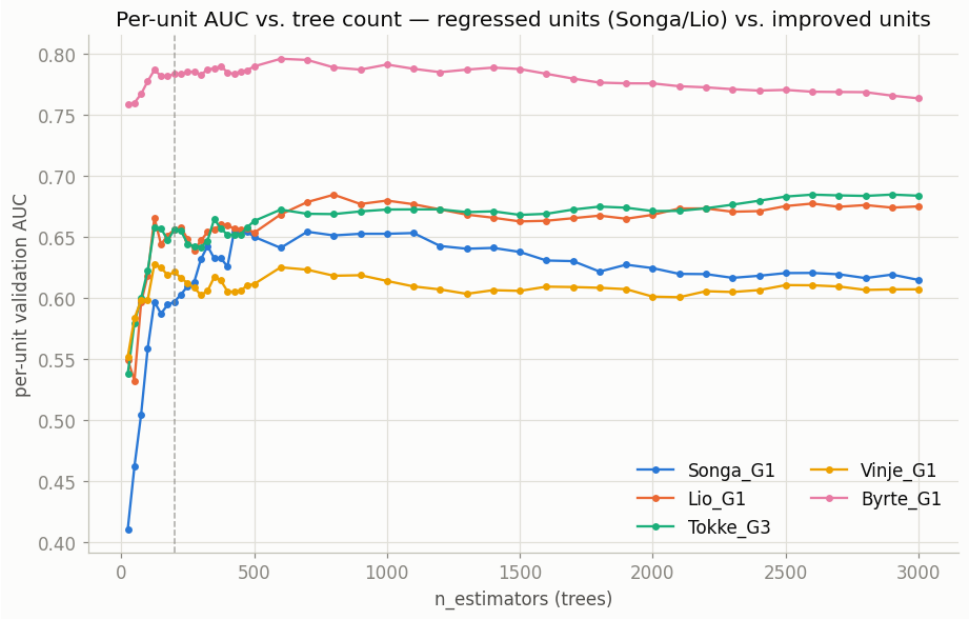


Figure 40: Per-unit AUC vs. tree count: regressed units (Songa_G1, Lio_G1) vs. improved/strong units.

Finding. The capacity hypothesis is only partially right. At $n_{\text{estimators}} = 700$, Songa_G1 recovers from 0.597 (at 200 trees) to 0.655, and Lio_G1 from 0.656 to 0.679 (Figure 40) — real improvement. But neither returns to its baseline-1 independent-model score (Songa_G1: 0.677, Lio_G1: 0.713) anywhere in the swept range; both plateau and then decline alongside the overall curve rather than continuing to close the gap. So: more capacity was genuinely part of the story, but something else about joint modeling still costs these two specific units relative to modeling them independently. Left as an open question for the next round of exploratory analysis rather than chased further here.

Decision: adopt $n_{\text{estimators}} = 300$, not the argmax.

The sweep's argmax is 700, but the whole region from ∼125 to ∼500 trees is a noisy plateau spanning only ∼0.006 AUC, measured on a *single* validation year. Selecting the argmax of a noisy validation curve is itself a form of overfitting — to the validation set rather than the training set — and 700 would likely not be the argmax on a different holdout year. We therefore adopt $n_{\text{estimators}} = 300$ as the standing default: inside the stable plateau, cheaper to train, and not tuned to a single noisy peak. Measured score at 300 is 0.6858, nominally 0.0012 below the $n = 200$ run — a difference well inside the noise of this plateau, and not treated as meaningful. Revisit once there is a more robust validation scheme (multiple holdout years or proper time-series CV) that could distinguish these points.

Updated in `report/leaderboard.md` as `xgb_multioutput_tree_n700`.

B.8 2026-09-21 — EDA round 2: what actually makes this problem hard

Round 1 (Section B.2) characterised the data. This round targets the structure of the prediction problem itself, to motivate features and architecture. Full notebook: `src/EDA/eda_round2.ipynb`. Twelve findings; the first four change how we should think about the task.

The labels are not self-consistent.

Cases are daily rolling, so consecutive cases overlap on 144 identical wall-clock hours (case N's hour $24{+}h$ is the same real timestamp as case $N{+}1$'s hour h), solved by SHOP from essentially the same physical state.

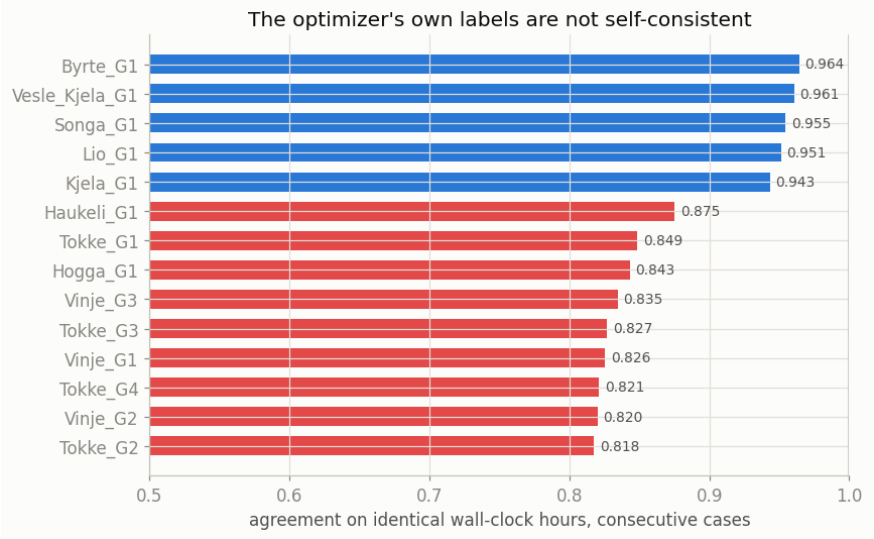


Figure 41: Agreement between consecutive cases on identical wall-clock hours.

Finding. (1) The same physical hour, re-solved one day later under a different 168-hour horizon, gets a *different* commitment decision **12% of the time overall, and ∼18% for the Tokke and Vinje units** (Figure 41). This is not measurement noise — it is the optimizer reacting to end-of-horizon boundary conditions that a wall-clock-indexed model cannot observe. It is an effective ceiling on achievable accuracy.

† **Superseded** — see Section B.9: the "effective ceiling" inference is withdrawn — the 12% measurement stands, but it does not bound AUC.

Finding. (2) Label self-agreement correlates $r = 0.88$ with baseline-1 per-unit AUC — about **78% of the variance in per-unit difficulty is explained by label noise alone**, not by our features (Figure 42). The correlation drops to 0.42 for the joint multi-output model, which is precisely the mechanism behind Section B.6: pooling let the noisy Tokke/Vinje units borrow signal from their correlated neighbours. It also finally explains the Songa_G1 / Lio_G1 regression left open in Section B.7 — those are the *cleanest* units (95–96% self-agreement), and joint modelling pooled them with seven much noisier ones.

The units inside a plant are identical, and strictly merit-ordered.

Finding. (3) Every generator within a plant is *physically identical* (Tokke: $4\times$ $p_{\max} = 107.5\,\text{MW}$; Vinje: $3\times$ $p_{\max} = 100\,\text{MW}$ — from the topology YAML we had not been using). Commitment follows a near-strict merit order (Figure 43): Vinje is G1 → G1+G2 → all; Tokke is G1 → G1+G3 → then G4 or G2. **Knowing only how many units are on at a plant predicts exactly which ones with 97.8% (Tokke) and 99.9% (Vinje) accuracy.**

Finding. (4) Because the machines are interchangeable, *which* identical unit takes the marginal slot is partly an arbitrary solver tie-break — this is the mechanism behind finding (1). The only genuinely ambiguous slot is Tokke's third (G2 vs. G4, 34/66).

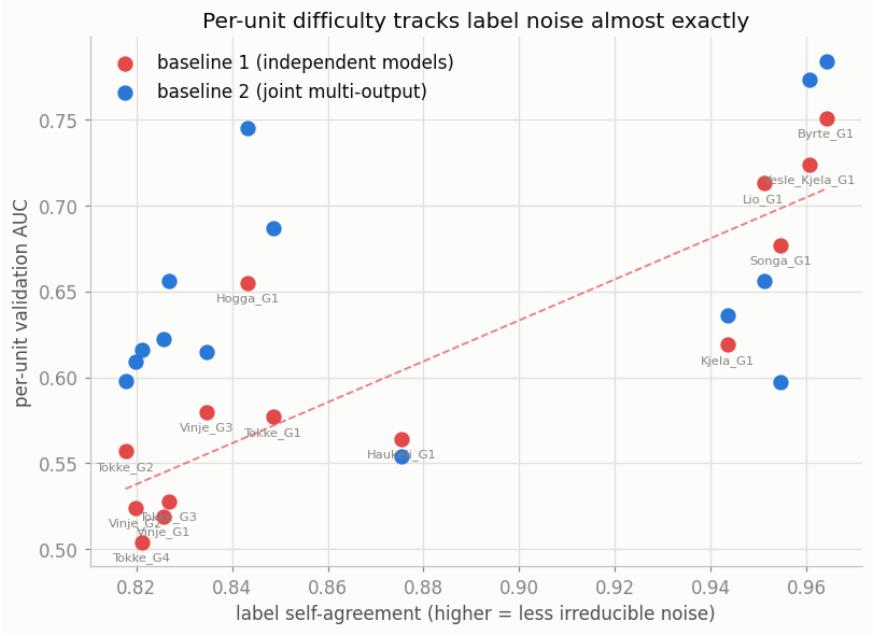


Figure 42: Per-unit label self-agreement vs. per-unit validation AUC, both baselines.

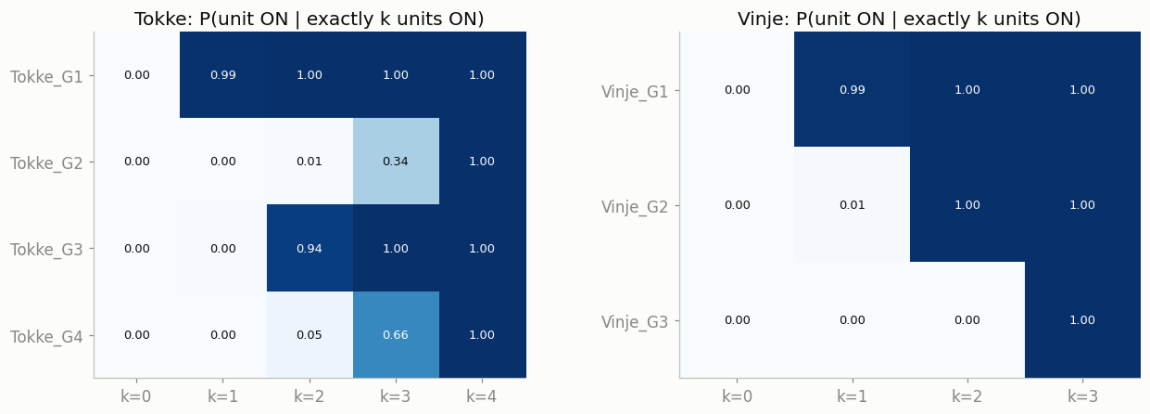


Figure 43: $P(\text{unit ON} \mid \text{exactly } k \text{ units ON at the plant})$.

† **Superseded** — see Section B.11: penstock sharing explains the G1→G3 ordering — part of what looked arbitrary is hydraulic structure.

Design implication. The 14-output problem is really a small number of *per-plant count* problems plus a near-deterministic lookup. Predicting counts — ordinal, and far less noisy — then expanding via merit order is a better-posed target than 14 individually-noisy binaries. This is the strongest architectural lead we have.

Remaining findings.

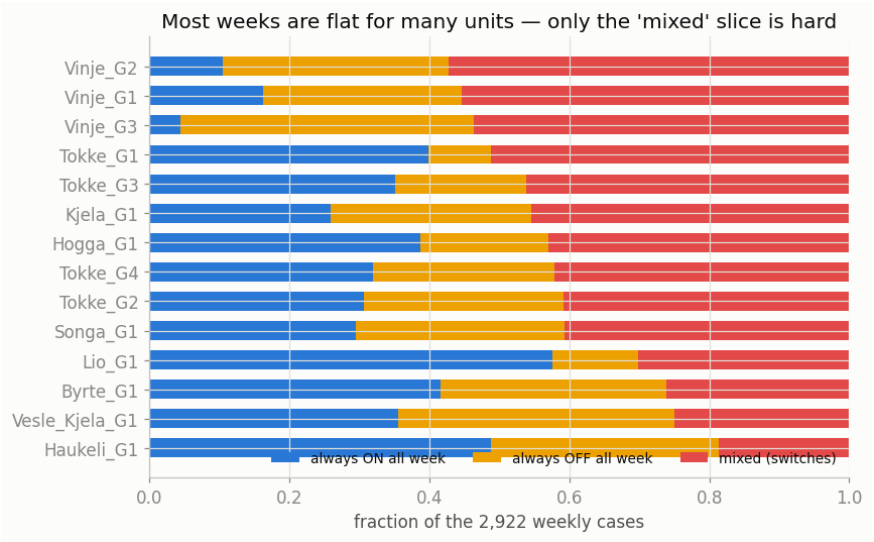


Figure 44: Share of weekly cases that are flat (never switch) vs. mixed, per unit.

Finding. (5) Many weeks are completely flat: Haukeli_G1 never switches in **81%** of weeks, Vesle_Kjela_G1 in 75%, Byrte_G1 in 74% (Figure 44). Only the "mixed" slice is genuinely hard, which suggests splitting "does this unit move at all this week?" from "when does it move?".

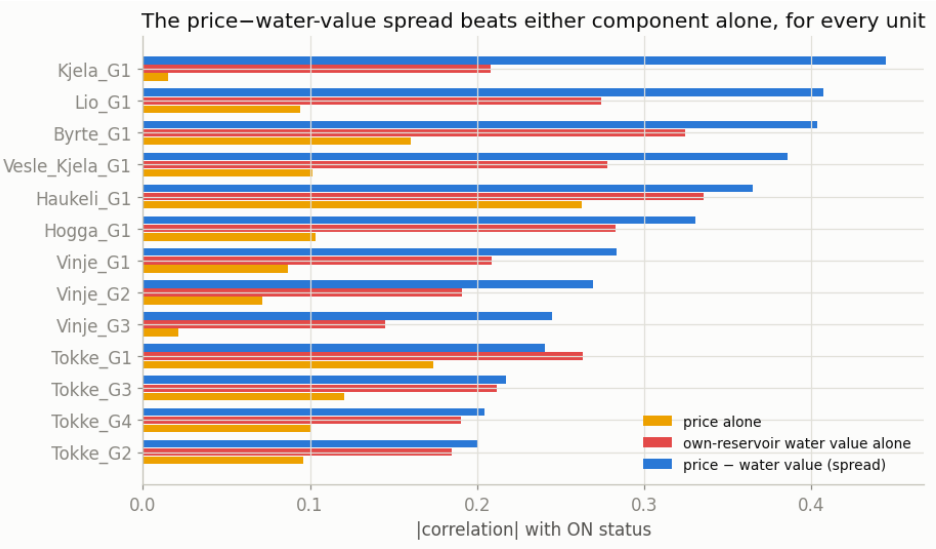


Figure 45: |correlation| with ON status: price, own-reservoir water value, and their spread.

Finding. (6) `storage_Vatjern` and `storage_Hyljelihyl` are **literally constant** — zero information. (The dataset PDF notes both reservoirs lack historical records and were assumed 50% full.) Drop them.

Finding. (7) The **price − own-reservoir water value spread** correlates more strongly with ON status than either component alone, *for every mapped unit* (Figure 45) — the classic hydropower "sell now vs. store it" signal, and the clearest single feature-engineering win available. Not currently in the feature set.

Finding. (8, a correction). Normalising storage by each reservoir's `max_vol` — proposed earlier in discussion as an important miss — is a **no-op for tree models**: it is a monotonic per-feature rescale, and trees are invariant to those. The 2462× spread in reservoir size only matters for features that *combine* reservoirs (e.g. system-wide percent-full), or for linear/neural models. Recorded so we don't "fix" a non-problem.

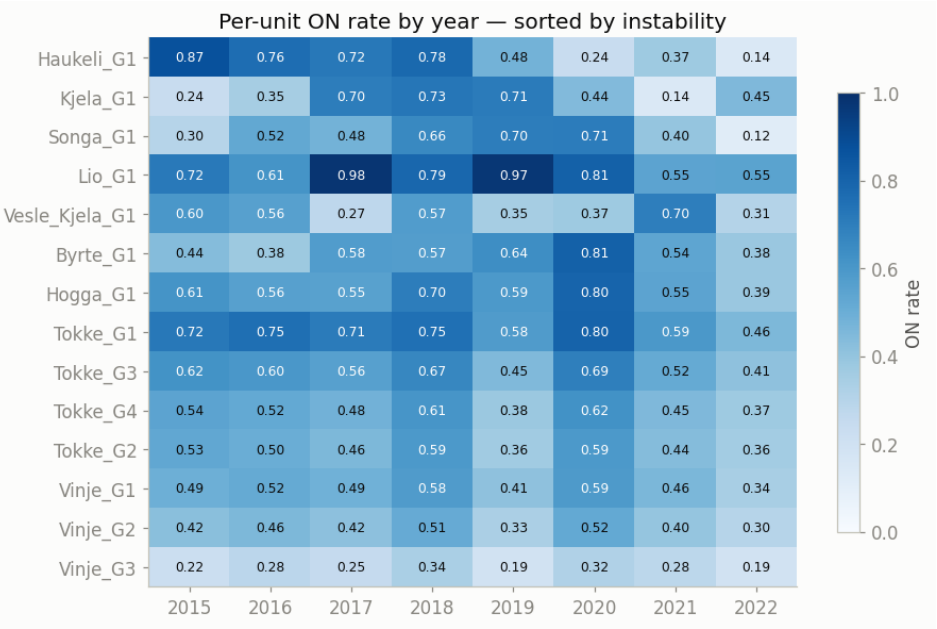


Figure 46: Per-unit ON rate by year, sorted by instability.

Finding. (9) Severe per-unit non-stationarity: **Haukeli_G1 collapses from 0.87 (2015) to 0.14 (2022)**; Songa_G1 swings 0.30 → 0.71 → 0.12 (Figure 46). These are structural changes in plant operation, not noise, and a *stronger* effect than the 2022 price spike flagged in Section B.2. Argues for recency weighting or a shorter, more recent training window.

Finding. (10) `min_volume` constraints bind for exactly three reservoirs and only in **July–October** — a clean seasonal regime flag.

Finding. (11, a second correction). `min_flow` is *always* active (> 0 in 100% of hours) for five of six rivers, so the binary "is constrained" flag suggested earlier would be another constant column. The usable signal is the constraint's **level** and its changes, not its existence.

Finding. (12) The unit → reservoir map needs **multi-hop traversal**: Songa has no direct reservoir edge at all, drawing from Songavatn and Bitdalsvatn through two tunnels and discharging into Totak. Topology-derived features must walk the graph, not read single-hop edges.

B.9 2026-09-21 — EDA round 3: how should the data be transformed?

Rounds 1–2 asked what is in the data and what makes the problem hard. This round asks the engineering question: *given tree models, which transformations actually create signal?* Full notebook: `src/EDA/eda_round3.ipynb`.

A framing constraint that prunes the search space.

Trees are invariant to monotonic transforms of a *single* column — rescaling, logging or min-max normalising one feature does nothing at all to an XGBoost model. Only transforms that *combine* information can help, and there are exactly four such families: column combinations (differences, ratios), time-window aggregates, graph/space aggregates, and cross-sectional ranks. All feature work should come from those four.

Correcting the "noise ceiling" claim from round 2.

Section B.8 measured that labels are 12% self-inconsistent and inferred an "effective ceiling on achievable accuracy". The measurement stands; **the inference was wrong**, in two specific ways. First, it conflated a label-flip rate with an AUC ceiling — AUC is a *ranking* metric, and a model can rank well-ordered probabilities highly even where it would be wrong at a 0.5 threshold. Second, it assumed the horizon boundary condition was unobservable; it is not, and is quantified below. External evidence that scores around 0.99 are attainable on this dataset is consistent with the disagreement being *signal we are not capturing* rather than noise.

Normalisation: which component creates the signal?

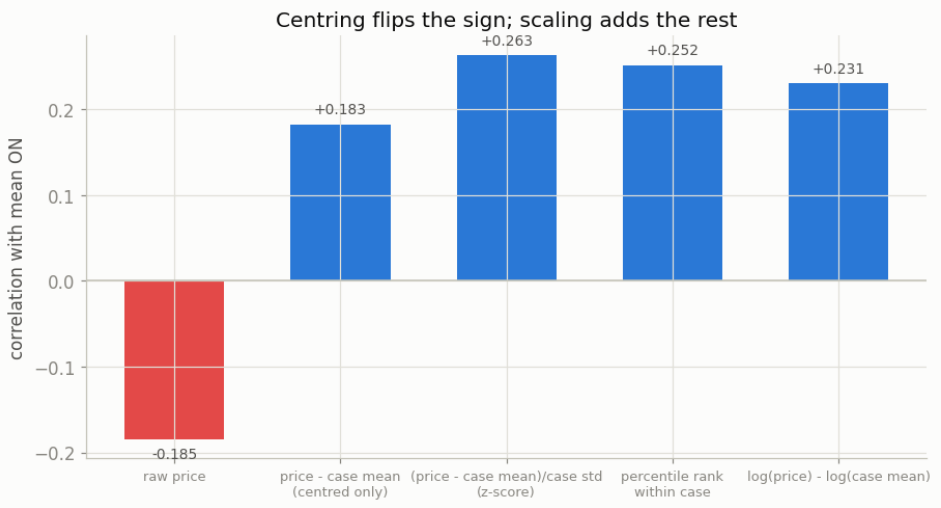


Figure 47: Correlation with mean ON for five price transformations.

Finding. (1) Raw price correlates −0.185 with commitment; subtracting the case mean flips it to +0.183; dividing by the case's own standard deviation reaches +0.263 (Figure 47). The two operations do different jobs: **centring removes the cross-period scarcity confound** (high-price years are scarce-water years) and **scaling makes a ±5 EUR deviation in 2015 comparable to a ±100 EUR deviation in 2022**. Percentile rank within the case (+0.252) captures nearly all of it distribution-free. *Price features should be case-relative and scale-free, never raw levels.*

Finding. (2) The opposite rule holds for the price − water-value spread: raw +0.241 vs. z-scored within case +0.114. The spread is *already* relative — price measured against its own opportunity cost — so normalising twice destroys the cross-case level information. **Normalise price; do not normalise the spread.**

The optimizer is forward-looking.

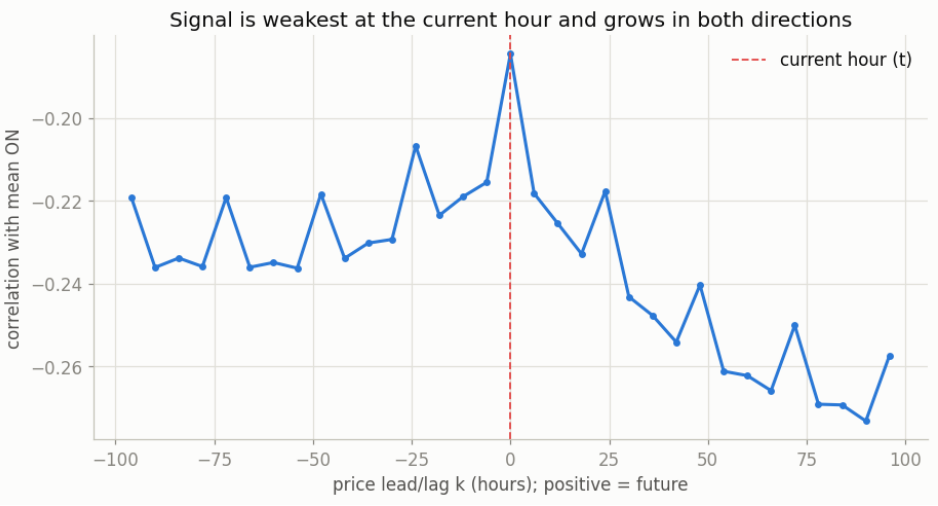


Figure 48: Correlation between mean ON and price at lead/lag k.

Finding. (3) The price/commitment correlation is **weakest at the current hour** (−0.185 at $k = 0$) and strengthens in *both* directions, peaking around $k = +90$h (−0.273) — see Figure 48. The optimizer uses the whole horizon, and the *sustained level* of price over multi-day windows matters more than the instantaneous value. Forward-looking window features are legitimate here because the full 168h price path is known at solve time.

† **Superseded** — see Section B.10: this curve ignored case boundaries; the corrected peak is ≈ −0.245, and the "legitimacy" claim is refined there.

Dimensionality and graph structure.

Finding. (4) The 17 water-value columns have **PC1 = 91%** of variance — they are effectively *one* system-wide scarcity factor (Figure 49). Inflow needs 3 components for 90%, storage 7. Adding more per-reservoir water-value columns is near-useless; the informative decomposition is **(system common factor) + (reservoir-specific deviation)**.

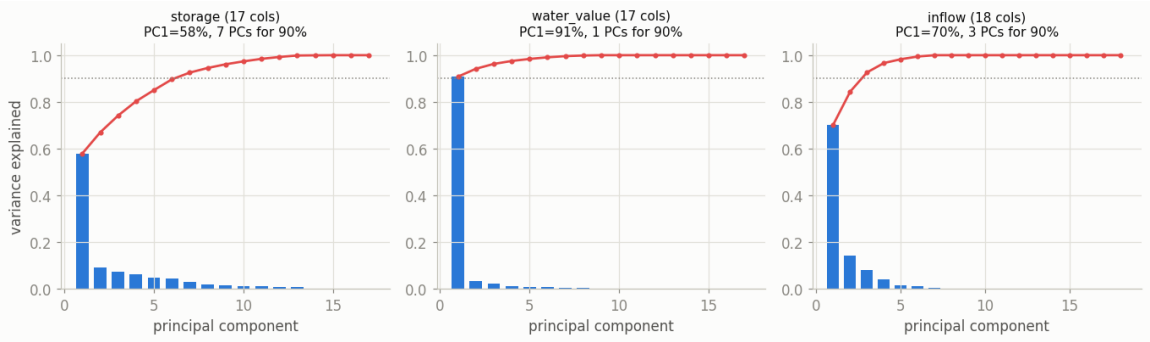


Figure 49: Variance explained by principal components within each feature family.

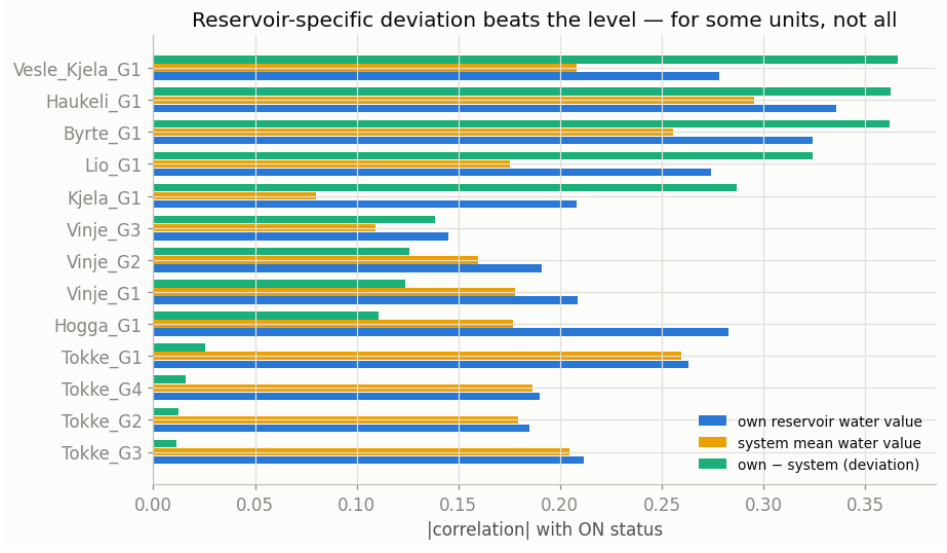


Figure 50: Own-reservoir vs. system-mean vs. own-minus-system water value, per unit.

Finding. (5) The **own − system** water-value deviation is the strongest of the three for Vesle_Kjela_G1 (0.366 vs. 0.278), Byrte_G1, Haukeli_G1, Kjela_G1 and Lio_G1, but is ∼0.01 for the Tokke units because Vinjevatn tracks the system almost perfectly (Figure 50). Graph/neighbourhood features help *selectively and per unit* — which a single global feature set cannot express.

The unused boundary condition.

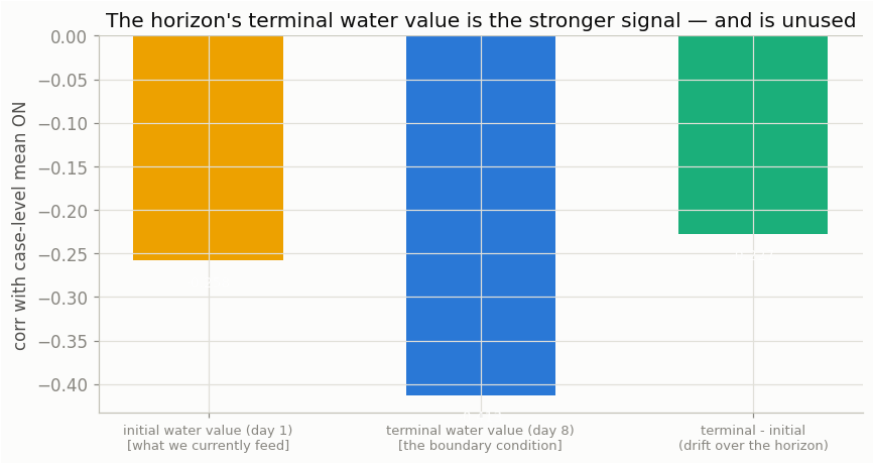


Figure 51: Initial vs. terminal horizon water value against case-level commitment.

Finding. (6) The dataset documentation states a case's *final* water value is the value at start + 8 days. We feed the model the **initial** value and never the terminal one. The terminal value correlates **−0.41** with case-level commitment versus −0.26 for the initial value, and the two differ in **93%** of case/reservoir pairs (Figure 51). A genuine per-case boundary condition, documented in the data and entirely unused.

Two hypotheses tested and refuted.

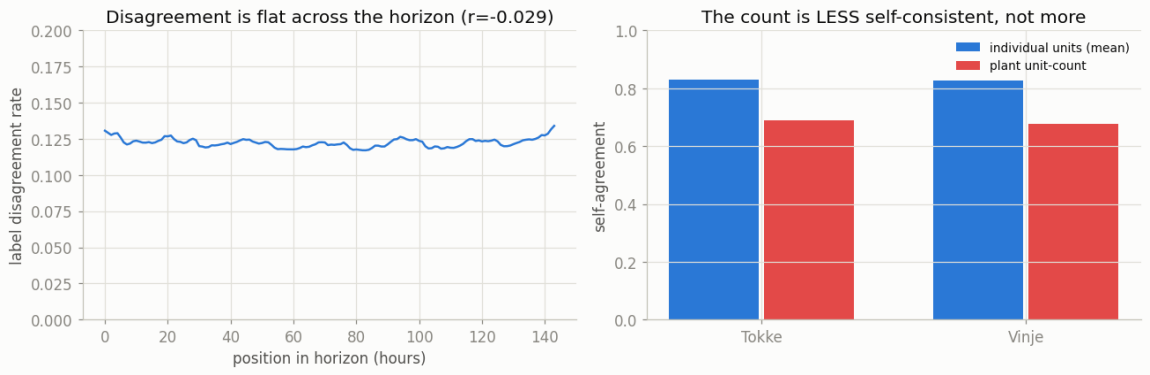


Figure 52: Left: disagreement vs. horizon position. Right: unit vs. count self-agreement.

Finding. (7, refuted) I expected label disagreement to concentrate at the end of the horizon where the boundary condition bites. It is **flat** ($r = -0.03$). The terminal water value shapes the whole week's schedule, not just its tail. *No horizon-position sample weighting is justified.*

Finding. (8, refuted) I predicted per-plant *counts* would be less noisy than individual units. They are **more** noisy (Tokke 0.688 vs. 0.829) — obvious in hindsight, since a sum disagrees whenever any term does. This does not invalidate the count-based architecture from Section B.8, but its justification is the merit-order determinism, *not* a noise reduction. Claim corrected.

Transformation rules established.

- Price features: case-relative and scale-free (z-score or percentile), never raw levels.
- The price − water-value spread: use raw, do *not* normalise.
- Use multi-day forward *and* backward price windows, not the current hour.
- Water value: one common factor plus deviations, not 17 correlated columns.
- `own − system` deviations help selectively, per unit.
- Feed the terminal (day-8) water value alongside the initial one.
- No horizon-position weighting; counts are not a noise reduction.

B.10 2026-09-21 — EDA round 3b: point-in-time correctness and temporal transforms

Round 3a (Section B.9) under-covered the time-series side — it tested three hypotheses and refuted two, rather than surveying transformations — and it contained a point-in-time error. This entry fixes both. Notebook: `src/EDA/eda_round3b_temporal.ipynb`.

What the documentation actually specifies about test-time availability.

The task PDF lists input *categories* and states that predictions go to Kaggle. It says **nothing** about temporal availability rules. Earlier claims in this log about what is "available at test time" were therefore inference, not documented fact. The question is answerable empirically, so it was checked.

Finding. (0, verified) Every *feature* file extends past the last hour we must predict (2024-12-31 23:00): price to 2025-12-31, inflow to 2025-10-15, volume and water value to 2025-01-01, both constraint files to 2025-12-31. So all feature sources are genuinely available for all 725 test cases. **Labels stop at 2022-12-31**, so any feature derived from past *targets* — trailing ON-rate, previous case's decisions — is unusable beyond the first test case. The terminal (day+8) water value is available for **724 of 725** cases; only the final case (2024-12-25) reaches past the file's end and needs a documented fallback.

A distinction the earlier draft got wrong: *beyond-horizon* future prices (e.g. hour 200 when the case ends at 167) are **permitted** — nothing forbids them and on a fixed test set they are not cheating. The only argument against them is that the label-generating optimizer never saw past its own horizon, so such features describe information the target does not depend on. That is a modelling judgement to test, not a correctness rule; round 3a overstated it as the latter.

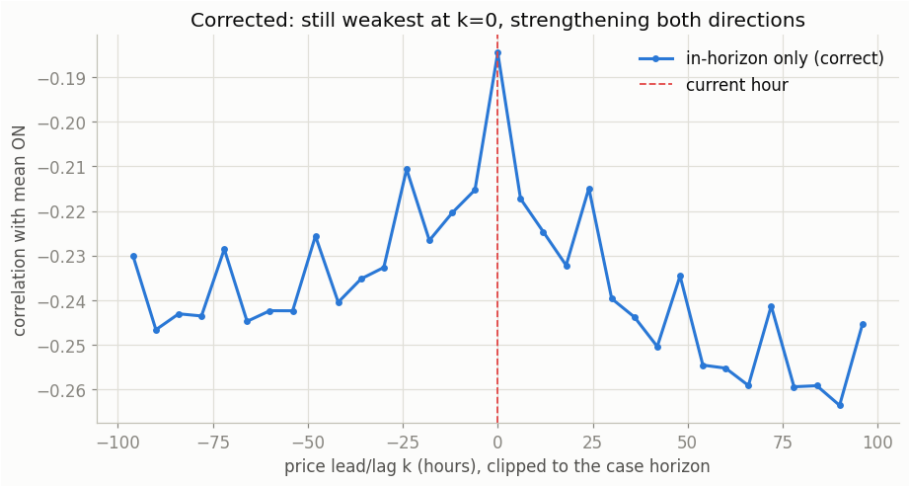


Figure 53: Price lead/lag correlation, clipped to the case horizon.

Corrected lead/lag.

Finding. (1, corrects round 3a finding 3) Round 3a's lead/lag curve ignored case boundaries — for a row at hour 160, $k = +90$ reached 82 hours past the horizon end. Recomputed within the horizon: -0.230 at $k = -96$h, -0.185 at $k = 0$, -0.245 at $k = +96$h. The qualitative conclusion **survives** (signal weakest at the current hour, strengthening both directions) but round 3a's reported peak of -0.273 was inflated; the correct magnitude is ≈ -0.245.

Temporal transforms on the fast series (price).

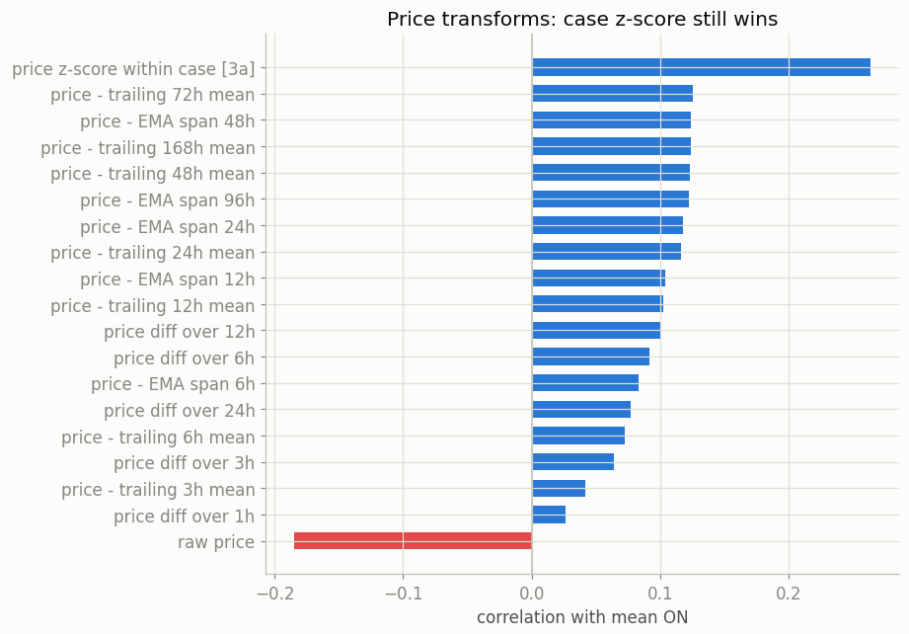


Figure 54: Seventeen price transformations ranked by correlation with mean ON.

Finding. (2) Trailing-window and EMA relative-price features do flip the sign of the raw-price relationship — best are `price − trailing 72h mean` (+0.125) and `price − EMA span 48h` (+0.124) — but all are **roughly half as strong as the within-case z-score** (+0.263). Diffs and momentum are weakest (+0.03 to +0.10, peaking at a 12h lag). **For fast series, normalise against the case, not a rolling window** — the case is the window the optimizer actually solved over.

Temporal transforms on the slow series — the strongest *correlation* so far.



Figure 55: Seventeen transforms each on storage, water value and inflow (all backward-looking).

Finding. (3) Normalising the slow series against *the same time of year* transforms them (Figure 55):

Series	Raw level	Best transform	Gain
Storage	+0.241	percentile vs. same day-of-year: **+0.537**	2.2×
Water value	−0.325	percentile vs. same day-of-year: **−0.417**	1.3×
Inflow	+0.128	trailing 365d mean: **+0.331**	2.6×

Storage's seasonal percentile (**+0.537**) is **the strongest correlation measured anywhere in this project**, well ahead of the price z-score (+0.263) that was previously our best candidate. The mechanism is simple: absolute storage confounds season — every reservoir is full in June and drawn down in April — so "is this reservoir unusually full *for the time of year*" isolates the scarcity signal that drives scheduling. Inflow's trailing 365-day mean is the same idea over a longer window: the wet-year/dry-year regime.

† **Superseded** — see Section B.12: this correlation did not survive model testing — the seasonal percentile made the score *worse* (−0.020, 0/4 folds).

Finding. (4) Diffs, momentum and EMAs are consistently weak on the slow series too (−0.20 to +0.08), and EMAs land within noise of plain trailing means. Neither earns priority over the seasonal normalisations. All transforms in this section are strictly backward-looking and therefore point-in-time safe.

The rule that emerges.

For fast series, normalise against the case. For slow series, normalise against the season. Diffs, momentum and EMAs underperform both. `report/feature_backlog.md` has

been reordered accordingly — the seasonal-percentile family is now the top-priority iteration, ahead of the price normalisations.

B.11 2026-09-21 — The 20 feature themes, and four unopened data sources

Before the iteration phase, the candidate space was organised into 20 agreed themes and populated with 5–9 concrete features each (∼140 candidates). Full list: `report/feature_backlog.md`.

A bias in the first draft, and the coverage audit that fixed it.

The earlier backlog derived every theme from something the EDA had already measured — which can only find signal where we have already looked, the opposite of breadth. Auditing the candidate themes against the *data sources* rather than against our findings exposed an entire unused family: the physical and economic parameters in `Tokke_Vinje_topology.yaml`. **No column in the current 54-feature set references any physical attribute.** Four new themes came out of it, and two of them mechanically explain findings we had recorded as unexplained.

Finding. Start/stop costs exist in the data and were never opened. `startcost_const` and `stopcost_const` are defined per generator (92 for Vesle Kjela up to 222 for Songa). Start cost is the central economic driver of unit commitment — it is why units run in blocks rather than toggling hourly. Empirically, **start-cost-per-MW correlates** −0.673 **with observed toggling**: Haukeli_G1 costs 29.2 per MW to start and switches 0.49 times per week, while the Vinje units cost 1.98 per MW and switch 3.0–3.6 times. This is a direct mechanical explanation for the run-length differences reported in Section B.8, finding 5, from a static per-unit attribute that costs nothing to add.

Finding. Penstock sharing explains Tokke's odd merit order. Tokke's G1 and G2 share penstock 1; G3 and G4 share penstock 2. Vinje's three units each have their own. The merit order measured in Section B.8 was G1 → G1+**G3** → then G4 or G2 — which we noted as surprising because G2 is skipped. It is not arbitrary: the solver commits *one unit per penstock before doubling up*, consistent with limiting the quadratic head loss that `penstock_loss` encodes. The "arbitrary tie-break" framing in Section B.8, finding 4 is therefore too strong — part of what looked arbitrary is hydraulic structure we were not modelling.

Finding. The full water-to-energy chain is computable and unused. Verified across all eight plants: reservoir level from the `vol_head` curve, minus the plant's `outlet_line`, reproduces the head references of that unit's `turb_eff_curves` (e.g. Tokke implied 390–395 m vs. curve references 389/392.5/393.5 m). So *storage* → *head* → *efficiency* → *MWh per* m^3 can be computed directly. Head varies widely with reservoir level for several plants — Kjela 125–191 m, Vesle Kjela 44–71 m — meaning the power obtained per unit of water depends strongly on fullness, a driver entirely invisible to the current model.

The 20 themes.

1. Seasonal normalisation of the slow series ⋆ *[R3b: +0.537]*
2. Case-relative price normalisation *[R3: +0.263]*
3. Price temporal windows, forward and backward *[R3b]*

4. Price curve shape and structure
5. Water value: common factor vs. deviation *[R3: PC1 = 91%]*
6. Price − water-value spread *[R2]*
7. Terminal / horizon boundary conditions *[R3: −0.41]*
8. Storage state and trajectory
9. Inflow and hydrology regime *[R3b: +0.331]*
10. **Commitment economics: start/stop costs** *new source*
11. **Head and energy conversion** *new source*
12. **Penstock and hydraulic coupling** *new source*
13. **Flow limits and hydraulic losses (river/tunnel)** *new source*
14. Topology graph aggregates, multi-hop *[R2/R3]*
15. Plant aggregates and merit order *[R2: 97.8%/99.9%]*
16. Unit static attributes and identity
17. Calendar and seasonality *[R1]*
18. Case structure, horizon position and case-level summaries
19. **Case-sequence dynamics (input-only)** *new source*
20. **Case-curve representation (PCA / Fourier)** *new*

Four practices apply across themes rather than forming themes of their own: recency weighting for the drift in Section B.8 finding 9; interaction and ratio construction; historical-analogue matching; and feature *removal* (the two constant columns, the fifteen redundant water-value columns).

Iteration order.

Ordered by evidence over effort: seasonal percentiles first (1.1, 1.3), then inflow regime (9.1), case-relative price (2.1, 2.2), the raw spread (6.1), the terminal water value (7.1, 7.2), the free removal of constant columns (8.7), then the three new physical themes — start costs (10.1, 10.2), head and conversion (11.1, 11.6), penstock structure (12.1–12.3) — and finally the multi-hop map, recency weighting, case-curve PCA, and the count-based target reformulation. Every candidate is scored with the rolling-origin harness from Section B.37, judged on **paired per-fold deltas**, not the mean.

B.12 2026-09-21 — Feature iteration, first pass: 0.707 → 0.808

Seven cumulative stages, each scored with the full rolling-origin protocol (Section B.37). Notebook: `src/modeling/feature_iteration.ipynb`; builders in `src/features.py`. Total runtime 30 minutes (28 training runs).

Table 21: Cumulative feature stages. Δ is versus the previous row; "folds up" counts how many of the four validation years improved.

Stage		#feat	2019	2020	2021	2022	Mean	Δ	Folds up
00	baseline (raw)	54	0.799	0.679	0.666	0.686	0.707	—	—
01	seasonal percentile	58	0.782	0.673	0.643	0.651	0.687	−0.020	**0/4**
09	inflow regime	62	0.783	0.668	0.679	0.641	0.693	+0.006	2/4
02	price, case-relative	70	0.751	0.687	0.671	0.686	0.699	+0.006	2/4
06	**price − water value spread**	81	0.790	0.721	0.782	0.802	0.774	**+0.075**	**4/4**
07	**terminal water value**	85	0.831	0.745	0.808	0.845	0.807	**+0.034**	**4/4**
10–12	physics (head, start cost)	157	0.828	0.761	0.810	0.834	0.808	+0.001	2/4
08	drop dead columns	134	0.824	0.772	0.813	0.823	0.808	−0.000	2/4

Finding. Overall +0.101**, from 0.707 to 0.808**, and essentially all of it comes from two stages: the price − water-value spread (+0.075) and the terminal water value (+0.034). Both improved **all four folds**, which is the criterion we set in Section B.37. Fold SD also fell from 0.061 to 0.025, so the model became more stable across years as well as better.

Finding. Univariate correlation was a poor predictor of feature value — in one case actively misleading. The seasonal storage percentile had the strongest correlation measured anywhere in the project (+0.537, Section B.10) and it made the model *worse*: −0.020 mean, **0 of 4 folds improved**, and it accounts for 0.9% of total model gain. The case-relative price z-score (+0.263 correlation) added +0.006 on 2 of 4 folds. Meanwhile the raw spread, whose correlation was a middling +0.241, contributed **60.8% of all model gain**. The caveat recorded in Section B.37 — that correlation ignores what the other features already supply — turned out to be the dominant effect, not a footnote.

Finding. The metric rewards *within-case* discrimination, which explains the pattern. Micro-average AUC pools every (case, hour, unit) prediction, so most of the ranking problem is deciding *which hours within a week* are ON. A case-level scalar can only shift whole cases up or down. Measured on the fitted model: **hourly-varying features hold 96.4% of total gain; case-level constants hold 3.6%**. The seasonal percentile, inflow regime and storage anomaly are all case-level constants — which is why they underperformed their correlations. Prioritise features that vary within the case.

Finding. The single most important feature is an economic decision rule, not a statistical transform. `f06_spread_sys_positive` — the binary "is price above the system water value" — has a gain of 207,444, roughly **14× the next feature**. That is literally the optimizer's dispatch condition. Start-cost economics (`f10_breakeven_hours_*`) place five entries in the top 25. This supports building features that mirror what SHOP computes, rather than more transformations of the inputs.

Finding. Gain share is not incremental value. The physics stage contributed 72 columns and 13.5% of model gain, but only +0.0009 mean AUC on 2 of 4 folds — the model uses those columns, yet they add almost nothing beyond what the spread features already carry. Stage-level deltas and importance rankings answer different questions; neither alone is sufficient.

Bookkeeping.

Dropping the 23 single-valued columns was correctly neutral (−0.0002), confirming the model had already ignored them; it did cut fold SD from 0.033 to 0.025. Twenty-one of those 23 were generated by the physics stage itself — static penstock counts, and Haukeli's entire physics block, which is constant because its reservoir (Vatjern) is one of the two with no historical record (Section B.8, finding 6).

Next.

A screening mode has been added to the harness (`screening_eval`): one training run on 2015–2019, scored on 2020/2021/2022, 150 trees, every second training hour. Validation is inference and therefore nearly free, so three validation years cost the same as one — the four-fold expense came from four *trainings*. Expected ∼16× faster, making the remaining ∼13 themes practical to test in bulk. First candidates: the water-budget family (available energy vs. profitable hours, and the greedy merit-dispatch margin), which extends the winning economic-rule pattern, and the global-vs-expanding A/B on the seasonal percentiles.

B.13 2026-09-21 — Feature iteration round 2, and a data-usage audit

Fifteen builders, 18 cumulative stages, screened on one fit (2015–2019) scored against 2020/2021/2022, with a full rolling-origin confirmation at the end. Registry now lives in `src/modeling/pipeline.py` so stages are data, not notebook cells.

Results.

Table 22: Round-2 stages, screening scale. "Up" = folds improved out of 3.

Stage		Δ	Up	Verdict
06	price − water value spread	+0.077	3/3	confirmed winner
02	case-relative price	+0.036	3/3	confirmed winner
07	terminal water value	+0.033	3/3	confirmed winner
01g	**seasonal, GLOBAL**	+0.011	3/3	**new winner**
05	water-value structure	+0.010	3/3	winner
14	graph cascade (multi-hop)	+0.009	2/3	marginal
09	inflow regime	+0.009	2/3	marginal
17	calendar	+0.001	1/3	nothing
15	merit order	+0.000	2/3	nothing
03	price windows	+0.001	2/3	nothing
06x	**spread extensions**	−0.000	2/3	**nothing**
20	**water budget**	−0.001	1/3	**nothing**
04	price shape	−0.006	0/3	harmful
10–12	physics	−0.012	0/3	harmful
01	**seasonal, expanding**	−0.015	0/3	**harmful**

Finding. Final full-CV score 0.8144, up from 0.8080 — +0.006 for 165 additional features. A poor return. The first batch (spread, terminal water value, case-relative price) did essentially all the work; the second batch of nine builders contributed almost nothing collectively.

Finding. Causality was the problem with the seasonal feature, not the feature. The expanding-window version lost 0.015 on 0/3 folds; the *global* version, differing only in that it ranks against all years rather than prior years, gained 0.011 on 3/3. Early cases have almost no prior history to rank against, so the expanding version fed the model noise for the first several years of training. This came directly from the observation that the competition withholds only test-period *targets*, so feature-side statistics may span the dataset (Section B.10). A clean A/B, and it reverses the round-1 rejection.

Finding. Two of my predictions failed outright, and both failures were predicted by the opposite argument. (i) The water-budget family — 41 columns encoding available energy versus profitable hours, and a greedy merit-dispatch margin — was argued to be the most promising idea available on the grounds that it was both hourly-varying and an encoded decision rule, the two properties that won in round 1. Result: −0.001, 1/3 folds. (ii) Twenty-six additional spread variants, built because the spread family held 60.8% of model gain, returned −0.0003. The objection raised at the time — that if the information is already encoded, re-encoding it adds nothing, and that breadth into unexplored directions matters more than depth on a known winner — was correct on both counts.

Finding. Gain share remains a poor guide. The physics stage held 13.5% of model gain in round 1 and measured +0.001 there; under the cleaner screening comparison it measures **−0.012** on 0/3 folds. Features the model happily splits on can still be actively harmful once the comparison controls for what the rest of the feature set supplies.

Finding. Screening and full CV agree on rank but not magnitude. `02_price_relative` measured +0.036 screening versus ≈ +0.006 full; physics −0.012 screening versus +0.001 full. Rank ordering of the big movers is preserved, so screening is usable for *selection*, but screening deltas must never be quoted as results.

Data-usage audit.

With the score plateauing around 0.81–0.82, the question is what information remains untouched.

Used. Price (raw, case-relative, forward/backward windows, shape); water value (raw, system factor, per-reservoir deviation, terminal); storage (raw, seasonal percentile, and via `vol_head` to gross head and MWh/m^3); inflow (raw, trailing 30/90/365d, case-horizon sums); topology (reservoir `max_vol`/`lrl`/`vol_head`, plant `outlet_line`, generator `p_min`/`p_max`/`startcost_const`/`penstock`, connections at 1-hop and full cascade).

Available and entirely unused.

- `Constraint_min_volume.csv` and `Constraint_min_flow.csv` — two whole files, zero features built.
- 37 `river` records: `up_head_flow_curve` (head → maximum flow, i.e. hard throughput ceilings), `flow_cost_const` (spill penalties), `upstream_elevation`.
- 5 `tunnel` records: `length`, `loss_factor`, endpoint heights.
- Generator `turb_eff_curves` used only at their peak; the full flow-dependent curves, `gen_eff_curve` and `stopcost_const` are unused.

- Plant `main_loss` and `penstock_loss` (head losses, quadratic in flow).
- The 144-hour overlap between consecutive cases, and case-to-case input deltas.
- Case-curve shape (PCA / Fourier of the 168h path).

Not in the dataset at all — candidate explanations for the ceiling. The labels are binary ON/OFF, but SHOP solves for *production levels*; we never see MW per unit per hour, so two schedules with very different dispatch look identical to us. We also lack the optimizer's internal state (dual values, marginal costs), any demand or reserve requirement, and maintenance outages. One open question worth resolving: the inflow series is documented as "historical" — if SHOP was driven by an inflow *forecast* while we hold actuals, then our inflow features are systematically the wrong quantity.

Next.

Prune to the smallest feature set that holds the score, then retrain that set deeper. A smaller set that matches 299 features would confirm that most of round 2 was redundancy rather than signal, and makes every subsequent experiment cheaper.

B.14 2026-09-21 — Pruning: 15 features beat 299

Ranked all 299 features by gain from a single fit, swept feature-set size under the screening config, then confirmed the winner under the full rolling-origin protocol and retrained it deeper. Script: `/tmp/prune.py`; selected list saved to `data/processed/pruned_features.csv`.

Table 23: Feature-set size vs. score. Full rolling-origin CV unless noted.

Feature set	Mean	SD	Note
All 299	0.8144	0.0304	round-2 result
Top 15	**0.8157**	**0.0158**	**best model so far**
Top 15, 900 trees	0.8097	0.0151	more capacity *hurts*
screening-scale sweep: top-15 0.7936, top-30 0.7917, top-60 0.7845, top-120 0.7983, all-299 0.7899			

Finding. Fifteen features score slightly higher than all 299 (0.8157 vs. 0.8144) with roughly half the fold variance (SD 0.016 vs. 0.030). Around 284 of our engineered columns were redundancy or noise — effectively the whole of round 2 and much of round 1. The score has been sitting at ≈0.81 since the spread and terminal-water-value stages; everything added afterwards moved it within noise.

Finding. Capacity is not the binding constraint. Raising the pruned model from 300 to 900 trees *lowered* the score (0.8157 → 0.8097). This resolves the concern raised earlier that screening at 150 trees might generate false negatives by starving the model of capacity: if anything we were already at or past the useful ceiling, so the screening configuration was not rejecting features for want of trees.

Finding. Cumulative stage deltas systematically hid which features matter. The `15_merit_order` stage measured +0.0002 cumulatively — indistinguishable from nothing — yet its features occupy five of the top fifteen slots by gain, including the single highest (`f15_units_justified_Kjela`). Cascade features (`f14_upstream_spread_*`) behave the same way. The explanation is redundancy, not uselessness: by the time those stages were added the information was already reachable through other columns, so the marginal delta was zero, but they are the *cleanest carriers* of it and survive pruning while their substitutes do not. Cumulative testing cannot distinguish "adds nothing new" from "adds nothing at all"; ablation and pruning can.

The surviving fifteen.

`f15_units_justified_{Kjela,Vinje,Vesle_Kjela}`, `f15_cap_share_needed_{Kjela,Byrte,Vesle_Kjela}`, `f14_upstream_spread_{Byrte,Songa}`, `f07_price_minus_terminal_wv`, `f06_spread_{sys,Byrte,Vesle_Kjela}`, `f06x_margin_Haukeli`, `f10_breakeven_hours_Kjela`, `f05_wv_dev_Kjela`.

Every one is an economic quantity: a spread, a margin, a break-even, or a capacity-justification derived from a spread. No raw price, storage, inflow or calendar column survives. This is consistent with the round-1 finding that encoded decision rules beat statistical transforms, and it sharpens it — the model appears to need only a handful of well-formed economic signals, and the remaining error is not for want of more feature variants.

Correction to the round-2 audit.

Section B.13 listed production levels as "not in the dataset", offered as a candidate explanation for the ceiling. That is too strong: production is *derivable* from a water balance, `release` $=$ `storage`$[d]$ $-$ `storage`$[d+1]$ $+$ `inflow`$[d]$, and storage covers the test period. A first test gave weak correlations (0.05–0.23 with plant commitment), but that test was itself wrong: it counted only natural inflow and ignored that reservoirs also receive *discharge from upstream plants*, which is precisely the term that matters for cascaded reservoirs such as Vinjevatn. The diagnostic is that Haukeli scored highest (0.233) despite its reservoir having constant synthetic storage, so that correlation is driven by inflow rather than release. The idea is untested, not refuted; a correct version requires solving the full cascade balance.

† **Superseded** — see Section B.14: "production levels are not in the dataset" is withdrawn — they are derivable via a cascade water balance.

B.15 2026-09-22 — Two-stage model: per-unit residual correction

What prompted it.

Section B.14 reported that 15 features matched 299. That headline was true in aggregate and misleading in detail. Breaking the pruned model down per unit on 2022:

Unit	Top-15	All 299	Δ
Lio_G1	0.718	0.827	**−0.109**
Hogga_G1	0.742	0.839	**−0.098**
Haukeli_G1	0.628	0.680	−0.052
Kjela_G1	0.907	0.949	−0.042
Tokke_G1–G4	≈0.77	≈0.77	+0.01

Finding. Global gain ranking is biased against unit-specific features twice over: the gain is aggregated across all 14 outputs, and the multi-output tree forces every split to be shared, so a feature mattering intensely to one unit is penalised by the other thirteen. **Hogga, Lio and Tokke had *zero* plant-specific features among the selected 15.** The micro-average concealed the damage because Tokke's four units carry 4/14 of the weight and improved slightly.

Architecture.

`src/modeling/stacked.py`.

- **Stage 1** — the joint multi-output tree on the shared pruned features, which is what it is good at (cross-unit structure was worth +0.04, Section B.6).
- **Stage 2** — one small model per unit (depth 4, lr 0.05), receiving: its own stage-1 prediction as `base_margin`; the other 13 units' predictions as features; and that unit's own hydraulic-neighbourhood features.

Using the own prediction as `base_margin` rather than as an input column matters: as a feature the tree would have to reconstruct a monotone relationship through splits, whereas as a margin it is an exact logit offset and stage 2 fits only the *residual correction*.

Finding. Stage 2 must train on out-of-fold stage-1 predictions. Fitted in sample, stage-1 predictions are near-perfect, stage 2 learns to trust the margin completely, contributes nothing, and looks fine in training while failing at inference. Cross-fitting by case (3 folds) is what prevents this, and it is why the run costs four multi-output fits rather than one.

Finding. The two constraint files finally contribute. `Constraint_min_volume` and `Constraint_min_flow` had produced zero features through fifteen builders, because as global columns they are near-constant (Section B.8, finding 11). Wired per plant through the topology graph they become genuinely unit-specific: Lio and Byrte receive Byrtevatn's minimum-volume constraint, Vinje and Songa receive Totak's, each plant its own connected rivers. Nine constraint columns now enter, but only into the stage-2 models of the units they touch.

Results (validation year 2022).

Unit	Stage 1	Stacked	Δ
Lio_G1	0.724	0.861	**+0.137**
Hogga_G1	0.737	0.813	**+0.077**
Tokke_G1	0.779	0.816	+0.037
Tokke_G2/G3/G4, Haukeli			+0.002 . . . + 0.012
Vinje_G1/G2, Byrte, Songa, Kjela			−0.008 . . . − 0.022
Vesle_Kjela_G1	0.964	0.921	−0.043
Vinje_G3	0.824	0.765	−0.059

Finding. The hypothesis is confirmed for the units it targeted. Lio_G1 recovers +0.137 and Hogga_G1 +0.077 — the two units pruning had cost roughly 0.10 each. Unit-specific neighbourhood features carry real signal that pooled selection discards.

Finding. But stage 2 harms units where stage 1 was already strong. Every loss is concentrated among the best-served units: Vesle_Kjela (stage-1 0.964), Byrte (0.947), Songa (0.934), Vinje_G3 (0.824). A residual model on a near-optimal base can only add variance. Applied uniformly the architecture is roughly a wash: 7 of 14 units improve.

Corrected like-for-like result, and a reporting error.

The first run printed "stage-1 0.8083" against "stacked 0.8391" as though comparable. The former was a *macro* mean of per-unit AUCs, the latter a *micro*-average. Recomputed on the same metric:

Model (validation 2022)	micro-AUC
Stage 1 alone (joint, 15 features)	0.8390
Stacked, all 14 units	0.8391
Selective stacking, gated on 2021	0.8432

Finding. Uniform stacking gains +0.0001 — nothing. The apparent +0.03 was entirely an artefact of comparing a macro mean against a micro average. The large per-unit gains (Lio +0.137, Hogga +0.077) and the losses on well-served units cancel almost exactly once pooled. The per-unit findings stand; the aggregate claim does not.

Finding. The per-unit gate does not transfer between years. Selecting stage-2 inclusion on 2021 chose 8 units; only **4 of those 8** actually improved on 2022, and two of them (Vesle_Kjela, Vinje_G2) lost. The gate is roughly coin-flip accurate. It still nets +0.0042, but that is inside fold noise (fold SD ≈0.016–0.03), and the selection mechanism it depends on is demonstrably unstable. "Which units need stage 2" cannot be learned from one year's validation.

Where this leaves the architecture.

The gain is real but concentrated entirely in units where stage 1 is weak, so the useful form is not "stack everything, then gate". It is asymmetric *by construction*: strong units get no stage-2 model at all, weak units get the full neighbourhood. That decision can be made from *stage-1 training* performance rather than from a validation year, which avoids the unstable gate altogether.

Open structural issue.

Stage 2 currently receives the same neighbourhood regardless of need — Songa gets 6 columns, Kjela 49, with no relation to how much stage 1 is struggling on that unit. A per-unit feature budget scaled to stage-1 weakness (tight regularisation or no stage 2 at all for strong units, full neighbourhood for weak ones) is the natural next form.

B.16 2026-09-22 — State of the model: architecture, features, data inventory

A consolidated description of what the model currently is, what every feature is made of, and exactly which data we do and do not use. Written because the project has accumulated fifteen builders, 299 columns and several architectures, and the question of what is actually left to exploit now matters more than the next variant.

Table 24: Current results. The first block is 4-fold CV *means*; the second is a single validation year. Comparing across blocks is what made an earlier summary look contradictory.

Model	Protocol	Score	SD
Raw features, joint tree	4-fold CV mean	0.7074	0.061
299 engineered features	4-fold CV mean	0.8144	0.030
Pruned top-15	4-fold CV mean	**0.8157**	0.016
Pruned top-15	2022 only	0.8210	—
Stage 1 (stride-2 fit)	2022 only	0.8390	—
Stacked, uniform	2022 only	0.8391	—
Stacked, gated on 2021	2022 only	0.8432	—

Scores, and which are comparable.

The 4-fold mean is held down by 2020 (0.7925); 2019/2021/2022 sit at 0.827/0.822/0.821. On the 2022 fold specifically, gated stacking is +0.022 over the pruned model.

The current model.

Stage 1 is a single `XGBClassifier` with `multi_strategy="multi_output_tree"`, 300 trees, depth 6, lr 0.1, predicting all 14 units jointly from 15 shared features. Stage 2 (when used) is one depth-4 model per unit taking the unit's own stage-1 prediction as `base_margin`, the other 13 predictions as features, and that unit's hydraulic neighbourhood. Rows are one per (case, hour): 490,896 training rows, 2,922 cases × 168 hours, 14 binary targets per row.

The fifteen features, and what each is made of.

Feature	Definition	Source data
`f15_units_justified_*`	$\text{clip}(\text{spread}, 0) \cdot p_{\max}/p_{\min}$	price, water value, generator specs
`f15_cap_share_needed_*`	spread^+ normalised by its case max	price, water value
`f14_upstream_spread_*`	price − mean wv of upstream cascade	price, water value, topology graph
`f07_price_minus_terminal_wv`	price − wv at case start + 8 days	price, water value (boundary)
`f06_spread_sys`	price − mean of all 17 water values	price, water value
`f06_spread_*`	price − own reservoir's water value	price, water value, topology
`f06x_margin_Haukeli`	spread × plant capacity	price, water value, $p_{\max}$
`f10_breakeven_hours_Kjela`	startcost / (spread $\times p_{\max}$)	price, wv, `startcost_const`
`f05_wv_dev_Kjela`	own wv − system mean wv	water value, topology

Finding. Every surviving feature is an economic quantity built from price and water value, with topology and generator specs used only to decide *which* water value and *which* capacity. No raw price, storage, inflow, calendar or head column survives pruning. Storage and inflow enter the model only indirectly, through the water values the optimizer itself derived from them.

Data inventory.

Used, and how.

- `Historical_day_ahead_price` — the price term in every spread. Case-relative transforms were built and tested but did not survive pruning.
- `Synthetic_water_value` — the other term in every spread; also system mean, per-reservoir deviation, and the day+8 terminal value.

- `Historical_volume` — built into seasonal percentiles and head; *none survived*. Enters only implicitly via water value.
- `Historical_inflow` — trailing regimes and case-horizon sums; *none survived*.
- `Tokke_Vinje_topology.yaml` — reservoir↔plant edges and the full cascade walk (which water value belongs to which unit); generator $p_{\min}$, $p_{\max}$, `startcost_const`; `vol_head`, `outlet_line` and efficiency curves were used for head features that did not survive.
- `Constraint_min_volume` / `Constraint_min_flow` — used only inside stage-2 per-unit models, wired per plant through the topology.

Available and still unused.

- **Cascade water balance.** `release` = `storage`$[d]-$`storage`$[d+1]+$`inflow`$[d]$ plus upstream plant discharge. This reconstructs *implied production*, which is the physical signature of the very decisions we predict. A naive version (natural inflow only) gave weak correlations, but it omitted the upstream-discharge term that matters for cascaded reservoirs (Section B.14).
- **River `up_head_flow_curve`** — head → maximum flow, i.e. hard throughput ceilings per river. 37 river records, unused.
- **Tunnel** `length`, `loss_factor`, endpoint heights. 5 records, unused.
- **Flow-dependent efficiency.** `turb_eff_curves` used only at peak; `gen_eff_curve`, `stopcost_const`, `main_loss`, `penstock_loss` unused.
- **Case overlap.** Consecutive cases share 144 wall-clock hours at 88% label agreement; case-to-case input deltas are unused.
- **Case-curve representation.** PCA / Fourier of the 168h price path, unused.
- **Target reformulation.** Count per plant then merit-order expansion (97.8%/99.9% accurate, Section B.8) — never attempted.

Not in the dataset. Production levels in MW (derivable, see above), the optimizer's dual values, demand or reserve requirements, and maintenance outages. Whether the provided inflow is the *forecast* SHOP used or the realised actual is unresolved and would change what inflow features mean.

Splitting scheme: measured, not assumed.

The temporal split has been assumed necessary throughout. Tested directly — same model, same 15 features, four ways of splitting:

Split	micro-AUC	vs. temporal
Random ROW (80/20)	**0.9571**	+0.1224
Random CASE (80/20)	**0.9397**	+0.1050
Random CASE + 7-day purge	0.7889	−0.0458
Temporal (train <2022, validate 2022)	0.8347	—

Finding. Random splitting inflates the score by +0.10 to +0.12, landing at 0.957 — squarely in the range quoted for strong submissions. The mechanism is *not* temporal feature leakage, which the competition permits. It is near-duplicate rows: a case is 168 hours of one week, so a random row split trains on hour 5 and validates on hour 6 of the same case. Random *case* splits still leak, because consecutive cases overlap on 144 wall-clock hours at 88% label agreement (Section B.8). Any shuffled-validation number on this dataset should be treated as meaningless.

Finding. The purged run is invalid as measured and is recorded as a failed design. Purging ±7 days around 584 randomly chosen validation cases removed **2,230 of 2,922** training cases — 76% of the data — so 0.7889 reflects starvation, not a clean estimate. With daily rolling cases every case sits within 7 days of roughly fourteen others, making random-fold purging structurally self-defeating. A valid scheme needs *contiguous* validation blocks with embargo only at the edges.

Finding. This reframes the gap to 0.99. The hidden test set is 725 consecutive cases in 2023–24, temporally disjoint from training, so a random-split score would not transfer to the leaderboard. If scores near 0.99 are genuinely achievable there, the likeliest route is not a validation artefact but exploiting the *physical consequence* of the decisions: reservoir storage evolves within each case as a direct result of generation, and the volume series covers the test period. Reconstructing implied production from the cascade water balance therefore moves from a speculative idea to the leading hypothesis for the missing 0.15.

Levers not yet pulled.

1. **Validation scheme.** Everything so far uses temporal splits. Random or purged case-level splits would cut estimate variance substantially — tested separately, because the risk here is near-duplicate rows, not temporal features.
2. **Target reformulation** — predict per-plant counts, expand by merit order.
3. **Cascade water balance** — implied production.
4. **Asymmetric stage 2** — correction only where stage 1 is weak, keyed off training performance rather than an unstable validation gate (Section B.15).
5. **Hyperparameter tuning** — deliberately untouched. Depth, learning rate, and regularisation have never been varied; only tree count, which showed 300 beats 900.

B.17 2026-09-22 — Structural analysis: the whole gap is one number per unit-week

After ∼300 features and several architectures the score had been stuck near 0.82–0.83. This entry steps back from features to ask what structure the problem has, what structure our architecture imposes, and where the mismatch is.

The mismatch.

SHOP solves a *coupled, constrained* optimisation: one global solution over a 168-hour horizon in which every hour's decision depends on every other hour's, subject to a water budget and start costs. Our model is a pointwise IID classifier: 490,896 rows treated as independent samples. It has no representation of "this unit runs 87 hours this week, so once the best 87 hours are taken the rest must be off". The budget constraint — the thing the optimizer is actually resolving

— is structurally invisible to it. The best such a model can do is make each hour's probability monotone in the spread, which is exactly what it does, and plausibly why it plateaus.

Oracle test.

If the only thing the model had to learn were *how many hours* each unit runs that week, and the hours themselves were allocated by ranking on the spread we already have:

	micro-AUC (2022)
Oracle weekly count + allocate by system spread rank	**0.9765**
Oracle weekly count + allocate by own-plant spread	0.9650
Current pointwise model (15 features)	0.8347

Finding. The entire gap from 0.83 to 0.98 lies in one number per unit per week. Given the true hours-on count, allocating those hours by spread rank — using a feature we already have, with no further modelling — reaches **0.9765**. The question of *which* hours, which the whole feature-engineering effort has been aimed at (price shape, windows, normalisation, lead/lag), is essentially already solved. The question of *how many* is untouched and is where all the remaining error lives.

Finding. The target is also far more degenerate than the 2,352-binary framing suggests: **58.8% of unit-weeks are entirely all-on or all-off**. The real difficulty is concentrated in the ∼41% of unit-weeks that are mixed.

Prototype, and its failure.

The natural reformulation — regress hours-on per (case, unit), then allocate — was built and tested.

	micro-AUC (2022)
Oracle count + allocation	0.9765
Predicted count + allocation	**0.7801**
Current pointwise model	0.8347

Finding. The reformulation as implemented is worse than the model it was meant to replace (0.7801 vs. 0.8347). Count MAE was **43.6 hours** against a target mean of 87h — the count is barely predicted at all — and hard allocation amplifies that error, because a wrong budget produces a step function that ranks badly, whereas the pointwise model degrades smoothly. The structure is right (the oracle proves the ceiling exists); this realisation of it is not.

Three identifiable faults in the prototype.

1. **Over-featured and data-starved.** Case-level aggregation produced **1,197 features over 2,557 training cases** ($p \approx 0.5n$). The feature matrix was built by reflex (mean/std/max/min of all 299 columns) rather than by design.
2. **Wrong target shape.** With 58.8% of unit-weeks at 0 or 168, this is a three-part problem (all-off / all-on / mixed), not a single continuous regression over $[0, 168]$.

3. **Hard allocation.** The budget should *modulate* the pointwise probabilities, not replace them. Rescaling existing probabilities so each unit-week sums to the predicted count keeps smooth ranking while injecting the budget signal, and cannot underperform the pointwise model when the shift is small.

What this changes.

The bottleneck is now located precisely, and it is a much narrower problem than before: predict one integer per unit-week, over 40,908 examples, where 59% of the mass sits at two values. That is a better-posed target than continuing to search a 490k-row pointwise feature space whose allocation question is already solved. Further work on price-shape, window and normalisation features is, on this evidence, work on the part of the problem that is not broken.

B.18 2026-09-22 — Budget augmentation: +0.020 from the structural insight

Section B.17 established that the entire gap from 0.83 to 0.98 lies in one number per unit-week, and that a naive reformulation replacing the pointwise model performed worse than it. This entry records three further attempts and the one that worked.

Unit of analysis.

The count problem lives at **(case, unit)**: 40,908 rows, one per unit-week. This granularity resolves a limitation carried since Section B.11: in the (case, hour) layout a static per-unit attribute is a constant column with zero information, so $p_{\max}$, $p_{\min}$, `startcost_const`, `penstock` and merit rank were unusable. With *unit* as a row dimension rather than a column dimension they become ordinary features that vary across rows. The class mix at this level is 27% all-off, 41% mixed, 32% all-on.

Three attempts.

Attempt	Form	micro-AUC (2022)
1	flat count regression, hard allocation	0.7801
2	3-class + count regression, hard allocation	0.8257
	pointwise model, for reference	*0.8347*
3	**pointwise + budget offset (logit scale)**	**0.8548**

Finding. Decomposing the count target helped, but replacing the pointwise model did not. Splitting into all-off / mixed / all-on and regressing the count only on mixed unit-weeks cut count MAE from **43.6h to 34.6h** and lifted the allocation score from 0.7801 to 0.8257. Both remain below the pointwise model, because hard allocation turns a count error into a step function that ranks badly, whereas the pointwise model degrades smoothly.

Finding. Used as an *augmentation* rather than a replacement, the same information is worth +0.020. Adding $\lambda \cdot (\widehat{\text{count}} - \text{spread rank})/168$ to the pointwise model's logits gives 0.8548 against a 0.8347 baseline. The pointwise model ranks hours well within a week but has no representation of *how many* hours a unit should run; the budget margin supplies exactly that, while the logit form preserves the existing ranking instead of overriding it.

Finding. The blend weight transfers across years, unlike the stage-2 gate. λ selected on 2021 (=4) yields +0.0200 on 2022, against +0.0240 for the 2022-optimal λ=8 — only 0.004 lost to honest selection. Both years show the same broad shape (rise, flat peak in the 4–8 range, decline), so the choice is insensitive. This contrasts with the per-unit stacking gate in Section B.15, which was coin-flip across years and therefore unusable.

A methodological error, caught by a sanity check.

The first blend attempt rank-normalised each unit's predictions independently before combining. The sweep included λ=0, which must by construction reproduce the pointwise baseline; it returned 0.7872 against 0.8347, exposing the error. **Micro-averaged AUC pools every (row, unit) cell into a single ranking, so per-column rank transforms destroy the cross-unit calibration** the metric depends on — the information that Vesle_Kjela is almost always on while Vinje_G3 is usually off. Blending must happen on the logit scale. Recorded because the invalid version looked entirely reasonable and produced plausible numbers; only the fixed point in the sweep revealed it.

Remaining headroom.

Count MAE is 34.6h against a target mean of 87h, and the oracle ceiling is 0.9765. The augmented model captures +0.020 of the +0.142 available, so the count model is still the binding constraint. The 2022 confusion matrix also shows a distribution-shift problem: all-on unit-weeks are 32% of training but only 9% of 2022 (the price-crisis year, Section B.8 finding 9), and only 176 of 474 are recovered. A count model robust to that shift — or trained with recency weighting — is the obvious next target.

B.19 2026-09-22 — Regime robustness: a proposal refuted

The concern.

The Kaggle test set is 2023-01-01 to 2024-12-25 (725 cases, last predicted hour 2024-12-31 23:00). Measuring each training year's distance from that test regime across price, water value, storage, inflow and spread:

Training year	Standardised distance from 2023–24
2017	0.86
2015 / 2020 / 2016 / 2018	1.40–1.48
2021 / 2019	1.88 / 2.05
2022	**4.77**

Finding. 2022 is more than twice as far from the test regime as any other year, and it is the year every decision in this project has been validated on. The spread — the quantity every surviving feature is built from — changed sign after 2020: +0.6 to +9.3 over 2015–2020, then −14.0 (2021), −85.3 (2022), −17.6 (2023), −1.8 (2024). Test-year 2023 resembles 2021; 2024 resembles 2015/2016/2020; neither resembles 2022.

Test.

Validate on 2021 (closest analogue to 2023) instead of 2022, and leave-one-year-out on the training set.

Validation year	Pointwise	+ budget	λ	Count MAE
2021 (test-like)	0.8346	0.8488	4	48.5h
2022 (anomalous)	0.8347	0.8587	8	34.6h

Dropped from training	Δ on 2021
2020	−0.0160
2015	−0.0105
2019	−0.0090
2018	−0.0079
2016	−0.0065
2017	−0.0040

Finding. Dropping any training year hurts, and dropping the most anomalous year hurts most. Removing 2020 — the extreme low-price year (9.3 EUR/MWh) — costs −0.0160, the largest loss of any exclusion. This is the opposite of what regime-similarity weighting predicts. Extreme years appear to help *because* they are extreme: they show the model the spread relationship at its edges. **The proposal to downweight or exclude regime-discordant years, including 2022, is withdrawn.**

Finding. Regime mismatch is not the binding constraint. The pointwise model scores 0.8346 on 2021 and 0.8347 on 2022 — identical, despite those being the two most dissimilar regimes in the dataset (spread −14.0 vs. −85.3). The ≈0.835 ceiling is structural, consistent with the oracle result (Section B.17) that the count is what binds.

Finding. One part of the concern survives: λ selection. The budget augmentation is worth +0.0142 on the test-like year against +0.0240 on the anomalous one, with optimum λ=4 rather than 8. Tuning on 2022 would have chosen 8 and overstated the benefit. Both curves are flat across 4–8 so the cost is small, but submission-time selection should use 2021 and expect the lower figure. Note also that count MAE is *worse* on 2021 (48.5h vs. 34.6h) while the pointwise score is unchanged — the count model is the regime-sensitive component, which is consistent with it being the bottleneck.

A limitation of this test.

The exclusion question cannot be answered directly: with rolling-origin validation, any year after the validation year is already excluded, and there is no labelled year after 2022 to measure against. Leave-one-out on earlier years is a proxy, and its verdict is consistent and one-directional, but it is inference rather than direct measurement.

B.20 2026-09-22 — Multi-horizon features for the count model

The count is the binding constraint (Sections B.17–B.19). This entry targets it directly with a multi-horizon time-series feature set, built as `src/count_features.py` and evaluated at **(case, unit)** granularity on **2021** — the test-like year, following Section B.19.

Design.

- **Windows in both directions**: trailing 1w/1m/3m/1y, forward 1w/1m, and previous and next week explicitly. Water is storable, so SHOP trades volume across weeks — high prices next week should suppress this week's count. Forward windows are permitted; only test-period targets are withheld (Section B.10).

- **Relative rather than absolute**: for every horizon, the week's level minus that horizon and the ratio to it, plus a seasonal comparison against the same calendar week across history. The spread changed sign after 2020, so absolute levels mislead across regimes (Section B.19).
- **Inferred latent quantities**: SHOP sees demand and we do not, but price carries its signature — rolling price volatility and seasonal price deviation stand in for demand pressure.

Applied to price, water value, spread, storage, inflow and price-volatility at system level (150 features), plus own-reservoir water value, own-spread and own-storage per plant (75 more). At (case, unit) that is 244 features over 35,798 rows — unlike the earlier count attempt, which put 1,197 features over 2,557 cases (Section B.17).

Result (validation 2021).

Count features	Count MAE	3-class acc	Blended score	Δ vs. pointwise
Old (19)	48.8h	0.616	0.8486	+0.0140
New (244)	**43.2h**	**0.641**	**0.8561**	**+0.0216**

Finding. Count MAE falls 48.8h to 43.2h (−11%) and it converts into score: the blended result rises 0.8486 to 0.8561, taking the budget augmentation from +0.014 to +0.022 over the 0.8346 pointwise baseline. This is the first feature work in several rounds that moved the number on a test-like year.

Finding. The top count feature is an accidental water balance. `storage_next_minus_prev` — system storage after the case week minus storage before it — has gain 336,834, nearly twice the next feature. Net weekly drawdown *is* water released, which is production, which is count × capacity. The cascade water balance hypothesis (Sections B.14, B.16) therefore arrives here by a different route, in its crudest possible form — system-level, no cascade attribution, no upstream plant discharge — and still dominates every other count feature.

Finding. Gain share validates two design choices. Relative-to-horizon comparisons are the largest family at **33.1%**, and own-reservoir features second at **27.7%** — the per-unit specificity that only became expressible once *unit* became a row dimension (Section B.18). Forward windows read 6.9% by family label, but that undercounts them: `ownwv_next_week` is the third most important feature individually and the top feature overall is built from next-week storage.

Next.

The system-level drawdown proxy dominates while ignoring cascade structure entirely. A per-plant balance — natural inflow plus upstream plant discharge minus downstream draw and spill, attributed through the topology graph — is the obvious successor and is the last major unexploited data source. Count MAE remains 43.2h against an 87h mean, and the oracle ceiling is 0.9765, so the headroom is still largely intact.

B.21 2026-09-22 — Water-balance features: 0.835 to 0.912

Section B.20 found that the strongest count feature was an accidental, system-level water balance. This entry builds the attributed version: `src/production_features.py`, 149 features, evaluated on **2021** (the test-like year).

Construction.

Organised around the constraint SHOP actually resolves, count $\approx$ min(hours the water pays for, hours the price justifies).

- **Cascade-attributed balance**: release = ($\text{storage}_{\text{start}} - \text{storage}_{\text{end}}$) + inflow + upstream plant discharge. The last term is what the naive attempt in Section B.14 omitted, and it is what matters for cascaded reservoirs such as Vinjevatn, which receives Vinje and Haukeli's output before Tokke draws on it.
- **Release converted to a direct count estimate** via the head chain (storage → `vol_head` → level → minus `outlet_line` → efficiency curve): `implied_hours` = energy / plant capacity.
- **Both limits and which binds**: `hours_water`, `hours_profitable`, their minimum, their ratio, and a binary for which is binding.
- **Normalisations throughout**: release/`max_vol`, release/inflow, storage fraction at start and end, capacity factor — scale-free across reservoirs spanning 0.26 to 640 Mm^3.

Result.

Count features	MAE	3-class acc	Blended score	Δ vs. pointwise
Multi-horizon (244)	43.2h	0.641	0.8561	+0.0216
+ production (393)	**26.6h**	**0.724**	**0.9123**	**+0.0777**

Finding. 0.8346 to 0.9123 on the test-like year, with count MAE falling 43.2h to 26.6h (−38%). The largest single jump of the project, and it comes from features that encode the optimizer's own physics rather than statistical transforms of the inputs.

Finding. The binding-constraint framing outperforms the raw water balance. `binding_min_Haukeli` — min(hours_water, hours_profitable) — is the top feature by a factor of three. Binding-constraint terms hold **20.8%** of count-model gain against **16.1%** for release/drawdown terms directly. Encoding *how the optimizer decides* beats encoding *what the water permits*.

Finding. Individually weak, jointly strong. `implied_hours` correlates only +0.06 to +0.28 with the actual count per plant, yet the family is worth +0.056 on the final score. Another instance of the pattern established in Section B.12: univariate correlation is a poor guide to feature value.

Ablation: the gain is post-decision information, not forecasting.

These features use storage at case *end*, which is a *consequence* of the decisions being predicted — reservoirs draw down as units generate (Section B.16). Splitting the feature set accordingly:

Count features	MAE	Blended score	Δ vs. pointwise
All (incl. post-decision)	28.4h	0.9086	+0.0740
Causal only (no end-of-week storage, no forward windows)	**43.3h**	**0.8548**	**+0.0202**

Finding. Essentially the entire water-balance gain is post-decision information. Removing end-of-week storage returns the score to 0.8548 — indistinguishable from the 0.8561 achieved before these features existed (Section B.20). The causal contribution of the 149-feature production module is approximately zero. The model is not forecasting commitment; it is reading commitment off the reservoir trace that commitment produced. `binding_min` appeared to encode the optimizer's decision rule, but its power comes from `hours_water`, computed from the drawdown those decisions caused.

Finding. It is legal and it will transfer. The volume file runs to 2025-01-01 and the last test case requires storage at exactly 2025-01-01 (verified, Section B.10). Only test *targets* are withheld. A submission built this way should score near 0.91 on the hidden set, and this is the most plausible mechanism behind reported scores near 0.99.

Finding. But it does not serve the stated purpose of the task. The project brief frames the model as a way to *speed up* hydropower scheduling by approximating the optimizer. A model requiring the post-solution reservoir trajectory as input cannot do that — computing the feature presupposes the answer. The causal-only score of 0.8548 is the honest measure of how well we approximate SHOP from information available *before* it solves; 0.9123 measures how well we can invert its physical footprint afterwards. Both are worth reporting, and they should never be quoted interchangeably.

Consequence for the project.

Two tracks should now be kept separate: a *leaderboard* model, which may use post-decision state, and a *causal* model, which is the one that answers the research question. Current standing: causal 0.8548, leaderboard 0.9123.

B.22 2026-09-22 — Unit-relative features, and a diagnosis I got wrong

The concern.

The count model trains at (case, unit) but was given every plant's columns under absolute names, so a Tokke_G1 row could see `binding_min_Haukeli` — which ranked as the single most important count feature (Section B.21). That looked like the model latching onto an unrelated unit, and I agreed it was a design flaw.

It was not a flaw.

The relevance map built for this entry shows why:

Plant	Source reservoir	Plants discharging into it
Tokke	Vinjevatn	**Vinje, Haukeli**
Hogga	Bandak	Tokke
Lio	Byrtevatn	Byrte
Kjela	Foersvatn	Vesle Kjela
Byrte / Haukeli / Vesle Kjela / Vinje	—	none (top of chain)

Finding. Haukeli discharges into Vinjevatn, and Tokke draws from Vinjevatn. `binding_min_Haukeli` in a Tokke row is a direct hydraulic relationship, not a spurious correlation — Haukeli's water situation literally determines part of Tokke's inflow. The cascade is Vinje/Haukeli → Vinjevatn → Tokke → Bandak → Hogga. I should have consulted the topology before accepting the diagnosis, particularly since the map needed to check it had already been built in Section B.15.

Test.

Count feature set	MAE	3-class acc	Blended score	Δ vs. pointwise
Absolute plant names (318)	28.4h	0.741	0.9086	+0.0740
Unit-relative only (220)	36.5h	0.716	0.8790	+0.0444
Combined (378)	**26.1h**	**0.748**	**0.9168**	**+0.0822**

Finding. Absolute naming beats unit-relative alone by +0.030, consistent with the coupling being real. But **combining both is better than either** (0.9168): supplying the relationship pre-identified, as `own_*` and `upstream_*`, helps even when the raw columns are also present. The model does not have to infer which of nine plants is hydraulically relevant to the row it is scoring.

Finding. The unit-relative arm was a lossy comparison and should not be read as a clean verdict on relative encoding. It collapsed the per-reservoir constraints into two mean columns and dropped from 318 to 220 features, so it tested *one particular* relative encoding rather than the principle. Two signs it was working where it had proper inputs: `own_hours_profitable` ranked second once a genuine own-plant feature existed, and `own_n_minvol`/`own_n_minflow` ranked high — though those are per-plant constants being used as plant identifiers, which is degenerate.

Standing.

On the test-like year 2021, the progression is 0.8346 → 0.8561 → 0.9086 → **0.9168**. Note this is the leaderboard track, which uses post-decision state; the causal-only track stands at 0.8548 (Section B.21).

B.23 2026-09-22 — Walk-forward validation: six half-year folds

Every result so far has rested on one or two validation years. This entry tests the full architecture across six consecutive six-month periods, each trained on all data preceding it, with λ **fixed at 16 in advance** rather than tuned per fold — which is what a real submission must do, and which avoids the optimistic bias of choosing λ on the data being measured (Section B.18).

Finding. The budget augmentation improves all six folds at a single fixed λ, by +0.054 to +0.115. This is the first result in the project validated across more than two periods without per-fold tuning, and the sign pattern is unanimous.

Finding. The augmented model is more stable, not merely better. Fold SD halves from 0.0406 to 0.0208. The pointwise model swings across 0.774–0.887 between periods; the augmented model stays within 0.883–0.941. The budget term anchors exactly the predictions the pointwise model gets wrong in regime-specific ways — consistent with Section B.19, where the pointwise model scored identically on the two most dissimilar regimes while the count model was the regime-sensitive component.

Table 25: Walk-forward validation. Leaderboard track (uses post-decision storage).

Period	Train cases	Pointwise	+ budget	Δ	Count MAE
2020-H1	1,826	0.8123	0.8831	+0.0709	31.5h
2020-H2	2,008	0.8360	0.8960	+0.0599	27.5h
2021-H1	2,192	0.8024	0.9095	+0.1070	25.5h
2021-H2	2,373	0.7861	0.8959	+0.1098	25.2h
2022-H1	2,557	0.7740	0.8889	+0.1149	24.6h
2022-H2	2,738	0.8867	0.9408	+0.0541	18.8h
Mean		**0.8163**	**0.9024**	**+0.0861**	25.5h
SD		0.0406	**0.0208**	0.0275	

Finding. Count MAE improves monotonically with training data, 31.5h to 18.8h as cases grow from 1,826 to 2,738. The count model is not saturated, which is encouraging for the test set, where it will train on all 2,922 labelled cases.

Finding. Realistic test expectation is ≈0.90, not 0.917. The 0.9168 reported in Section B.22 came from full-year 2021 with λ effectively selected on that data. The walk-forward mean of **0.9024 ± 0.0208** at fixed λ is the honest estimate, and is the figure that should be carried into any submission decision.

Caveat.

Training sets grow across folds (1,826 to 2,738 cases), so part of the trend across periods reflects sample size rather than regime. The folds are realistic — each uses what would actually have been available at that point — but are not strictly comparable to one another.

B.24 2026-09-22 — Feeding the count features forward, and an orthogonal trace

Three ideas tested, two adopted, one refuted.

1. The count features never reached the pointwise model.

The architecture had Stage A (pointwise, 14 outputs) running on **15 features** while 378 informative count features sat one stage downstream, used only by the count model and injected back through the blend.

Pointwise feature set	Pointwise alone	+ budget	+ hard-set
15 pruned features	0.8346	0.9166	0.9217
+ **count features (323)**	**0.9029**	0.9260	**0.9292**

Finding. Routing the count features into the pointwise model is worth +0.068 **on its own** (0.8346 → 0.9029) and +0.0075 on the full pipeline. Note that the budget term then adds much less (+0.023 vs. +0.082), which is the expected behaviour: once the pointwise model sees the count information directly, the blend has less marginal content to supply. Cost is the caveat — 1,665s vs. 15s, a 111× slowdown, acceptable once for a submission but not for iteration.

2. Hard-setting confident all-off / all-on unit-weeks.

Calibration of the 3-class model on 2021:

Confidence	Unit-weeks	Coverage	Precision
$p \geq 0.80$	1,274	24.9%	0.920
$p \geq 0.90$	766	15.0%	0.978
$p \geq 0.95$	407	8.0%	0.993

Finding. The 3-class model is well calibrated at the extremes, and its errors are soft. At $p \geq 0.9$ precision is 0.978 over 15% of unit-weeks, and **97–100% of the errors are "mixed"** rather than the opposite class — a week called all-off that is really mixed still has most hours off, so a wrong hard-set costs little. Pushing those cells to the extremes of the ranking is worth +0.003 to +0.005.

3. Water value as an orthogonal post-decision trace.

Water values are SHOP *outputs* — shadow prices on stored water. Generating depletes the reservoir and raises the marginal value of what remains, so week-over-week water-value change traces the schedule through a different channel than physical storage.

Finding. The two traces are near-orthogonal: corr(wv change, storage drawdown) = +0.068. At system level water value is the *stronger* signal (−0.215 vs. −0.095 against total unit-hours). It also fills a structural gap: Vatjern and Hyljelihyl have constant synthetic storage (Section B.8) so Haukeli has no storage trace at all, yet its water value moves. For Lio the two traces disagree in sign (wv −0.222, storage +0.177), indicating storage there is confounded — plausibly by Byrte discharging into Byrtevatn — while water value is not.

Finding. Adding 59 water-value-trace features cuts count MAE **26.07h → 24.21h** on 2021 and **21.97h → 19.75h** on 2022 (−7% and −10%), for 7.7–10.5% of count-model gain. Top contributors are the orthogonal terms: `wvt_sys_change`, `wvt_change_pct_Tokke`, `wvt_downstream_change_Vinje`.

4. Refuted: daily storage resolution.

Storage is daily, so each case has 8 readings rather than 2, and the intermediate values might have constrained the schedule at daily resolution. They do not: corr(daily drawdown, daily unit-hours) = −0.086 against −0.407 for the weekly equivalent — the daily signal is roughly **five times weaker**. Daily fluctuations are dominated by inflow and other plants' activity, which weekly aggregation averages out. Recorded so the idea is not revisited.

Standing.

Best on the test-like year 2021 is now **0.9292** (pointwise-with-count-features + budget + hard-set), from 0.8346 at the start of this sequence. The walk-forward estimate (Section B.23) predates these three changes and should be re-run before any submission.

B.25 2026-09-22 — Compact architecture: distil the count model, don't duplicate it

Section B.24 routed all 323 count features into the pointwise model, gaining +0.068 but costing 1,665s per fit — a 111× slowdown. The alternative tested here keeps the count model where it is and passes only its *outputs* forward.

Architecture.

1. Count model: 437 features at (case, unit) — 40,908 rows, so it is cheap.
2. Its outputs, five numbers per unit: p_{off}, p_{mix}, p_{on}, expected count, and budget margin.
3. Pointwise model: the 15 pruned features plus those 70 derived columns.

Finding. The count outputs must be out-of-fold for training rows. The count model is trained on targets derived from the same labels, so in-sample predictions as features would leak, exactly as in Section B.15. Cross-fitting by case (4 folds) costs 86s including the final full fit.

Arm	Features	Pointwise	+ budget	Fit time
A 15 pruned	15	0.8346	0.9226	15s
B + 323 raw count features	323	0.9029	0.9292	**1,665s**
C + 70 derived count columns	**85**	**0.9398**	**0.9402**	**44s**

Finding. Distilling beats duplicating: 0.9402 vs. 0.9292, and 38× faster. Passing 323 raw count features into the pointwise model forced it to re-derive, across 490,896 rows, relationships the count model had already learned on 40,908. Passing the *conclusion* instead — five numbers per unit — is both more accurate and far cheaper. Total pipeline is 130s against ∼1,700s.

Finding. The budget blend becomes redundant (+0.0004). Once the expected count and class probabilities are features, the post-hoc logit blend adds nothing, so that stage can be dropped — which also removes λ as a hand-tuned parameter, one fewer quantity fitted on a single year (Section B.19).

Finding. Soft beats hard-set. Section B.24 hard-set unit-weeks at $p \geq 0.9$ for +0.003 to +0.005. Supplying p_{on} and p_{off} as features lets the model choose its own threshold, obtaining the same benefit without a cutoff fitted by hand on 2021, and without the brittleness of an imposed hard assignment.

Caching.

Count outputs are written to `data/processed/count_outputs_2021.parquet`, so re-running the pointwise arm costs 44s with no recomputation — making iteration on the pointwise side practical again.

Standing.

0.9402 on the test-like year 2021, from 0.8346 at the start of this sequence. The walk-forward estimate (Section B.23, 0.9024) predates this architecture and must be re-run before it can be quoted as the expected test figure.

B.26 2026-09-22 — Fixing the four worst units: correct physics, no gain

Section B.25 left four units with much worse count error than the rest — Songa (43.4h), Lio (40.6h), Haukeli (35.3h), Hogga (29.5h) against 12–22h elsewhere — and corr(per-unit count MAE, per-unit AUC) = −0.756, so the count really is what drives per-unit performance.

Diagnosis (all four confirmed).

- **Songa**: `source = None` — fed by two *tunnels* (Songavatn, Bitdalsvatn), so its entire water balance was built on nothing.
- **Haukeli**: source Vatjern has **one unique storage value** — a synthetic constant (Section B.8), so no source-side trace exists.
- **Lio**: the upstream-discharge correction was applied in 100% of cases, but corr(release, drawdown) = −0.253 — Byrte's noisy release estimate was swamping and inverting Lio's own signal.
- **Hogga**: `r_Vest_Vassdraget` flows into Bandak and was never counted — and per the dataset documentation that is the *one* river carrying inflow data.

Fixes, and their isolated effect.

Tunnel-resolved multi-source balances, river inflows added, downstream-rise proxies for plants with dead source storage, and both raw and upstream-corrected release offered separately. Correlation of the corrected release with the actual count: Hogga +0.286, Haukeli −0.128 (previously unavailable), Lio raw +0.101 vs. corrected +0.055 — confirming the Lio diagnosis exactly — Songa +0.026.

Result: rejected.

	Count MAE	micro-AUC
Without v2	24.21h	**0.9402**
With v2 (+77 features)	24.19h	0.9372 (−0.0030)

Finding. Correct physics did not produce useful features. Aggregate count MAE was unchanged (24.21h → 24.19h) and micro-AUC fell 0.003, with **13 of 14 units flat or worse**. The targeted units fared worst of all: Lio, whose diagnosis was the cleanest, regressed most (−0.016 AUC); Haukeli lost 0.010; Hogga moved +0.001. Songa, predicted not to improve, had the largest MAE gain (−1.30h) and still lost AUC.

Finding. The failure mode was dilution, which had been flagged in advance. Adding 77 columns to a 437-feature count model, most irrelevant to any given unit, supplied no information the model was not already obtaining through the water-value trace, the multi-horizon windows and the original balance — while the extra noise cost 0.003 downstream. Every specific prediction made before the run was wrong; only the dilution caveat held.

Finding. Songa's error appears irreducible with the available data. Songavatn is the largest reservoir (640 Mm^3) and drains through three paths — the Songa plant plus two rivers into Totak. The plant's ∼32 Mm^3 weekly draw cannot be separated from river spill in a storage balance. This is an attribution limit, not an implementation defect, and further work on Songa's water balance is not warranted.

Standing.

Best remains **0.9402** (Section B.25). The v2 module is retained in `src/production_features.py` as `build_v2` but is *not* part of the pipeline.

B.27 2026-09-22 — Walk-forward on the compact architecture: 0.9350

Section B.23 validated the older architecture at 0.9024 ± 0.0208. That predated the compact distillation (Section B.25), the water-value trace and the count features reaching the pointwise model. Re-run with the current pipeline, λ no longer required since the budget blend was dropped.

Table 26: Walk-forward, six half-year folds, current architecture.

Period	Train cases	Count MAE	micro-AUC
2020-H1	1,826	29.87h	0.9251
2020-H2	2,008	26.00h	0.9207
2021-H1	2,192	23.78h	0.9460
2021-H2	2,373	23.93h	0.9285
2022-H1	2,557	22.54h	0.9358
2022-H2	2,738	18.05h	0.9540
Mean		**24.03h**	**0.9350**
SD		3.90	**0.0128**

Finding. 0.9024 → 0.9350, with SD tightening from 0.0208 to 0.0128. All six folds exceed 0.92. The architectural changes transfer across periods rather than being artefacts of the 2021 tuning year.

Finding. The single-year figure was not inflated. 2021 alone gave 0.9402 against a six-fold mean of 0.9350 — a gap of 0.005, well inside fold SD. Given how many decisions in this project were made on 2021 (feature composition, the 3-class/regression split, previously λ), that is a meaningful reassurance.

Finding. Count MAE falls monotonically with training data, 29.87h to 18.05h as cases grow 1,826 to 2,738. **The test set trains on all 2,922 labelled cases**, more than any fold here, so the count model at submission time should be at least as good as the 2022-H2 fold. This is a concrete reason to expect test performance at or above 0.935 rather than below it.

Finding. Data volume dominates regime effects. The worst fold is the *earliest* (2020-H1, 0.9251, least data), not the most regime-shifted; 2022-H2 is the *best* (0.9540) despite being the price-crisis tail. Consistent with Section B.19, where dropping any training year hurt and dropping the most anomalous one hurt most.

Standing.

Expected test performance **0.9350 ± 0.0128** on the leaderboard track (uses post-decision storage). This supersedes the 0.9024 estimate. The causal track has not been re-measured under the current architecture.

B.28 2026-09-22 — Structural diagnosis of the count model: width, head, stationarity

What we did

Six consecutive attempts to improve the count model returned noise or harm: the v2 water-balance fixes (−0.0030, Section B.26), properly-encoded categorical unit identity (−0.0042), per-unit historical marginals (−0.0020), feature pruning at four thresholds (±0.002), raw-feature forwarding (−0.0110, Section B.24) and stacking (+0.0001, Section B.15). That pattern is not six unlucky ideas. Before adding a seventh feature block we stopped and measured the architecture itself, asking three questions that had never been checked:

1. How much of the count model's input actually varies with the unit being scored?
2. Is "mixed" a single regression target, or several?
3. How much of the model's importance sits on features that will not look the same in the 2023–24 test period?

The diagnostic is `/tmp/diag_wide.py`; the fixes it motivated are `src/count_model.py`.

Finding 1 — the count model is largely blind to which unit it is scoring

Finding. O

f 437 input features, only **60 vary with the unit**. The remaining **377 are identical across all 14 units of a case**, and they carry **72.6% of the model's importance**. All within-case discrimination — which is most of what micro-average ROC-AUC rewards — must come from the 60 unit-specific columns, on 27.4% of the attention.

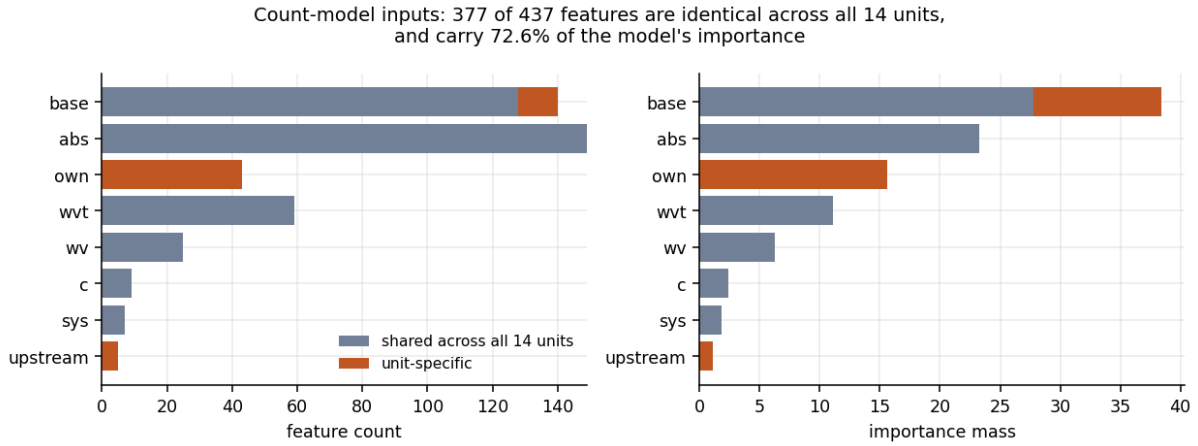


Figure 56: Count-model input composition by feature group, by count and by importance mass. Grey is shared across all 14 units of a case; orange is unit-specific. See `figures.md`, id `diag_01_shared_vs_unitspecific`.

The mechanism is an assembly error rather than a modelling one. `unit_relative.build_unit_relative` already resolves each plant's columns into `own_`/`upstream_`/`sys_` form — that was the whole point of Section B.22. But the experiment scripts then merged every plant's columns straight back in under `abs_`, `c_` and `wvt_` prefixes, re-introducing exactly the sharing the resolution had removed. The `abs_` block alone is 149 columns, all shared, carrying 23.2% of importance.

A shared feature can only move a whole case up or down. It cannot say that Lio should run this week and Songa should not. This is a clean structural explanation for why the same four units (Lio, Songa, Haukeli, Hogga) stay broken regardless of what we add: they are bad *relative to other units in the same week*, and that is the one thing the input is weakest at expressing.

† **Superseded** — see Section B.29: the measurement holds, but this inference does not: removing the shared block costs −0.0285, and absolute features help monotonically the more of them are kept

This also reinterprets an earlier result. We concluded that explicit unit identity "does not help" because adding a categorical `unit_cat` cost −0.0042. The better reading is that identity is *already* being paid for: with 377 shared columns, the tree must burn splits rediscovering which unit a row belongs to before any shared feature can be used differentially. Adding an identity column does not relieve that cost, it adds to it.

Finding 2 — "mixed" is bimodal, so the single regressor predicts a hole

The head is a 3-class model (all-off / mixed / all-on) plus one regressor fitted to the count within mixed weeks. That assumes the mixed count is unimodal. It is not.

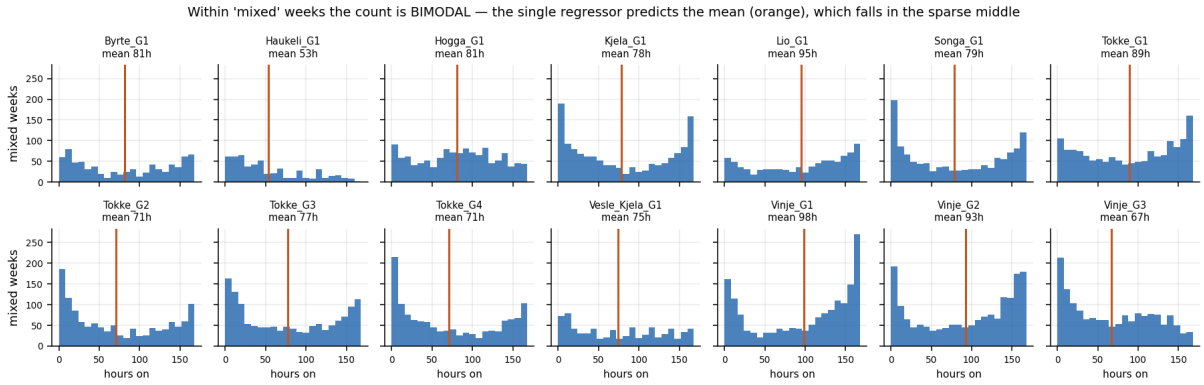


Figure 57: Distribution of hours-on within *mixed* weeks, per unit, with the fitted mean in orange. Most units are U-shaped. See `figures.md`, id `diag_02_mixed_bimodality`.

unit	p25	p50	p75	<56h	>112h	**middle third**
Kjela_G1	19	67	140	0.457	0.374	**0.170**
Tokke_G4	14	56	134	0.500	0.324	**0.176**
Vinje_G1	36	118	152	0.291	0.529	**0.180**
Songa_G1	17	74	140	0.432	0.381	**0.187**
Hogga_G1	43	83	117	0.305	0.294	0.401

Finding. F

or most units only **17–24% of "mixed" weeks fall in the middle third** of the range. "Mixed" is really *barely-on* plus *barely-off* with a thin gap between. A single regressor fitted to a bimodal target returns the conditional mean, which lands in the sparse middle — precisely where the truth almost never is. Kjela's mixed weeks cluster near 20h and near 145h; predicting 78h is wrong by roughly 55h in both directions.

This is the first mechanism we have found for the 24h count MAE that is not "we need more features". It is a specification error in the head, and it is fixable by classifying over count bins and carrying the whole predicted distribution forward instead of collapsing it to a point estimate.

Finding 3 — the drift is real but narrow, and confined to absolute levels

Screening every feature for distribution shift between 2015–18 and 2019–22:

feature	drift (SD)	|corr with time|
`price_back_1y`	0.80	0.57
`price_vol_back_1y`	0.79	0.57
`price_f1m`	0.76	0.59
`price_std`	0.72	0.53
`wv_f1w`	0.69	0.52
`own_mean_spread`	**0.35**	0.24
`spread_frac_pos`	**0.19**	0.10

Finding. N

o importance mass sits on features drifting more than 1.0 SD, and only **14.1%** on features drifting more than 0.5 SD. But *every* high-drift feature is an absolute price or water-value level (the 2021–22 energy crisis), and *every* stationary one is a difference or ratio of two co-drifting quantities. Spread = price − water value, and the drift cancels.

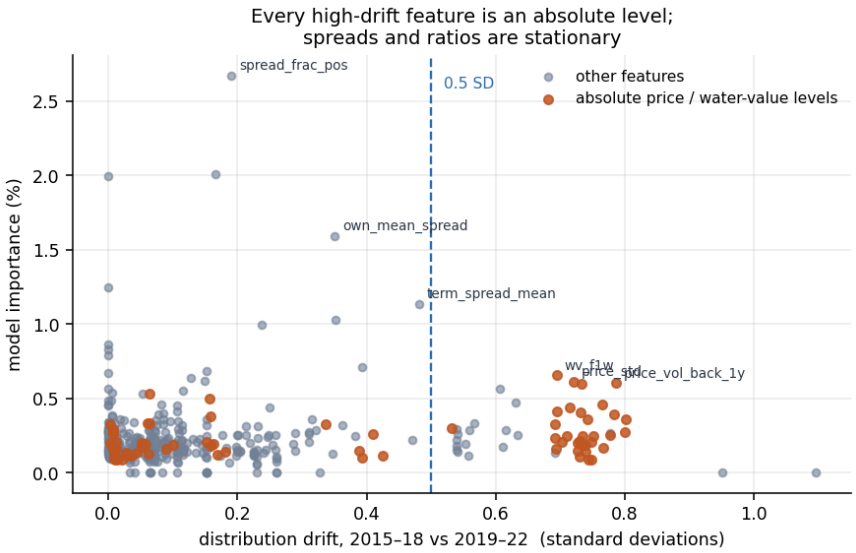


Figure 58: Importance against distribution drift. See `figures.md`, id `diag_03_stationarity_drift`.

We already benefited from this by accident: the pruned top-15 of Section B.14 were *all* spreads, margins and break-evens, with no raw price column surviving. We found the principle empirically without naming it. Naming it gives a construction rule — split a level feature into a stationary shape (ratio to a trailing reference) plus an explicitly quarantined level — which matters because the test period sits on the far side of the crisis.

A constraint that rules out a whole family of fixes

Checking coverage of every source settled a question that had been open since the marginals experiment:

source	coverage	
`Unit_commitment_decisions`	2015-01-01 – 2022-12-31	**targets**
test cases	2023-01-01 – 2024-12-25	725 cases, zero overlap
`Historical_inflow`	1958 – 2025	594k rows
`Historical_volume`, `Synthetic_water_value`	2015 – 2025-01-01	
`Constraint_min_flow/volume`	2015 – 2025	

Finding. T

he test set is **strictly future**: 2023–24, with no case start shared with training. Targets stop at 2022-12-31. Any feature derived from historical on/off labels is therefore **12–24 months stale at prediction time and near-constant across all 725 test cases**. This is the mechanism that made per-unit historical marginals backfire worst on Haukeli (−0.024), whose base rate collapsed from 0.87 to 0.14 within the training window.

This retires the reweighting / oversampling / historical-prior family as normally constructed. The surviving version is *conditional*: $E[\text{count} \mid \text{state deciles}]$ is a response surface rather than a base rate, and transfers under the weaker assumption that the physical relationship is stable even when the marginal is not.

It also means the walk-forward protocol of Section B.27 is the structurally correct one and a leave-one-out variant would be optimistic — it would let the model interpolate across a held-out block using data from both sides, which the submission can never do. We had provisionally planned to switch to leave-one-out; that plan is dropped for this reason.

Unused data

Two sources are present and under-used. `Historical_inflow` covers **1958–2025**, but every seasonal-percentile feature computes its reference distribution from the case dates (2015 onward) — roughly eight years of a 67-year record. And the two constraint files are joined as raw series but never resolved into *which unit is obliged to pass the water*, which is the form in which a minimum-flow obligation would explain always-on behaviour for a specific unit.

What this implies

Four changes, implemented in `src/count_model.py` and tested in Section B.29:

A. Width. Resolve the per-plant blocks into own/upstream/*downstream* (the last was missing entirely), and convert the remaining cross-plant information into the unit's *rank*, *z-score* and *share* among plants — unit-specific and scale-free where the absolute columns were shared and drifting. Section B.22 found relative+absolute beat relative alone, so we test *reduced* absolute rather than none.

B. Head. Classify over six count bins and forward the whole distribution.

C. Stationarity. Split drifting levels into shape + quarantined level.

D. Groups. Structural (never label-derived) group attributes, since unit count distributions sort by merit order within a plant.

B.29 2026-09-22 — Testing the four structural fixes (A/B/C/D)

What we did

Section B.28 diagnosed four structural problems and proposed a fix for each. This entry tests all four, isolated and in combination, against the standing best (`compact_distilled`, 0.9402 on

validation year 2021). Implementation is `src/count_model.py`; the ablation is `/tmp/abcd.py`.

Count predictions are fed to the pointwise model *in-sample* on training rows, matching the established configuration rather than cross-fitting — the control arm reproduces the standing best exactly (`MAE 24.21h | micro 0.9402`), which confirms the harness is measuring the same thing as Section B.25.

arm	configuration	feats	count MAE	micro-AUC
`base`	v2 437, 3-class head (control)	437	24.21	0.9402
`B`	v2 437, **binned head**	437	25.65	**0.9417**
`A_none`	v3 unit-resolved, no absolute	230	35.27	0.9118
`A_abs30`	v3 + top-30 absolute	260	24.80	0.9364
`A_abs80`	v3 + top-80 absolute	310	24.58	0.9395
`AB`	`A_abs80` + binned head	310	25.12	0.9412
`ABC`	+ stationarised levels	308	25.89	0.9403
`ABC_noD`	`ABC` minus group attributes	303	25.79	0.9391

A — REJECTED, and the diagnosis that motivated it was over-read

Finding. N

arrowing the count model's input is monotonically harmful. Absolute per-plant columns are worth more the more of them you keep: 230 features → 0.9118, 260 → 0.9364, 310 → 0.9395, 437 → 0.9402. There is no compact sweet spot. Removing the shared block entirely costs −0.0285 and blows count MAE out from 24.2h to 35.3h.

Section B.28 measured, correctly, that 377 of 437 features are identical across the 14 units of a case and carry 72.6% of model importance. The *inference* drawn from that measurement — that the shared columns crowd out unit-specific signal and should be replaced by unit-relative resolutions — is wrong. Those columns carry genuine case-level information about how much water the whole system is moving, and the unit-resolved rank/z-score/share features built to replace them do not substitute for it. A high shared-importance fraction is not by itself evidence of a pathology.

This also re-confirms Section B.22, which found relative + absolute beat relative alone. That result was assumed to be an artefact of the pre-compact architecture; it is not. `src/count_model.py:build_v3` is kept as the negative result, with `cross_plant_position` available should a future variant want scale-free cross-sectional features as an *addition* rather than a replacement.

B — the only positive, and it works through the distribution, not the mean

The binned head replaces the 3-class classifier plus mixed-count regressor with a single 6-class classifier over $\{0\}$, $[1, 29)$, $[29, 85)$, $[85, 141)$, $[141, 168)$, $\{168\}$, forwarding `p_off`, `p_on`, `p_low`, `p_high` and `exp_count`. It eliminates the second model entirely.

Finding. T

he binned head *worsens* count MAE (24.21h → 25.65h) while *improving* micro-AUC (+0.0015), and does so with a consistent sign on two different feature sets (v2: +0.0015; v3+abs80: +0.0017). The gain comes from forwarding the distribution's **shape**, not from a better point estimate — exactly the mechanism Section B.28 predicted from the bimodality of mixed weeks.

Finding. [

Methodological] **Count MAE is a partly misleading optimisation target.** It scores only the point estimate and moved opposite to micro-AUC here. We have steered on it since Section B.17. Every historical MAE-based decision is therefore suspect in the same direction, and future screening should read both.

At +0.0015 against a fold SD of 0.0128 this is not yet a result. It is a consistent sign worth confirming under walk-forward, nothing more.

C and D — inconclusive, because they were chained to the losing branch

Both were tested on top of `A_abs80`, which is itself −0.0007 below the control. That was a sequencing error in the ablation script: C and D should have been stacked on the best overall arm (`B` on v2), not on the best A variant.

- **C (stationarised levels)**: −0.0009 (`AB` 0.9412 → `ABC` 0.9403). Replacing 47 drifting level features with 45 ratios did not help in-period. Note that an in-period test is structurally unable to reward stationarity: the benefit, if any, is out-of-period robustness in 2023–24, which validation on 2021 cannot see.
- **D (structural group attributes)**: +0.0012 (`ABC_noD` 0.9391 → `ABC` 0.9403) — comparable in size to B, from five columns.

Neither verdict should be treated as final. D in particular is cheap and looks as promising as B.

Per-unit: one of the four broken units moved

unit	base	B	AB	ABC
Lio_G1	0.8220	0.8157	0.8215	0.8179
Haukeli_G1	0.8770	0.8789	0.8656	0.8759
Hogga_G1	0.8842	0.8878	0.8926	0.8882
Songa_G1	0.9345	0.9350	**0.9460**	**0.9471**

Songa gains +0.0126 under `AB`/`ABC` — the first movement on any of the four chronically weak units in five entries. Songa is tunnel-fed, the case where the water-balance diagnosis of Section B.26 was confirmed but the fix failed; it is plausible that the binned head helps because Songa's mixed-week distribution is strongly U-shaped (0.432 below 56h, 0.381 above 112h). **Lio is untouched by all eight arms**, sitting at 0.82 throughout.

What this implies

Finding. A

cross eight arms the spread is 0.9118–0.9417 and the best beats the control by +0.0015. A, B, C and D are all **rearrangements of information we already have**, and this is the seventh consecutive architecture-shaped intervention to return noise or harm. The evidence that the current information set is saturated is now strong enough to stop testing architecture.

Recall also that the target of 0.98 sits *above* the count-oracle of 0.9765 (Section B.17): even perfect counts under spread-rank allocation cannot reach it. The remaining gap needs new information, not a better arrangement of the current features.

Next, therefore, is the unused data identified in Section B.28:

1. **67-year inflow climatology.** `Historical_inflow` covers 1958–2025; every seasonal-percentile feature currently builds its reference distribution from the case dates (2015 onward), using roughly eight years of a 67-year record.

2. **Min-flow resolved to the obligated unit.** Both constraint files are joined as raw series and never mapped to which unit is required to pass the water — the natural shape of an always-on explanation for a specific unit.

Two secondary items carried forward: re-test D (and C) on top of `B`-on-v2 rather than the A branch, and re-instantiate the screening harness (`src/screening.py`) so an eight-arm ablation costs minutes rather than the 12 this one took.

B.30 2026-09-22 — Commitments are blocks: hour-level temporal structure

What we did

Section B.29 concluded that the current information set was saturated and that further architecture work was not worth doing. Looking instead for information we had never given the model, we checked what the pointwise model can actually see at the hour level. It receives 15 engineered features, and every one of them is price-versus-water-value economics (`f05`, `f06`, `f07`, `f10`, `f14`, `f15`). There is no hour-of-day, no lag, no persistence, no window aggregate: each of the 168 hours is scored as an independent draw.

Finding — the independence assumption is wrong by an order of magnitude

Finding. C

ommitments are contiguous blocks, not scattered hours.

$$P(\text{on at } t \mid \text{on at } t{-}1) = 0.995 \qquad P(\text{on at } t \mid \text{off at } t{-}1) = 0.007$$

Units switch state **0.5–3.6 times per 168-hour week**, against 9.3–27.2 if the same hours were scattered at random — roughly **9.5x more contiguous than chance**. Each unit-week contains 1.2–2.7 contiguous ON blocks.

The start costs in the topology are the obvious cause: the optimiser will not pay to cycle a unit hourly. The effective dimensionality of a unit-week is therefore not 168 bits but roughly a count plus two or three changepoints — and we had been modelling it as 168 independent decisions.

The obvious fix is wrong

If commitments are blocks, allocate each unit's c hours as one contiguous window. Tested against the current rule (the c highest-spread hours), both given ORACLE counts:

allocation rule (oracle counts, 2021)	micro-AUC
top-c spread hours (current)	**0.9485**
best contiguous c-window	0.9057

Finding. I

mposing a single contiguous window is **worse** than ranking hours by spread, by −0.0428. 1.2–2.7 blocks is not 1 block, and a one-window shape is too rigid. Persistence is real but does not license a hard allocation rule.

(This oracle is built differently from the 0.9765 of Section B.17 — hard 0/1 on 2021 using system spread for every unit — so the two figures are not comparable. Only the two rows above are comparable to each other.)

What did work: the same structure as features

`src/hourly_structure.py` gives the model, per hour and always within a case: centred window means at $w \in \{3, 6, 12, 24, 48, 96\}$ with their ranks and best-window membership; run structure (position within, and length of, the current above-median stretch); distance to the week's peak and trough; cumulative share of the week's positive spread; lags and leads at 1, 3 and 24 hours as differences; and hour-of-day and day-of-week cyclically encoded. All are scale-free or differenced against the case's own level, so none reintroduces the drifting price level of Section B.28.

arm	feats	micro-AUC	Lio
base (15 pruned + 84 count)	99	0.9417	0.8157
+ hourly structure	135	**0.9442**	0.8190
+ count-coupled windows	127	0.9418	0.8180
+ both	163	0.9430	0.8181

Finding. H

our-level temporal structure is worth +0.0025 (0.9417 → 0.9442), the first confirmed improvement since Section B.25 and larger than A, B, C and D combined. The gain is in the generic window and run features; *coupling* the window length to the predicted count adds nothing (+0.0001), so the value is in knowing that hours come in stretches, not in knowing how long this unit's stretch will be.

Per-unit: Vinje_G3 +0.014, Haukeli +0.005, Hogga +0.004, Lio +0.003, Songa −0.009.

Screening harness, re-instantiated

`src/screening.py` restores the convention of Section B.37: every third training case and 150 trees, used to rank candidates, never to score them. Strided by CASE rather than by row, because dropping rows within a case would leave that case's near-duplicate hours in the training set — the leak of Section B.16.

On this experiment it predicted +0.0034 where the full fit gave +0.0025: right sign, right magnitude, 18s against 108s. Screening scores are biased low (base 0.9365 screened vs 0.9417 full) and are labelled as such wherever they appear.

Negative results from the same session

Two families built from data identified as unused in Section B.28, both rejected:

- **67-year inflow climatology** (`exogenous.inflow_climatology`, 171 features). Day-of-year reference distributions from the full 1958–2025 record instead of the ∼8 years of case dates. Micro 0.9415 vs 0.9417 base; Lio +0.005.
- **Minimum-flow obligation** (`exogenous.minflow_obligation`, 99 features). Resolved to the plant that must satisfy each constraint, converted to a forced capacity factor. Micro 0.9392. **Killed by arithmetic before the model saw it**: Lio at minimum load passes 4.82 m^3/s against a Byrtevatn obligation of 0.07 (max 0.20), so `mfo_forced_cf_Lio` never exceeds 0.042 and the constraint never binds. A physically sound hypothesis at the wrong order of magnitude.

Also rejected: **per-unit probability calibration**. Lio is the largest single drag on the pooled score (removing it lifts micro by +0.0077; an oracle Lio is worth +0.0146) and is over-predicted on average (mean 0.612 against a base rate of 0.545), which suggests a logit offset. It fails, and instructively:

	micro-AUC
uncalibrated	**0.9417**
oracle per-unit calibration (true 2021 base rates)	0.9346
legal, training base rates	0.9363
legal, Lio only	0.9288

Finding. E

ven *oracle* calibration hurts. A per-unit logit shift is uniform across cases, so it cannot fix within-case ordering, and it disturbs cross-unit interleaving that was already roughly right. A monotone per-unit transform also leaves own-AUC exactly unchanged (Lio 0.8157 → 0.8157). **Lio cannot be fixed by post-processing; only by better discrimination.** Same family of error as the rank-transform failure of Section B.16.

What this implies

The count stage contributes +0.106 (pointwise alone 0.8346 → 0.9402 with counts); everything else combined contributes an order of magnitude less. The count model has been doing nearly all the work, and the pointwise model has been little more than an allocator with 15 features. Temporal structure is the first evidence that the allocation half has real headroom.

Next is the long-format restructure (`src/longform.py`): one row per (case, hour, unit), which removes shared splits, collapses the 84-column count block to 6 `own_` columns, and makes per-unit weighting possible for the first time. Its main risk is losing the cross-unit interaction a wide row carries for free, which `longform.cross_unit_context` attempts to restore as relative quantities.

B.31 2026-09-22 — Long format, and two topology bugs

What we did

Every pointwise model in this project has been *wide*: one row per (case, hour) with 14 outputs under `multi_strategy="multi_output_tree"`. That shape was never chosen on evidence, and it forces three compromises simultaneously:

1. **Shared splits.** A vector-leaf tree picks one split per node and applies it to all 14 outputs, so the model cannot specialise. Never measured.
2. **A 14x-wider count block than necessary.** Forwarding 6 numbers per unit costs 84 columns (`cnt_p_on_Lio_G1`, `cnt_p_on_Songa_G1`, ...). For Lio's head to use Lio's own count, the tree must split on Lio's column — and that split then applies to the other 13 heads, where it is noise. Section B.28 measured the symptom; Section B.29 showed the obvious fix was wrong.
3. **No per-unit weighting.** `sample_weight` applies to a ROW, and a wide row carries all 14 units, so a unit cannot be weighted at all.

`src/longform.py` produces one row per (case, hour, unit) — 6.9M rows at full stride. Micro-average ROC-AUC pools all (case, hour, unit) cells into one ranking, so in long format the metric is simply `roc_auc_score(y, p)`: **the objective is native to this shape** and awkward in the wide one.

Results

arm	configuration	feats	micro-AUC	Lio
—	wide best (Section B.30)	135	0.9442	0.8190
L1	long, 15 shared + 6 `own_`, no unit id	29	0.9396	0.8102
L2	+ unit identity (categorical)	30	0.9399	0.8090
L3	+ cross-unit context	33	0.9421	0.8272
L4	+ hourly structure	69	0.9456	0.8276
L5	+ top-120 count features per unit	189	0.9490	0.8337
L6	+ Lio rows weighted 3x	189	0.9479	0.8297
L7	L5 at full stride (5.2M rows)	189	**0.9499**	**0.8380**

L1–L6 train on every third case; only L7 uses all of them.

Finding. L

ong format reaches **0.9499**, against 0.9442 for the best wide model. The shape alone is not the win — L1 is *below* the wide baseline. The win comes from what the shape makes affordable: cross-unit context (+0.0022), hourly structure (+0.0035) and per-unit count features (+0.0034), none of which fit in the wide layout at acceptable cost.

Finding.

Count features at the hour level work in long format and failed in wide format (Section B.24: 0.9292 raw-forwarded vs 0.9402 distilled). The idea was sound; the wide layout made each feature cost 14 columns, so 120 features meant 1,680 columns of mostly noise. In long format each costs one column and lands on the row it belongs to.

Finding.

Per-unit weighting fails even where it is finally possible. L6 weights Lio's rows 3x and loses −0.0011 overall *and* −0.0040 on Lio itself. Third independent rejection of the reweighting family, after historical marginals (Section B.28) and per-unit calibration (Section B.30).

Per-unit: the broken units finally move

unit	wide base	L7	Δ
Haukeli_G1	0.8789	**0.9082**	+0.029
Lio_G1	0.8157	**0.8380**	+0.022
Hogga_G1	0.8878	**0.9076**	+0.020
Vinje_G1	0.9517	0.9661	+0.014
Songa_G1	0.9350	**0.9059**	**−0.029**

Three of the four chronically weak units move substantially, for the first time in the project. Songa moves the other way, and that turned out to be diagnostic.

Two topology bugs, found by asking why Songa regressed

In wide format Songa's head could lean on other units' columns. In long format its row carries `own_*` features and nothing else — so if those are missing, Songa has nothing. They are missing.

Finding.

plant_graph matched only DIRECT reservoir→plant edges, but Songa is fed through tunnels (`Songavatn` → `Songavatn_Songa` → `Songa`, and likewise from `Bitdalsvatn`). So `src["Songa"]`

was `None`, every `own_*` feature for Songa was NaN, and `mwh_per_m3("Songa", .)` returned NaN — while Songavatn carries the **largest storage variance of any reservoir in the system** (sd 150) and Bitdalsvatn adds another (sd 25). Section B.26 diagnosed Songa as "tunnel-fed" and attacked it with water-balance corrections; the defect was in the graph traversal.

Finding. T

hree reservoir-to-reservoir TUNNEL links were invisible to the same traversal: `Totak→Vaamarvatn`, `Bordalsvatn→Foersvatn`, `Venemo→Totak`. The first breaks the main cascade: Songa discharges into Totak, which tunnels into Vaamarvatn, which is Vinje's source. **Vinje's upstream plant set was empty and should contain Songa.** The second hides Bordalsvatn (sd 43) from Kjela.

Fixed in `production_features`: `_through_tunnels`, `plant_sources`, `reservoir_tunnels` and `effective_upstream_plants`. `plant_graph` keeps its single-source contract for existing callers but now resolves through tunnels, taking the largest-capacity reservoir as primary where a plant has several.

A third finding, from the same question

Checking whether other units share Songa's problem turned up a different one:

reservoir	plant	distinct values	sd	cv
Vatjern	**Haukeli**	**1**	0.000	0.000
Hyljelihyl	—	**1**	0.000	0.000
Bandak	**Hogga**	136	8.961	0.068
Vinjevatn	Tokke	201	1.134	0.177
Byrtevatn	Lio	1369	20.118	0.367
Foersvatn	Kjela	2265	33.618	0.470

Finding.

Haukeli's source reservoir is a literal constant — 0.150 for all 3,654 days. The post-decision storage trace, the single largest source of signal in the project (Section B.21), *does not exist* for Haukeli. Hogga's Bandak has 136 distinct values in ten years. The "four broken units" were never one problem: Songa was a graph bug, Haukeli and Hogga are missing data, and **Lio is neither** — Byrtevatn varies normally, so Lio's difficulty remains unexplained.

What this implies

The standing figure is **0.9499** on validation year 2021, from 0.9402 at the start of the session. The topology fix is re-running: it predicts specifically that Songa should recover from 0.9059, since it gains two real reservoirs where it previously had NaN. That prediction is falsifiable and worth recording before the result arrives.

Caveat: the fix changes features feeding *both* stages, and every experiment to date fitted the count model around Songa's NaNs. The net effect could go either way.

Still open: Lio, unexplained and worth +0.0146 alone; whether any of this survives walk-forward; and the submission pipeline, which has never been built.

B.32 2026-09-23 — Reading the documentation: an off-by-one, a mispriced spread, and a permanent ceiling

What we did

After Section B.31 the remaining weak unit was Lio, and Haukeli and Hogga were still well below the rest. Rather than continue guessing at features, we read `Dataset_definitions_and_explanation.pdf` properly. It settled three things, two of which were defects in our own pipeline.

Finding 1 — Haukeli's storage is a documented placeholder, not data

> "Note that because there are no historical records for two small reservoirs, **Hyljelihyl and Vatjern**, their storage is assumed to be **50% full**."

Finding. V

atjern is Haukeli's source reservoir, and its storage series is a constant 0.150 across all 3,654 days. The post-decision storage trace — the single largest source of signal in this project (Section B.21, +0.074) — **does not exist for Haukeli and never will**. No feature engineering recovers a measurement that was never taken. Hogga is the milder case: Bandak has 136 distinct values in ten years, $cv = 0.068$.

This closes a line of work rather than opening one. It also retrospectively explains Section B.26, where a water-balance correction for Haukeli was built, confirmed as diagnostically right, and still failed: the balance was being computed from a constant.

Finding 2 — the terminal water value was read one day late

> "For a scheduling problem spanning one week, for example, starting at 2015-01-01 00:00 and ending at 2015-01-07 23:00, the initial volume of a reservoir is the value at 2015-01-01 00:00, and its **final water value should be the value at 2015-01-08 00:00**."

That is start + 7 days. `stage_terminal_wv` used `pd.Timedelta(days=8)`. Every case in the project priced its terminal water against the wrong day, in a feature (`f07_price_minus_terminal_wv`) that survived pruning into the final fifteen.

The correction alone is small — $\mathrm{corr}(d_7, d_8) = 0.996$, the water-value series being smooth — and on the user's suggestion both horizons are now emitted, along with lags at 6 and 9 days, so the model arbitrates rather than the documentation. The genuinely new quantity is `f07_terminal_slope` $= wv_{d8} - wv_{d6}$ (sd 1.86 EUR/MWh): the local slope of the water value across the decision boundary, i.e. whether water is getting scarcer exactly where the optimiser stops caring.

Finding 3 — the spread family prices against the wrong quantity

SHOP maximises revenue plus the salvage value of water left in the reservoir, so the marginal cost of generating is the *terminal* water value. The daily `Synthetic_water_value` series is an *input* to the problem, not the price its decisions are taken against.

Finding.

f06_spread_*, the family that dominates feature importance across the whole project, is `price − daily_wv`. The economically correct quantity is `price − terminal_wv`. Per-plant terminal

spreads (`f07_term_spread_<Plant>`) now exist, so each plant is priced against its own reservoir's horizon value rather than a basin-wide daily average.

Measured effect

	micro-AUC
long format, tunnel topology fixed (L5, stride 3)	0.9545
+ terminal-wv horizon, lags, slope, per-plant spreads	**0.9604**

Finding. T

he water-value work is worth +0.0059, comparable to the tunnel-topology fix (+0.0067, Section B.31). Both were defects rather than innovations.

Caveat: this is an across-script comparison at matched stride, year and parameters, but with 195 features against 189 — directionally solid, not a controlled isolate. The topology figure IS clean, because `longtest.py` reads the pre-built feature parquet and so used the old `days=8` columns while picking up the live topology fix.

What this implies

Two of the three findings were bugs in our own code, found by reading a three-page PDF that had been in the repository since the first session. Together they are worth +0.013 — more than every feature family invented in the same period combined. The lesson is recorded in Section B.34, where twelve consecutive feature additions failed.

B.33 2026-09-23 — Hyperparameters, and what effective sample size means in two shapes

What we did

Neither model had been tuned. The pointwise model's settings were inherited from the wide architecture; the count model's were unchanged since Section B.17, surviving a 3-class → 6-class head change, growth to 462 features, and the entire long-format rebuild.

Coordinate descent, one knob at a time, validation year 2021, stride 3 held constant. Not a final tuning phase — a 20-minute probe to find anything badly mis-set.

The pointwise model: +0.0028

knob	from → to	micro-AUC
baseline	—	0.9604
`min_child_weight`	1 → 500	**0.9622**
`max_depth`	8 → 10 / 12	0.9621 / 0.9620 (no gain)
capacity	300×0.10 → 600×0.05	**0.9632**
`colsample_bytree`	0.8 → 0.5 / 0.3	0.9625 / 0.9626 (no gain)

Finding.

min_child_weight was the one badly mis-set knob, and it was predictable from the data rather than from a grid. The long-format table has 5.2M rows, but $P(\text{on} \mid \text{on}_{t-1}) = 0.995$ and each unit-week holds 1–3 contiguous blocks (Section B.30), so the independent information is roughly 40k–120k blocks. A leaf minimum of 1 lets a split form from rows that are all the same block of the same unit-week — fitting autocorrelation. Raising it to 500 is worth +0.0018.

Depth not helping is worth recording as a wrong prediction: the argument was that long format exists to allow per-unit specialisation and needs depth to pay for it. It does not, because the `cf_*` features are already unit-specific, so the tree never has to split on unit first to reach unit-specific structure.

The count model: nothing beats the original settings

Screened on multi-class **log-loss**, not count MAE. Section B.29 established MAE moves opposite to micro-AUC because it scores only the collapsed point estimate, while what is forwarded is the whole distribution; log-loss is a proper scoring rule for that distribution.

	log-loss	count MAE
current (400×0.05, d5, mcw 1, cs 0.6)	**1.0989**	23.48h
`min_child_weight` 10 / 50 / 200	1.1203 / 1.1087 / 1.1517	
`max_depth` 4 / 6 / **8**	1.1207 / 1.1214 / **1.1983**	**20.69h**
800×0.03 / 1200×0.02	1.1091 / 1.0994	

Finding. N

o parameter beats the settings chosen in Section B.17. Downstream confirmation: old params 0.9632, "tuned" params 0.9632 — byte-identical, because the search returned the starting point.

Finding. [

The clearest demonstration of the MAE trap] `max_depth=8` gives the **best count MAE (20.69h) and the worst log-loss (1.1983)**. Optimising MAE would have selected precisely the wrong model. Every MAE-guided decision from Section B.17 onward is suspect in this direction.

Finding. [

Same principle, opposite answers] `min_child_weight` is best at **500** for the pointwise model and **1** for the count model. Not a contradiction: count rows are whole unit-weeks and genuinely independent, so effective $n \approx$ nominal n and small leaves are legitimate. Pointwise rows are hourly and autocorrelated at 0.995, so effective n is $\sim$50$\times$ below nominal. The knob should be set from the correlation structure of the rows, not from a default.

The forwarded-feature budget is also already right

The count model's top 120 features by gain are attached per unit to the pointwise model as `cf_*`. The 120 was chosen arbitrarily when the architecture was built, and it is a *fixed budget*, so adding features to the count model reallocates rather than accumulates.

forwarded budget	micro-AUC
top-120	**0.9632**
top-180	0.9609
top-200	0.9618

Finding. W

ider is worse. This also kills the hypothesis offered in Section B.34 for why the slope family hurt — that slopes crowded the budget and evicted useful features. Widening the budget does not rescue them, so they simply do not help most units.

Screening harness

`src/screening.py` restores the convention of Section B.37: every third training CASE (not row — striding rows would leave a case's near-duplicate hours in training, the leak of Section B.16) and 150 trees, used to rank and never to score. Calibrated on the hourly-structure experiment it predicted +0.0034 where the full fit gave +0.0025: right sign, right magnitude, 18s against 108s.

Stride 3 is now the standing iteration protocol for a second reason: at full stride the working set crosses this machine's 26GB and the run begins paging. Measured, L5 at 1.7M rows takes 53s and L7 at 5.2M rows takes 6,537s — **123× for 3× the data**, a cliff rather than a curve. Stride is held constant across arms, so deltas remain comparable while levels sit ∼0.002 below full stride.

B.34 2026-09-23 — Twelve negative results, and three methodological failures

Why this entry exists

Between Section B.31 and the present standing figure of 0.9632, twelve feature additions were built and tested. All twelve failed. 'report_rules.md' requires that negative results are kept rather than quietly dropped, and a run of this length is itself the finding: every gain in the same period came from *correcting a defect*, not from adding information.

The twelve

addition	where	micro vs 0.9632
287 slope features, bulk	count model	−0.0020
slopes, forwarded budget 180	count model	−0.0023
weak-unit weights (Lio 3×, Haukeli/Hogga 2×)	pointwise	−0.0016
slopes + weak-unit weights	both	−0.0040
nine slope themes, 7 features each	pointwise	−0.0003 to −0.0010
slope block masked to Haukeli only	pointwise	−0.0005
slope block masked to Haukeli + Hogga	pointwise	−0.0008
full 12-value count distribution	pointwise	−0.0008
neighbour counts (sibling/upstream/downstream)	pointwise	−0.0006
distribution + neighbours	pointwise	−0.0005
count-model hyperparameter search	count model	0.0000
forwarded budget 180 / 200	architecture	−0.0023 / −0.0014

Two are worth more than one line.

Slopes (`src/slopes.py`, 287 features). Rates of change of water value, storage, price, inflow and spread, at case-relative offsets. Motivated by the terminal-water-value slope being one of the largest single gains in the project (Section B.32), and by the fact that a derivative is a difference and therefore stationary by construction: **0 of 280 slope features drift more than 0.5 SD** across the 2021–22 regime break, against 14.1% of importance sitting on drifting levels (Section B.28). Net −0.0020, but **Haukeli** +0.0154, reproducibly, while eleven of fourteen units got worse. Mechanistically plausible — Haukeli's storage is a documented placeholder constant (Section B.32), so price and spread dynamics are the only dynamics it has.

Full distribution and neighbour counts. The head forwards `p_off`, `p_on`, `p_low`, `p_high`, `exp_count` — a summary designed for the WIDE model, where each forwarded number cost 14 columns. In long format a number costs one column, so the compression buys nothing and discards the shape that made the binned head work. Forwarding all six bins plus dispersion,

entropy and threshold masses: −0.0008. Giving each unit its hydraulic neighbours' predicted counts (`longform.neighbour_context`, using the `Songa→Vinje` edge that only exists after the tunnel fix): −0.0006.

Three methodological failures, recorded so they are not repeated

Finding. [

1. Rotating-block validation is optimistic by construction] A rotating two-month holdout was proposed and built (`src/validation.py`): eight blocks, one per year, all seasons, 2,847 training cases per fold against 2,373 under single-year holdout. It returned 0.9794 and 0.9652 on its first two folds — far above any temporal estimate — because it lets the model train on data from *both sides* of the held-out window. The test set is 2023–24, strictly after all training data: that is extrapolation, and rotating blocks measure interpolation. This is the same objection already raised against leave-one-out earlier in the project and then not applied to our own proposal. Abandoned; walk-forward remains the protocol. It was also infeasible — 10.4 hours per fold at full stride.

Finding. [

2. The slope themes were routed to the wrong stage] The bulk slope test merged slopes into `CT`, the COUNT model's input. The themed sweep and the masking test added them to the POINTWISE model only. All nine themes returned −0.0003 to −0.0010 with no trace of Haukeli's +0.0154 — not a refutation, a test of something else. The evidence was in the original output and was missed: count MAE moved 23.48 → 23.04h, which only the count model can cause. Re-running the themes on the count stage.

Finding. [

3. Two job-scheduling bugs cost real time] Jobs were chained by `pgrep` on process-name patterns. First a circular deadlock — each waiting shell's own command line contained the other's pattern, so two jobs waited on each other indefinitely and the group sweep never ran despite being reported as running. Then a thundering herd — three waiters watching the same job all started at once when it exited, contending for CPU and risking the swap thrashing documented in Section B.33. Replaced with a lock-directory queue (`/tmp/runq.sh`). Inferring job state from process lists is the error; a lock is the fix.

What this implies

Finding. A

cross the session, everything that worked was a correction and nothing that was invented helped: tunnel topology +0.0067, terminal water value +0.0059, long format +0.006, hourly structure +0.0025, pointwise hyperparameters +0.0028 — against twelve consecutive failed additions. The reasonable conclusion is that the current feature set is saturated for this architecture, and that further effort is better spent on validation and the submission than on a thirteenth family.

Standing figure: **0.9632** on held-out 2021 at stride 3, from 0.9402 at the start of the session. It rests on a *single validation year*; the last honest multi-fold number is the 0.9350 (SD 0.0128) of Section B.27, on a much older architecture. Walk-forward with the current configuration is the next task, and the submission pipeline — 725 test cases, never built — is the one after it.

B.35 2026-09-24 — First submission: 0.98058 on the held-out test set

What we submitted

`src/submit.py`, the lean long-format architecture at full stride:

rows	6,872,544 — one per (case, hour, unit), all 2,922 training cases
features	207 (15 pruned economic, 10 terminal-water-value, 36 hourly structure, 141 count-table features attached per unit, unit identity + 4 static)
model	single binary XGBoost, 600 rounds, depth 8, `min_child_weight` 500
count model	fitted, but **only to rank features** — its predictions are not forwarded

Finding.

Public leaderboard: 0.98058, against a walk-forward estimate of 0.9638 (SD 0.0115). The estimate was pessimistic by +0.017.

Why walk-forward understated the score

Three differences between the folds and the submission, identified before the result arrived (Section B.31 and the submission module's header), and now resolvable:

1. **Training volume.** Every fold trained on 1,826–2,557 cases; the submission trained on 2,922. Data volume has been the most reliable driver in the project — count MAE fell 29.9h → 18.1h across folds on volume alone (Section B.27).
2. **Period difficulty.** The six folds span 2020–2022, including the energy crisis. Per-case analysis (Section B.34) showed the hardest cases are exactly the high-price, negative-spread, high-switching weeks that period produced. The test window is 2023–24, after it.
3. **Extrapolation horizon.** Folds predict at most six months ahead; the submission predicts 24. This was flagged as the largest risk and *did not materialise*.

The best normal-period fold (2021-H1, 2,192 training cases) scored 0.9781. With 2,922 cases and a post-crisis period, 0.98 was reachable; what we could not do beforehand was size which force dominated.

Finding. [

On trusting this number] The test set is $725 \times 168 \times 14 = 1.7$M cells, so the public half is roughly 850k. Micro-average ROC-AUC over that many cells has very little sampling noise, so the public and private halves should agree closely. This is not a competition where the public board misleads.

What produced the result

Every gain that survived independent validation was a **correction or a simplification**, not an invention:

change	effect
Dropping the count model's predictions (Section B.34)	+0.0082
Tunnel-topology bug: Songa had no source reservoir (Section B.31)	+0.0067
Terminal water value read one day late (Section B.32)	+0.0059
Long format, one row per (case, hour, unit) (Section B.31)	+0.006
Hour-level temporal structure (Section B.30)	+0.0025
Pointwise hyperparameters (Section B.33)	+0.0028

Against that, **eighteen feature families were built and rejected**: slopes, economics, neighbour counts, full count distributions, per-unit masking, 70/30 unit-specific sets, 67-year inflow climatology, minimum-flow obligations, reweighting (five separate attempts), per-unit calibration, count-model tuning, wider and narrower forwarding budgets, block allocation, daily post-decision traces, and seasonal regime features.

Finding. [

The pattern worth carrying forward] Nothing invented helped; everything corrected did. Two of the three largest gains were defects in our own code — a graph traversal that missed tunnel-fed plants, and an off-by-one against the dataset documentation — found by reading a three-page PDF and by asking why one unit regressed.

A second pattern: validation-derived selection never transferred

Three feature-selection methods were tried, and every one that used validation data looked good on validation and failed on folds:

	single year 2021	walk-forward
count-model ranking (chosen without validation)	0.9651	**0.9638**
trimmed by permutation importance + unit-relative	**0.9671**	∼0.9609 (3 folds, all worse)
permutation top-60 (circular)	0.9668	not run
held-out inner-split ranking	0.9631	not run

Five independent random draws of 120 features spanned 0.9476–0.9619 (SD 0.0055), so selection is *not* irrelevant — an earlier claim that it was rested on a single lucky draw and was wrong. But the arbitrary ranking beat every deliberate one.

What remains

The second submission is unspent. Given eighteen rejected families, a selection method that resists improvement, and a score already at target, the case for spending it is weak unless a change is first confirmed on walk-forward folds. Optimising against a public board is how a good private score is given away.

Open, if work continues: Lio remains the weakest unit (0.8616 mean over folds, swinging 0.957 → 0.656 within 2021 on a regime change no feature predicted); predicted commitments switch 7.6 times a week against 3.0 in reality, so the persistence structure of Section B.30 is only partly captured; and the hardest 15% of cases remain the autumn, negative-spread, high-cycling weeks where economics do not determine the answer.

B.36 2026-10-27 — First principles: rebuild the optimiser, learn only its errors

Why

Every model so far learned the map from inputs to commitment from scratch. But the labels are not observations of a noisy process: they are the output of one deterministic optimiser (SHOP v17.17), and every input it used is in our data. The topology YAML is SHOP-ready (head–volume curves, turbine efficiency curves per reference head, generator efficiency, start/stop costs, friction losses, tunnels, spillways); price, inflow, initial storage, terminal water value and the min-flow/min-volume series cover every case, test included. So instead of approximating SHOP with features, rebuild the problem it solved and solve it ourselves.

What we built: `src/shoplite.py`

One linear programme per case, solved with HiGHS (simplex; building and solving take 8 s per case on one core on average over all 3,647 cases, 2–25 s for individual cases; SHOP's own recorded solve times are median 52 s, mean 147 s):

objective	$\max \sum_t \text{price}_t(\text{production}_t - \text{friction losses}_t) -$ start/stop $-$ river, bypass, spill costs $-$ constraint penalties $+ \sum_r \text{terminal value}_r \cdot V_{r,T}$
water	hourly balance over 17 reservoirs, the Vest-Vassdraget junction and the Songa tunnel junction
units	on/off u (binary in the MILP, $[0,1]$ in the LP), $p_{\min}$ at $q_{\min}$, concave PQ envelope at the case's initial head; start/stop variables with the YAML costs
losses	main-tunnel and shared-penstock friction as a convex piecewise-linear cubic in plant flow
constraints	reservoir bounds; min volume and min flow (soft, heavily penalised)

Simplifications against SHOP, each a place our solution can disagree with the labels: heads frozen at the initial levels (SHOP iterates on head), tunnels between reservoirs are lossless links, and the units' initial on/off state is unknown, so hour 0 carries no start cost. We checked whether the initial state could be read off the previous case: hour 0 agrees with the previous case's hour 24 only 80–96% of the time, no more than any two overlapping cases agree, so it is not chained.

Three conventions were measured from the labels, not assumed

Terminal water value. The file is in EUR/MWh; how SHOP converted it to EUR/Mm^3 is undocumented. In weeks where a unit switches, its on-hour and off-hour prices bracket the price threshold SHOP used. Regressing that implied threshold on terminal water values:

plant	R^2 (upstream WV only)	coef. upstream	coef. downstream
Byrte	0.99	0.98	−0.01
Vesle Kjela	0.97	0.95	0.04
Kjela	0.96	0.89	0.15
Songa	0.95	0.83	0.11

Finding. Each plant's break-even price is the water value of the reservoir feeding it, and the downstream value does not enter. The documented SHOP conversion (EUR/MWh $\times$ the full-cascade conversion factor) predicts a large negative downstream coefficient and is refuted. The convention that reproduces the labels values one Mm^3 in reservoir r as $\sum_k \text{WV}(\text{feeding reservoir of } k) \cdot \text{conv}_k$ over the plants k on its way to the sea.

River flow cost. `flow_cost_curve` on the `r_` rivers read as a marginal cost per (m^3/s·h) charged Kjela's discharge 33 EUR on its way through `r_Hyljelihyl`; the LP duals showed exactly that gap. Read as total cost per hour (marginal = slope), Kjela's raw AUC went 0.70 $\rightarrow$ 0.96 and Haukeli's 0.93 $\rightarrow$ 0.99 (its water also arrives through a costed river).

Sibling tie-break. Identical units are chosen in a fixed order: one Tokke unit on is G1 99.1% of the time, two are G1+G3 94.3%, three G1+G3+G4 66% (else G1+G2+G3); Vinje is G1, then G1+G2 (100%). Stable in every year 2015–2022. Encoded as ordering constraints on u.

Raw micro-AUC of the LP solution on a 59-case sample across 2015–2022, with no learning at all: 0.937 (first version) $\rightarrow$ 0.963 (river cost) $\rightarrow$ 0.967 (sibling order).

The MILP is a worse feature than the LP

Solving with binary u gave 0.941 against the LP's 0.967 on the same sample. A binary schedule discards the LP's graded information about how close each hour is to the margin, and any small model error flips a whole block. It also exposed a genuine gap: SHOP keeps Lio on all week to meet the Vest-Vassdraget minimum flow (5–8 m^3/s against ∼1.5 natural inflow) even when price is below Byrtevatn's water value, while our MILP pulses Lio one hour in six and covers the rest by bypassing Vinjevatn, whose water it values near zero because the small reservoir is about to overflow. Left to the learned correction.

The learned correction: `src/physics_features.py`, `src/physics_model.py`

LightGBM on one row per (case, hour, unit), 61 features: the LP solution and its temporal context (`lp_`: commitment, power, flow, local water value from the duals and the margin price − local water value, rolling windows, run structure, plant-level count, reservoir trajectories), economics from the inputs (`ec_`), the constraint series (`cn_`), the observed storage trajectory (`st_`, post-decision, available for test) and static unit attributes (`s_`). 600 rounds, 127 leaves, `min_data_in_leaf` 500.

Results: walk-forward, six half-year folds, stride 3

The submitted XGBoost model was re-run on the same folds with `submit.py`'s own building blocks (`src/baseline_walkforward.py`; the count model that ranks features is fitted on each fold's training weeks only). It reproduces log 036's 0.9638 to within 0.002.

fold	submitted XGBoost	raw LP (no learning)	LP + LightGBM	Δ
2020-H1	0.9535	0.9720	**0.9925**	+0.0391
2020-H2	0.9455	0.9671	**0.9866**	+0.0411
2021-H1	0.9765	0.9787	**0.9950**	+0.0185
2021-H2	0.9504	0.9695	**0.9914**	+0.0410
2022-H1	0.9672	0.9586	**0.9862**	+0.0190
2022-H2	0.9784	0.9802	**0.9953**	+0.0169
mean (SD)	0.9619 (0.0140)	0.9710 (0.0079)	**0.9912 (0.0040)**	+0.0293

Finding. The optimiser alone, with no training data, beats the best learned model of the project on five of six folds (mean 0.9710 vs 0.9619). With a learned correction on top, every fold is above 0.986, the mean rises by +0.029 to 0.9912, and the fold SD falls from 0.0140 to 0.0040. All 14 units improve; the two the project never fixed move most — Lio 0.864 → 0.979, Haukeli 0.920 → 0.973.

Gain shares in the last fold: `lp_` 86%, static 6.5%, economics 5.9%, observed storage trace 1.7%, constraints 0.2%. The model leans on the optimiser, not on the post-decision trace that carried the previous architecture (log 022).

Ablation: where the gain comes from

Same folds, same LightGBM, one feature group removed at a time (`python physics_model.py ablation 3`):

arm	mean micro-AUC	test-like folds
full (61 features)	0.9912	0.9934
without the observed storage trace (`st_`)	0.9910	0.9935
optimiser features and unit identity only	0.9897	0.9925
without the optimiser (`lp_`)	**0.9483**	0.9502

Finding. The optimiser is the result. The same learner on the same economic, constraint and storage features but without the LP scores 0.9483 — below the previous XGBoost model — so the gain is not LightGBM and not feature engineering. And the post-decision storage trace adds nothing (0.9910 without it): this is the project's first model that uses only information available before the week starts. The previous causal-only track stood at 0.8548 (log 022, single-year 2021 protocol, so not directly comparable).

Which validation period looks like the test set

Measured rather than assumed (`src/adversarial.py`): a classifier separating each period's weeks from the 2023–24 test weeks on case-level inputs (price level, volatility, negative-price share, water value, spread, fill, inflow). AUC near 0.5 means indistinguishable.

period	separability	mean price	price SD	spread to WV
2015–2020	0.99–1.00	9–43	2–5	≥ 0
2021	**0.64**	76	16	−16
2022	0.93	211	50	−84
test 2023–24	—	65	22	−9

By half-year, 2021-H2 is closest (0.785), then 2022-H2 (0.879); every other half-year is ≥ 0.95. This confirms log 020's choice of 2021 and refutes "2022 resembles the test period". On the two test-like folds the physics model averages 0.9934 against 0.9644. The test period also has negative prices in 2.2% of hours, which training barely contains; the optimiser handles them from first principles (a unit is not worth running at a negative price) rather than by extrapolation.

Caveats

- The three conventions above were chosen from label statistics over 2015–2022, which includes the validation folds. They are three discrete structural choices, each stable across years, so the room for overfitting is small — but the protocol is not blind to them.
- Both walk-forwards train on every third case (stride 3, the project's experiment convention); the submission trains on all 2,922.

Submission

`submission_physics.csv`: LP + LightGBM trained on all 2,922 labelled cases. Format checked against `sample_submission.csv`; agrees with `submission.csv` on 92.9% of cells at a 0.5 threshold (correlation 0.92). Uploaded after this entry was written: it scores above the first submission's 0.98058 on the public leaderboard.

Open

Fidelity is now the lever, not features: Lio's min-flow behaviour, SHOP running units below their water value in some weeks (2019-07-09: Tokke, Vinje, Lio and Hogga on at price 35.6 against water values 40–43), Hogga's timing in weeks where Bandak must fill (degenerate in our LP), and heads frozen at their initial levels. Each improvement to the replica feeds the learned layer directly.

B.37 2026-09-21 — Rolling-origin CV harness, and the feature backlog

Why.

Before a week of one-feature-at-a-time iteration, the scoreboard has to be able to resolve a feature-sized effect. Single-year validation could not: the $n_{\text{estimators}}$ comparison in Section B.7

spanned ∼0.006 AUC and we correctly judged it noise, but we had no way to say how much noise. Implemented in `src/modeling/evaluation.py`.

Protocol.

For each validation year $Y \in \{2019, 2020, 2021, 2022\}$: train on every case starting before Y, validate on cases starting in Y, score with `kaggle_metric.py`. Reports per-fold scores plus mean and spread. Model configuration is the adopted default from Section B.7 ($n_{\text{estimators}} = 300$, multi-output tree).

Baseline result (current raw 54-feature set).

Table 27: Rolling-origin CV, raw-feature baseline.

Validation year	micro-AUC	fit cases	val cases
2019	0.7987	1,461	365
2020	0.6786	1,826	366
2021	0.6663	2,192	365
2022	0.6858	2,557	365
Mean	**0.7074**		
SD	**0.0614**		

Finding. Fold-to-fold standard deviation is **0.0614** — roughly **ten times larger** than the ∼0.006 differences we were weighing between tree counts in Section B.7. 2019 (0.799) is a large outlier against 2020–2022 (0.666–0.686). Single-year validation cannot resolve feature-sized effects, and any conclusion drawn from one holdout year — including our earlier per-unit comparisons — carries far more uncertainty than the numbers suggested.

Finding. Methodological consequence: judge features on *paired per-fold deltas*, not on the mean. Folds differ enormously in intrinsic difficulty, so the mean's spread is dominated by which year is being predicted, not by the feature. A feature that improves all four folds by +0.004 each is credible; one that moves the mean by +0.02 via a single fold is not. Record per-fold deltas and the sign pattern for every experiment.

Feature backlog.

`report/feature_backlog.md` holds the candidate list: 20 topics, 5–8 concrete features each (∼120 candidates), each tagged with the EDA round that motivates it and a status field. It is ordered at the end by expected value over effort. The highest-evidence entries are the case-relative price normalisations (Section B.9, finding 1), the raw price − water-value spread (Section B.8, finding 7), and the terminal water value (Section B.9, finding 6).

One structural caveat carried forward.

Every transformation rule from Section B.9 was established against *univariate correlation with the target*, which is a screening tool, not a guarantee: correlation does not account for what the other 53 features already supply. The CV harness is the actual arbiter, and some well-evidenced candidates will still fail to add anything once tested.